Computer Architecture

Exam Study Guide

A complete, teaching-first revision of the full syllabus — performance & Amdahl, ISA and dependences, pipelining and hazards, branch prediction, Tomasulo and speculation, cache and virtual memory, cache coherence, multiprocessors, and vector/SIMD/GPU — with every previous-year question solved step by step.

Source materials analysed	Coverage
Computer Architecture.pdf — 87 pages of lecture notes	Every topic, formula, diagram and worked example
ca1.png … ca41.png — 41 scanned past-paper questions	100% of questions solved, topic-wise, with full reasoning

> **How to use this guide.** The chapters follow the order of the lecture notes. Each chapter teaches the concept from scratch, then gives the formulas with every variable explained, then the diagrams, then the exam-relevant insights — and immediately afterwards the past-paper questions on that topic, solved line by line. Read a chapter, then attempt the papers listed for it in the final index (§16).

Contents

1. Introduction to Computer Architecture . . . 3
2. Measuring Performance: CPU Time, CPI, MIPS & Amdahl's Law . . . 6
3. Instruction Set Architecture: RISC vs CISC . . . 12
4. Dependability & Reliability . . . 16
5. Pipelining: The Assembly Line of the CPU . . . 17
6. Exploiting ILP: Loop Unrolling & Scheduling . . . 26
7. Branch Prediction & Control Hazards . . . 29
8. Dynamic Scheduling: Scoreboarding & Tomasulo's Algorithm . . . 34
9. Hardware Speculation & the Reorder Buffer . . . 39
10. The Memory Hierarchy & Cache Memory . . . 44
11. Virtual Memory & the TLB . . . 53
12. Multiprocessor Cache Coherence . . . 55
13. Multiprocessors & Multicores . . . 61
14. Vector, SIMD & GPU Architectures . . . 65
15. Formula & Quick-Reference Sheet . . . 70
16. Previous-Year Question Index & Revision Checklist . . . 72

1. Introduction to Computer Architecture

What the subject is about; architecture vs organization

Computer Architecture is the study of how a computer is designed and built, at the level where a programmer can still see the machine. The subject sits on the boundary between hardware and software: it decides what the hardware exposes to the compiler, and how the hardware is organised internally to make that exposed interface fast and cheap.

Every computer, from a watch to a supercomputer, is a **stored-program machine** (von Neumann model): a processor reads instructions from memory, executes them one after another, and writes results back. The instructions and the data live in the same memory, which is why the model is called *stored-program*.

Architecture vs Organization

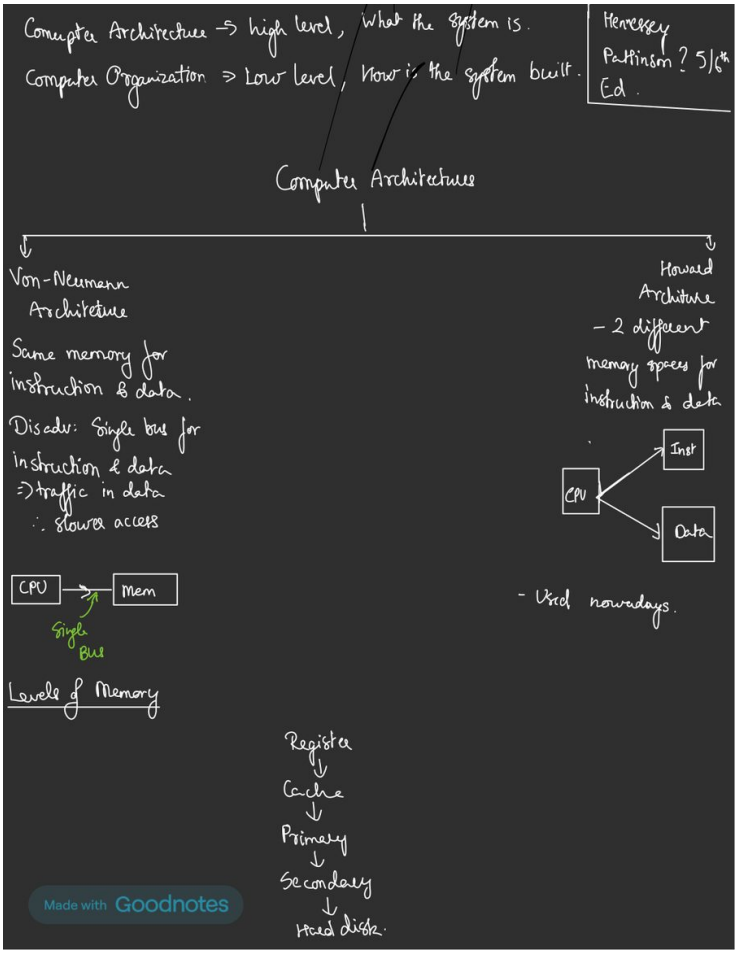
These two terms are used constantly in the exam, and they are *not* the same. A useful way to remember the difference: **architecture** answers "*what* does the machine do?", while **organization** answers "*how* is it built?".

Attribute	Computer Architecture (what)	Computer Organization (how)
Level	High-level / logical view	Low-level / physical view
Decides	Instruction set, data types, addressing modes, registers visible to the programmer	Data paths, control signals, memory technology, clock, buses, cache hardware
Sees	Only the ISA (instruction set architecture)	The whole hardware implementation
Example	Does the ISA have a floating-point multiply instruction?	Is the FP multiplier built from one unit or pipelined across several stages?
Impact	A program written for the architecture runs on all machines with that architecture	Different organizations of the same architecture can have very different speeds and costs
Same for all?	Yes — architecture is fixed for a family (e.g. x86, MIPS, RISC-V)	No — organization changes every generation

Exam line to remember: Architecture is *programmer-visible* and changes slowly; organization is *transparent to the programmer* and changes with technology.

The stored-program computer: the classic block diagram

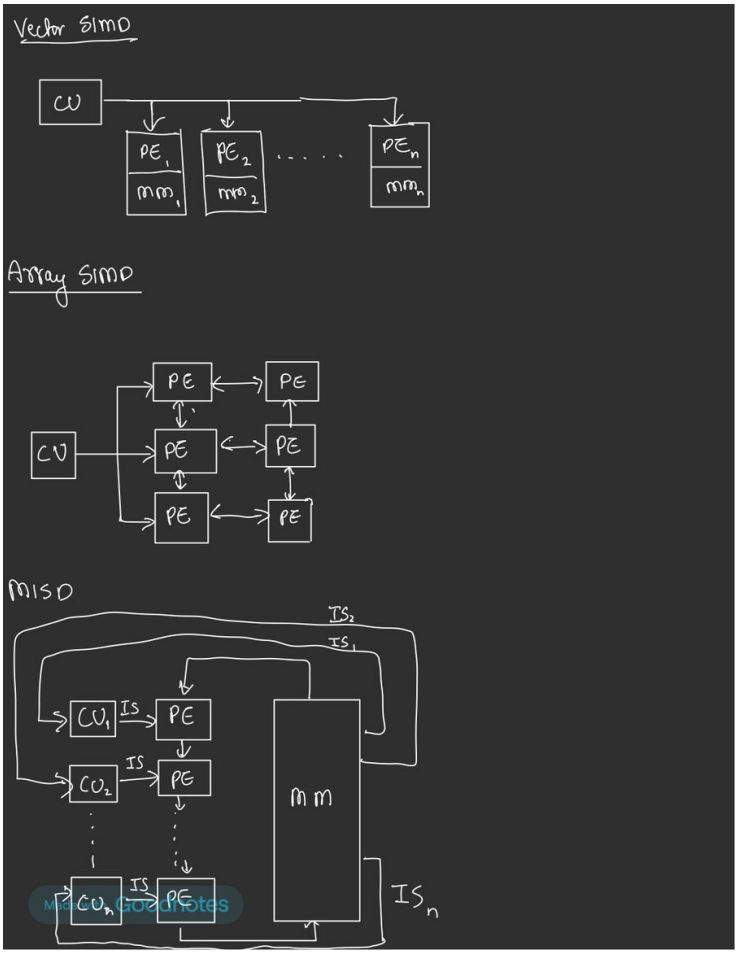
The von Neumann machine has five classic units: the **control unit** (decides what to do), the **arithmetic-logic unit / ALU** (does the arithmetic), the **memory** (stores programs and data), and **input/output** devices. Registers (PC, IR, MAR, MDR, general-purpose registers) sit inside the CPU. The CPU repeatedly performs the **fetch-decode-execute** cycle.



Von Neumann (stored-program) organisation and the fetch–decode–execute cycle (source: lecture notes, p. 2).

Unit	Role
Control Unit (CU)	Fetches the instruction, decodes it and generates control signals so every other unit acts in the right order.
ALU	Performs arithmetic (add, multiply) and logic (AND, OR, compare) operations.
Registers	Very fast on-chip storage. PC = program counter, IR = instruction register, MAR/MDR = memory address/data registers.
Memory	Holds both the program and its data; addressed by location.
I/O	Moves data in and out (keyboard, disk, network).

Flynn's taxonomy — how machines handle instruction/data streams



Flynn's taxonomy: SISD, SIMD, MISD, MIMD (source: p. 6).

A computer processes two streams: an **instruction stream** and a **data stream**. Depending on whether each stream is single or multiple, Michael Flynn classified all computers into four types. This classification appears in almost every exam.

Class	Full name	Meaning	Examples
SISD	Single Instruction, Single Data	One instruction works on one piece of data at a time — the classic scalar uniprocessor.	Old PCs, classic CPU
SIMD	Single Instruction, Multiple Data	One instruction is applied to many data elements in parallel. Vector/array processors and GPU threads.	Vector processors, GPUs, SSE/AVX
MISD	Multiple Instruction, Single Data	Several instructions act on the same data (rare; mostly theoretical).	Fault-tolerant arrays (rare)
MIMD	Multiple Instruction, Multiple Data	Independent processors run independent instruction and data streams.	Multicore CPUs, clusters

> **Why it matters:** SIMD is the basis of vector processors and GPUs (Topic 19). MIMD is the basis of multicore and shared-memory multiprocessors (Topics 16–18).

Program execution at the instruction level

A program is a sequence of instructions. Each instruction passes through a series of steps. In the simplest single-cycle machine every step happens in one clock period. In practice we break the work into **stages** so several instructions can overlap — this is pipelining (Topic 5).

- **Fetch** — read the instruction at address PC from memory; increment PC.
- **Decode** — work out what the instruction means and which registers it needs.
- **Execute** — perform the operation in the ALU.
- **Memory** — access data memory (only for load/store).
- **Write-back** — store the result in a register.

> **Key idea of the whole subject:** almost every technique in this guide — pipelining, caches, branch prediction, out-of-order execution — exists to keep the processor busy and to hide the fact that memory and operations take many cycles.

2. Measuring Performance: CPU Time, CPI, MIPS & Amdahl's Law

The single most examined numerical topic

Performance is about *time*. The only true measure of a computer's speed is how long a real program takes to run. But to reason about *why* one machine is faster, we break that time into parts we can measure and change. This section builds the performance equation from scratch, because almost every numerical question in the papers uses it.

2.1 Response time vs throughput

Concept	Definition	How you improve it
Response (execution) time	Time between the start and the completion of one task.	Faster clock, lower CPI, fewer instructions.
Throughput	Total work done per unit time (e.g. tasks per second).	More cores / more parallelism, better overlap (pipelining).

> **Classic distinction:** adding processors usually improves throughput but not the response time of a single task. Pipelining improves throughput without shortening the latency of one task.

2.2 The performance equation (the master formula)

A program's execution time depends on three things, and they multiply:

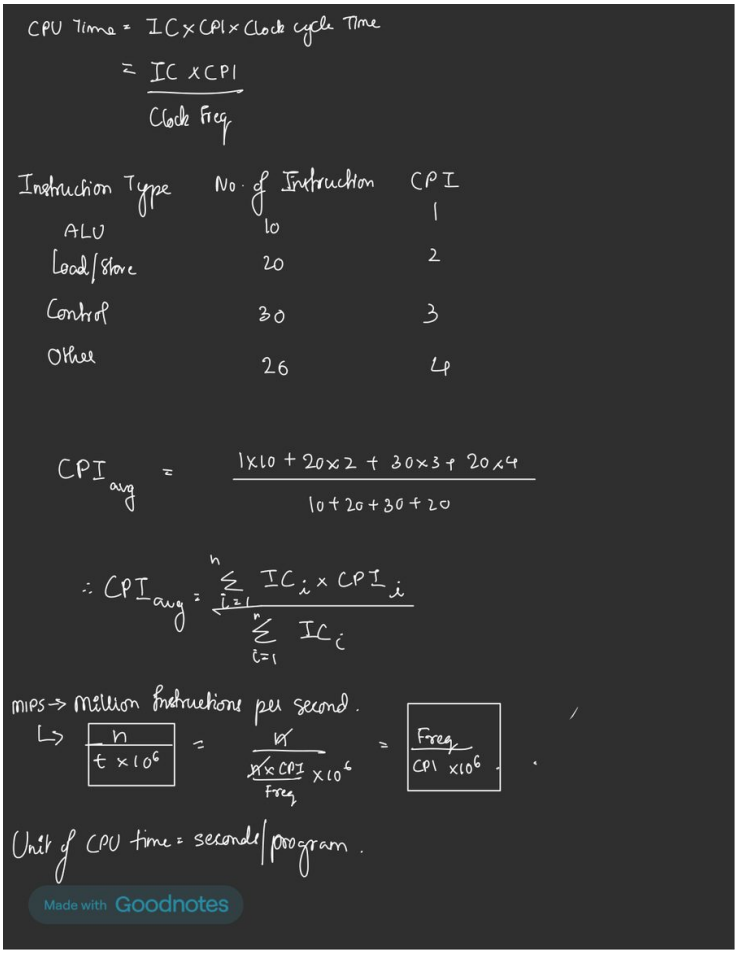
```
CPU Time = Instruction Count (IC) × CPI × Clock Cycle Time CPU Time = (IC × CPI) / Clock Frequency
```

Term	Meaning	Units
IC (Instruction Count)	Number of instructions the program executes.	instructions
CPI	Average **Cycles Per Instruction** = total cycles / IC.	cycles per instruction
Clock Cycle Time (CCT)	Duration of one clock period, CCT = 1 / f.	seconds (or ns)
Clock Frequency (f)	Number of cycles per second.	Hz (GHz = 10^9 Hz)

Because different instruction types cost different numbers of cycles, the **average CPI** is a weighted average of the CPI of each class:

```
CPI = Σ (CPIᵢ × Fᵢ) , where Fᵢ = fraction (or share) of instructions of class i, Σ Fᵢ = 1
Equivalently CPI = (Σ CPIᵢ × countᵢ) / IC
```

2.3 Deriving the equation (why it is a product)



CPU-time and CPI worked examples from the lecture notes (source: p. 28).

Suppose a program has *IC* instructions. On average each instruction takes *CPI* clock cycles. So the total number of clock cycles is *IC* × *CPI*. Each cycle lasts *CCT* seconds. Multiplying gives the total time. This is why the three factors are independent levers: you can reduce any one of them to speed up the program.

> **Trap:** reducing instruction count may raise CPI (e.g. CISC), and raising clock speed may raise CPI (deeper pipeline). The three factors are *not* independent in real designs.

2.4 MIPS — million instructions per second

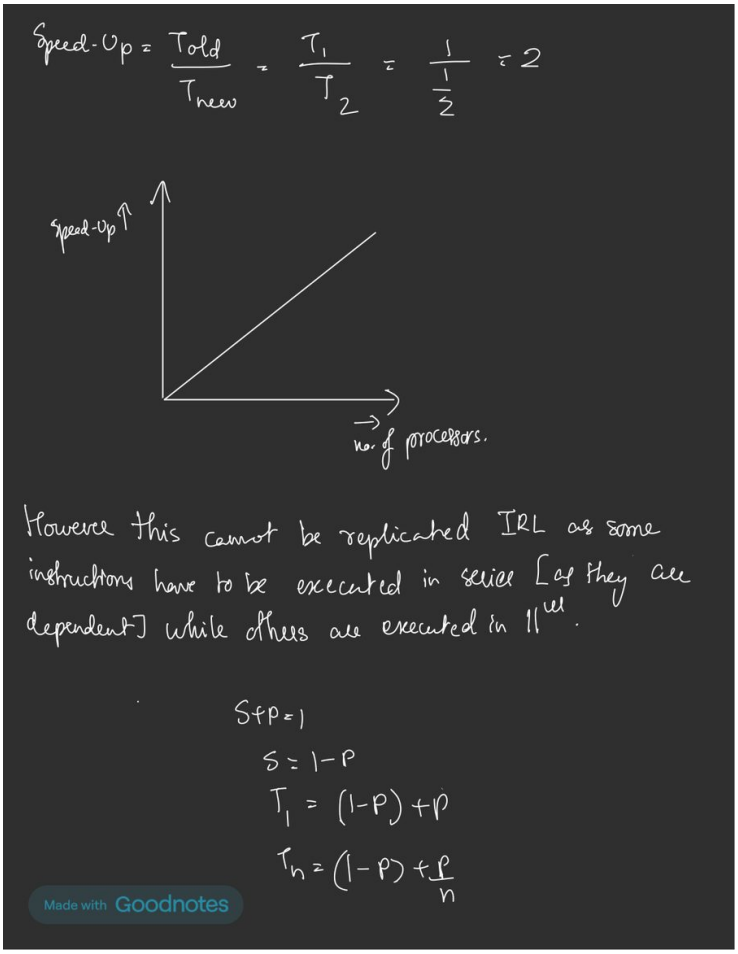
```
MIPS = IC / (Execution Time × 10⁶) = f / (CPI × 10⁶)
```

MIPS measures how many millions of instructions are executed each second. It is easy to quote but dangerous to trust, because:

- It does not account for what an instruction does — a machine with tiny instructions looks fast.
- It varies between programs on the same machine (different instruction mixes).
- It ignores I/O and memory behaviour.
- It can be used only to compare machines with the *same* ISA.

> **Memory hook:** MIPS is *meaningless* across different ISAs. Use CPU time for fair comparisons. (This is exactly the point of PYQ ca1/ca19.)

2.5 Speed-up and Amdahl's Law



Deriving Amdahl's law and the effect of the serial part (source: p. 19).

Amdahl's Law answers: *"If I speed up part of the machine, how much faster does the whole program get?"* The answer is limited by the part you did **not** speed up.

```
Speed-up S = Old execution time / New execution time = T_old / T_new Amdahl: S = 1 / [ (1 − f) + f / k ] f = fraction of the original time that is enhanced (0 ≤ f ≤ 1) k = speed-up of the enhanced portion (k > 1)
```

Derivation. Let the original time be 1 unit. The enhanced fraction *f* is sped up by a factor *k*, so it now takes *f/k*. The unenhanced fraction (1 − f) is unchanged. The new total time is (1 − f) + f/k. The speed-up is the ratio of old to new: 1 / [(1 − f) + f/k].

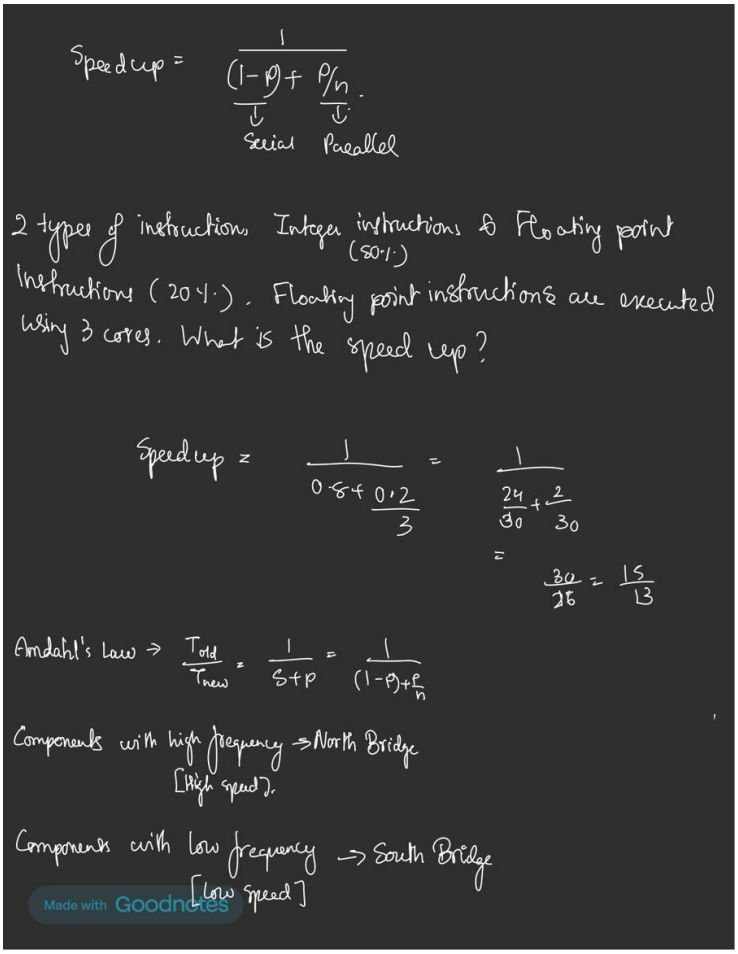
2.6 The ceiling: the serial fraction limits everything

As k → ∞ (the enhanced part becomes infinitely fast), the speed-up tends to a hard ceiling:

```
Maximum speed-up = 1 / (1 − f) as k → ∞ If 10% of the code cannot be parallelised (1 − f = 0.1), the maximum speed-up is 1/0.1 = 10× — no matter how many processors you add.
```

> **Interpreting f:** in parallel-speedup problems, f is the *parallelisable fraction* and k is the *number of processors* P. Then S = 1 / [(1 − f) + f/P].

2.7 Amdahl with a slowdown elsewhere (trade-off version)



Deriving the performance/Ahmdahl relationships from first principles (source: p. 20).

Real design changes can slow down *other* parts. If enhancing one part speeds it by k_1 but slows another fraction by k_2 ($k_2 < 1$ means slower), extend the law to add the slowed fraction:

$T_{new} = (1 - f_1 - f_2) + f_1/k_1 + f_2/k_2$, $S = 1 / T_{new}$ f_1 = fraction enhanced, k_1 = its speed-up; f_2 = fraction slowed, k_2 = its speed factor

After the change, the **new share** of a component is its new time divided by the new total time. The relative importance of components shifts, which is why a second enhancement may be less valuable than it first appears (PYQ ca36).

2.8 Mean performance and benchmarks

Mean	Formula	When to use
Arithmetic mean	$(1/n)\ \Sigma\ T_i$	When timing equal work (total time matters).
Harmonic mean	$n\ /\ \Sigma\ (1/R_i)$	For rates such as MIPS or FPS — the correct average of rates.
Geometric mean	$(\Pi\ R_i)^{1/n}$	For normalised ratios (SPEC) — the only mean that gives the same result when you change the reference machine.

> **Exam rule:** average *times* with the arithmetic mean, average *rates* with the harmonic mean, and average *normalised ratios* with the geometric mean.

2.9 Quantitative principles of design

A handful of principles explain almost every design decision in the subject. Examiners like to ask for these by name.

Principle	Meaning	Where you see it
Make the common case fast	Optimise what happens most often.	Small instructions, fast ALU path, cache for the common access
Amdahl's law	Speed-up is limited by the unenhanced part.	Deciding which component to improve
Locality	Programs reuse data and instructions that are near each other.	Caches, TLBs, prefetching
Parallelism	Do independent work at the same time.	Pipelining, superscalar, multicore, vector/SIMD

2.10 Design constraints: power, thermal and the multicore shift

For decades, transistor size shrank (Moore's law) and Dennard scaling kept power density constant, so clock speeds rose every generation. Dennard scaling ended around 2005: power density started rising with each shrink, so clock frequency could no longer increase freely.

Designers turned to **multicore** (more cores at moderate frequency) and to power-aware techniques instead.

Constraint	Effect on design
Power (dynamic)	P C·V²·f — reducing voltage is the biggest lever.
Power (static/leakage)	Rises as transistors shrink; limits how low voltage can go.
Thermal	Heat must be removed; limits sustained frequency (and motivates throttling).
Energy	Battery life and cost; motivates DVFS (dynamic voltage/frequency scaling).
Multicore shift	Instead of a faster single core, put more cores on the chip.

2.9 Exam-relevant insights

- **Never** compare MIPS across ISAs; compare CPU time.
- CPI is an *average* — always weight by the fraction of each instruction class.
- Amdahl's Law always has the form 1 / (unimproved + improved). Identify f first, then plug in.
- Speed-up = old/new; "X% faster" means new = old/(1+X/100).
- Watch units: GHz = 10^9 Hz; convert cycles to time with CCT = 1/f.
- A "65% improvement" in AMAT means the new AMAT is 35% of the old (see PYQ ca15/ca26).

> **Common mistake:** in Amdahl problems, students speed up the wrong fraction. Read the question twice and write down f and k explicitly before computing.

Previous-Year Questions — Performance & Amdahl

Q (ca3 · Q2). Derive and explain Amdahl's law.

Solution. Let a program take time T. A fraction f of that time can be enhanced; the enhancement makes that fraction k times faster. The unenhanced part (1−f)T is unchanged. New time = (1−f)T + (fT)/k. Speed-up S = T / new time = 1/[(1−f) + f/k]. As k→∞, S→1/(1−f): the serial part sets the ceiling.

```
T_new = (1−f)T + (fT)/k ; S = 1 / [(1−f) + f/k] ; S_max = 1/(1−f)
```

Q (ca4 · Q1). A program has a portion enhanced to run 4× faster, giving a system speed-up of 3.3×. What is the fraction enhanced?

```
3.3 = 1/[(1−f) + f/4] → 0.3030 = (1−f) + 0.25f = 1 − 0.75f 0.75f = 0.6970 → f = 0.929 = 92.9%
```

Answer: f ≈ 92.9% of the original time must be in the enhanced portion.

Q (ca4 · Q2). A quad-core could speed up a computer by 4×, but rarely does. Use Amdahl to find the percentage of execution that must be parallelised for overall speed-ups of 3, 2, 1.5, 1.25.

```
S = 1/[(1−f) + f/4]. Solve for f: f = 4(S−1) / (3S). S=3 → f = 4(2)/(9) = 0.889 = 88.9% S=2 →
f = 4(1)/(6) = 0.667 = 66.7% S=1.5 → f = 4(0.5)/4.5 = 0.444 = 44.4% S=1.25→ f = 4(0.25)/3.75=
0.267 = 26.7%
```

Answer: to gain 3× you must parallelise ~88.9%; for 2× ~66.7%; for 1.5× ~44.4%; for 1.25× ~26.7%. Notice how quickly the required parallel fraction grows.

Q (ca4 · Q3). A benchmark: 39% loads, 12% stores, 28% ALU, 6% multiplies, 2% divides, 10% conditional branches, 3% unconditional branches. CPIs: loads/stores 5, ALU 3, multiply 6, divide 15, branch 4. Which single ALU enhancement is best?

```
CPI_base = 0.39(5)+0.12(5)+0.28(3)+0.06(6)+0.02(15)+0.13(4) = 1.95 + 0.60 + 0.84 + 0.36 +
0.30 + 0.52 = 4.57 (a) cond-branch CPI→2 : ΔCPI = 0.10×(4−2) = 0.20 → new 4.37 (b) multiply
CPI→3 : ΔCPI = 0.06×(6−3) = 0.18 → new 4.39 (c) divide CPI→7 : ΔCPI = 0.02×(15−7)= 0.16 → new
4.41 (d) ALU CPI→2 : ΔCPI = 0.28×(3−2) = 0.28 → new 4.29
```

Answer: (d) — improving the ALU (non-multiply/divide) gives the largest CPI reduction (0.28), because ALU operations are the most frequent.

Q (ca5 · Q4 / ca19 · Q1). MIPS ISA at 500 MHz: ALU 3 cycles, branch/jump 2, sw 4, lw 5. A program executes 200M ALU, 55M branch/jump, 25M sw and 20M lw. Find the CPU time.

```
IC = 200M + 55M + 25M + 20M = 300M instructions Total cycles = 200M×3 + 55M×2 + 25M×4 + 20M×5
= 600M + 110M + 100M + 100M = 910M cycles T_clock = 1/500 MHz = 2 ns CPU time = 910M × 2 ns =
1820 ms = 1.82 s
```

Answer: CPU time = 1.82 s (CPI = 910M/300M ≈ 3.03).

Q (ca7 · Q1a). Amdahl with communication overhead. (a) Speed-up with 8 processors if, for every processor added, communication overhead is 0.5% of the original execution time. (b) Speed-up with N processors if, each time the number of processors doubles, the overhead increases by 0.5%.

```
(a) With P processors, serial part s, parallel part (1−s), plus overhead 0.5%×(P−1): T_new = s + (1−s)/P + 0.005(P−1) (times the original time) For P=8: S = 1 / [ s + (1−s)/8 + 0.005×7 ] = 1/[ s + (1−s)/8 + 0.035 ] (substitute the given serial fraction s). (b) With N processors, overhead = 0.005×log₂N: S(N) = 1 / [ s + (1−s)/N + 0.005·log₂N ]
```

Idea being tested: communication overhead grows with the number of processors, so speed-up saturates and can even fall — this is the practical limit Amdahl's plain form ignores.

Q (ca33 · Q1). Three modification versions affect components X, Y, Z whose execution-time fractions are 30%, 45%, 25%. Given each version's speed factors, find the overall speed-up of each and rank them.

```
S_version = 1 / [ 0.30/k_X + 0.45/k_Y + 0.25/k_Z ] (read k_X,k_Y,k_Z from the table for each version; k<1 means slower)
```

Method: evaluate the weighted sum for each version, take the reciprocal, then rank from largest S. A component that is slowed (k<1) increases its term and can pull S below 1.

Q (ca36 · Q1a). (i) A new FP unit speeds FP by 2×; FP is 20% of execution time — overall speed-up? (ii) The change also slows data-cache accesses by 1.5× (2/3 speed-up); data-cache accesses are 10% of time — overall speed-up now? (iii) What % of the new execution time is FP, and what % is data cache?

```
(i) S = 1/[(1−0.20) + 0.20/2] = 1/[0.80 + 0.10] = 1/0.90 = 1.111 (≈11.1% faster) (ii) now also slow 10% by factor 2/3: T_new = (1 − 0.20 − 0.10) + 0.20/2 + 0.10/(2/3) = 0.70 + 0.10 + 0.15 = 0.95 → S = 1/0.95 = 1.053 (iii) new shares: FP = 0.10/0.95 = 10.5% ; data cache = 0.15/0.95 = 15.8%
```

Answer: (i) 1.11×; (ii) 1.05× (the cache slowdown eats most of the gain); (iii) FP ≈ 10.5%, data cache ≈ 15.8% of the new time.

Q (ca36 · Q1b). Vector hardware makes a computation 10× faster; the measured vectorisation is 70%. What vectorisation would be needed to equal a further 2× speed-up of the vector unit (i.e. 20× total)?

```
Current: S = 1/[(1−0.7) + 0.7/10] = 1/[0.30 + 0.07] = 1/0.37 = 2.703 Target : a 2× faster vector unit gives 20× on the vector part: S' = 1/[(1−f) + f/20]. Set S' = 2.703 0.37 = (1−f) + f/20 = 1 − 0.95f → 0.95f = 0.63 → f = 0.663 = 66.3%
```

Answer: the compiler team would need only ≈66.3% vectorisation (actually less than the current 70%) — meaning the current vectorisation already beats the proposed 2× hardware speed-up. State the comparison explicitly.

3. Instruction Set Architecture: RISC vs CISC

The hardware/software contract

The **Instruction Set Architecture (ISA)** is the complete set of instructions, registers, addressing modes and data types that a processor understands. It is the *interface* between hardware and software: the compiler targets the ISA, and the hardware implements it. Two programs written for the same ISA run on every processor of that family, no matter how differently the hardware is built.

What the ISA fixes

- **Instruction set** — which operations exist (add, load, branch …).
- **Data types** and their widths (integer 8/16/32/64-bit, IEEE floating point).
- **Registers** visible to the programmer and their count.
- **Addressing modes** — how an operand's address is computed (immediate, register, displacement …).
- **Encoding** — how instructions are laid out as bits (fixed vs variable length).

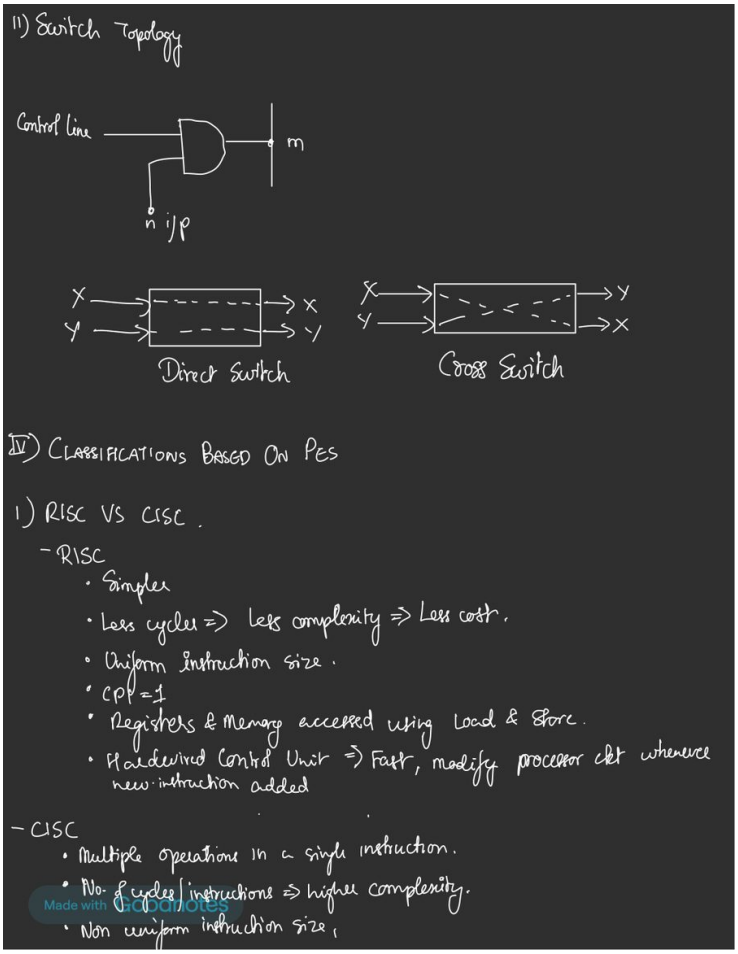
RISC vs CISC — the great divide

In the 1970s-80s, instruction sets grew complex to make compilers easy and to close the “semantic gap” with high-level languages. These are **CISC** (Complex Instruction Set Computer) machines. Studies showed compilers rarely used most complex instructions, and complex instructions are hard to pipeline. The **RISC** (Reduced Instruction Set Computer) movement kept the instruction set small and regular so that each instruction executes in one cycle and the pipeline runs fast.

Aspect	CISC	RISC
Instruction count	Many; variable-length, complex	Few; fixed-length, simple
Instructions per operation	One complex instruction does a lot	Several simple instructions
CPI	High (many cycles per instruction)	Low — close to 1
Clock rate	Lower	Higher (simple stages)
Pipelining	Hard (variable work per instruction)	Easy (uniform work per stage)
Registers	Few	Many (register–register model)
Memory access	Many instructions may access memory	Only load/store access memory
Control unit	Micro-programmed	Hard-wired
Decoding	Complex	Simple and fast
Compiler role	Hardware does much work	Compiler does much work (optimisation)
Examples	x86, VAX	MIPS, ARM, RISC-V, SPARC

> **Why RISC won the performance race:** a simple, regular instruction set lets you build a deep pipeline with a fast clock. *Loss of power in the instruction set is repaid in speed of the pipeline.* Modern x86 chips are CISC *outside* but decode into RISC-like micro-operations *inside* — the best of both.

Addressing modes



RISC vs CISC characteristics compared (source: p. 12).

Mode	Operand is…	Example (MIPS-style)
Immediate	A constant inside the instruction	`addi r1,r2,4`
Register	A register's contents	`add r1,r2,r3`
Base + displacement	memory[base + const]	`lw r1,20(r2)`
Register indirect	memory[register]	`lw r1,0(r2)`
Indexed	memory[base + index]	`lw r1,(r2+r3)`
PC-relative	memory[PC + const] — used for branches	`beq r1,r2,L`

Data representation: integers and IEEE 754 floating point

The ISA also fixes how numbers are encoded, and the exam often asks about floating point. Know the three integer schemes and the IEEE 754 layout.

Scheme	How a negative number is stored	Range (n bits)	Notes
Sign-magnitude	Top bit is the sign; rest is magnitude.	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	Two zeros (+0, −0); awkward arithmetic.
One's complement	Flip every bit.	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	Still two zeros; end-around carry.
Two's complement	Flip every bit, then add 1.	-2^{n-1} … $+(2^{n-1}-1)$	One zero; subtraction = addition; used everywhere.

IEEE 754 single precision (32 bits) stores a number as sign × 1.fraction × $2^{(\text{exponent}-\text{bias})}$:

```
IEEE 754 single: [ sign : 1 | exponent : 8 | fraction : 23 ] Value = (−1)^sign × 1.fraction × 2^(exponent − 127) Exponent bias = 127 ; exponent 0 and 255 are reserved IEEE 754 double: [ sign : 1 | exponent : 11 | fraction : 52 ], bias = 1023
```

Special case	Exponent	Fraction	Meaning
Zero	all 0	all 0	±0
Denormalised	all 0	non-zero	tiny numbers, no implicit leading 1
Infinity	all 1	all 0	±∞ (e.g. divide by zero)
NaN	all 1	non-zero	Not a Number (e.g. 0/0)

Example (also seen in the notes, p. 15): represent 0.4 as a single-precision float.

```
0.4 × 2 = 0.8 → 0 ; 0.8×2 = 1.6 → 1 ; 0.6×2 = 1.2 → 1 ; 0.2×2 = 0.4 → 0 ; 0.4×2 = 0.8 → 0 …
0.4 = 0.011001100110011…₂ = 1.1001100110011…₂ × 2□²  exponent = −2 + 127 = 125 = 01111101₂ ;
fraction = 1001100110011… (23 bits)  0.4 = 0 01111101 10011001100110011001100
```

Instruction-level parallelism (ILP) — the goal

ILP is the number of independent operations the hardware can overlap in a pipeline. If instructions are independent, they can execute simultaneously. If they depend on one another, they must wait. Almost all of the rest of this guide is about *finding and exploiting ILP*.

Technique	Static or dynamic	Idea
Pipelining	Hardware (dynamic)	Overlap stages of consecutive instructions
Superscalar	Dynamic	Issue >1 instruction per cycle
VLIW	Static (compiler)	Compiler packs independent ops into one long instruction
Loop unrolling	Static (compiler)	Replicate the body to expose more independent work
Scoreboarding / Tomasulo	Dynamic (hardware)	Issue out of order, resolve hazards in hardware
Speculation (ROB)	Dynamic	Execute past branches / predicted paths safely

Dependences vs hazards — the key distinction

A **dependence** is a property of the *program* (its meaning). A **hazard** is a property of the *pipeline* — a situation where overlapping would give the wrong answer. Dependences are fixed; hazards depend on the implementation and can sometimes be engineered away.

Type	Symbol	Meaning	Can we remove it?
True (data) dependence	RAW — Read After Write	Instruction j needs a value that instruction i produces. This is a **real** dependence.	**No** — must preserve the order (but can forward the value).
Antidependence	WAR — Write After Read	j writes a location that i reads. j must not overwrite before i reads.	**Yes** — by register renaming.
Output dependence	WAW — Write After Write	i and j write the same location; the final value must come from the later one.	**Yes** — by register renaming.
Control dependence	branch	An instruction is executed only if a branch goes a certain way.	Reduce with prediction / speculation.

> **Memory hook: RAW is real** (True); WAR and WAW are *name* dependences — they exist only because two instructions happen to use the same register name, so renaming kills them. This is the single most examined distinction in the subject.

A worked example (used in PYQ ca1/ca22). Consider the loop:

```
A[i] = A[i] * B[i]; /* S1 */ B[i] = A[i] + c; /* S2 */ A[i] = C[i] * c; /* S3 */ C[i] = D[i] * A[i]; /* S4 */
```

Dependence	Between	Type	Reason
S1 → S2	S1 produces A[i], S2 reads A[i]	RAW (true)	Real data flow
S1 → S3	S1 writes A[i], S3 writes A[i]	WAW (output)	Both write A[i]
S3 → S4	S3 writes A[i], S4 reads A[i]	RAW (true)	Real data flow
S2 → S3	S2 reads A[i], S3 writes A[i]	WAR (anti)	Name reuse

Renaming removes the name dependences (WAW S1→S3 and WAR S2→S3) by giving S3 a fresh register, e.g. `A2[i] = C[i]*c`. The true dependences (S1→S2, S3→S4) stay.

Previous-Year Questions — ISA, dependences & renaming

Q (ca1 · Q2a / ca22 · Q2). Find all true, output and antidependences in the loop below and eliminate output dependences and antidependences by renaming.

```
for (i = 0; i < 100; i++) { A[i] = A[i] * B[i]; /* S1 */ B[i] = A[i] + c; /* S2 */ A[i] = C[i] * c; /* S3 */ C[i] = D[i] * A[i]; /* S4 */ }
```

Solution. Work through every pair of statements, asking “do they touch the same location, and in what order?”

- **RAW (true) dependences** — a later statement *reads* what an earlier one *writes*:
  - S1 → S2: S1 writes A[i]; S2 reads A[i]. RAW on A[i].
  - S3 → S4: S3 writes A[i]; S4 reads A[i]. RAW on A[i].
- **WAW (output) dependence** — two statements write the same location:
  - S1 → S3: both write A[i]. The final value of A[i] must be S3's, so order matters.

- **WAR (antidependence)** — a later statement writes what an earlier one reads:
  - S2 → S3: S2 reads A[i]; S3 writes A[i]. S3 must not overwrite before S2 reads.

Renaming. Give the "second writer" of A[i] a new name so no two live values share a register. S1 writes A[i] and S3 overwrites A[i] — rename S3's target to A2:

```
A[i] = A[i] * B[i]; /* S1 : writes A[i] */ B[i] = A[i] + c; /* S2 : reads A[i] */ A2[i] = C[i] * c; /* S3 : writes A2[i] (renamed) */ C[i] = D[i] * A2[i]; /* S4 : reads A2[i] */
```

After renaming, the WAW (S1→S3) and the WAR (S2→S3) disappear. The true dependences S1→S2 and S3→S4 remain because they carry real data. Renaming therefore removes *false* dependences only — it never removes a true dependence.

> **Exam tip:** list the statements, then for each later statement check every earlier one. You will always find the RAW pairs by following the data flow, and the WAW/WAR pairs by looking for repeated register names.

Q (ca2 · Qb). Consider the loop below. Are there dependences between S1 and S2? Is the loop parallel? If not, show how to make it parallel.

```
for (i = 0; i < 100; i++) { A[i] = A[i] + B[i]; /* S1 */ B[i+1] = C[i] + y[i]; /* S2 */ }
```

Solution.

- Look at the *subscripts*, not just the array names. S1 writes `A[i]` and reads `A[i], B[i]`. S2 writes `B[i+1]` and reads `C[i], y[i]`.
- S1 writes **B[i]**; S2 writes **B[i+1]**. For the same *i* these are different elements, so there is **no** dependence between S1 and S2 in the same iteration.
- But across iterations: S1 in iteration i writes B[i], and S2 in iteration i−1 writes B[i]. Both write B[i] (different iterations) — a **WAW loop-carried output dependence** on B[i]. There is no RAW involving B because S1 never *reads* B[i+1] and S2 never reads B[i].
- So the only cross-iteration dependence is the output dependence on B between S1 of iteration i and S2 of iteration i−1.

Is it parallel? The loop is **not** fully parallel as written, because the two statements write overlapping B elements across iterations. **Fix:** rename the B array written by S2 into a separate array B2, so the two writers no longer collide:

```
for (i = 0; i < 100; i++) { A[i] = A[i] + B[i]; /* S1 */ B2[i+1] = C[i] + y[i]; /* S2 : renamed output */ }
```

Now S1 writes A/B and S2 writes B2 — no overlap — so every iteration is independent and the loop can be executed in parallel.

4. Dependability & Reliability

MTTF, MTTR, availability and how to keep a farm alive

A system is **dependable** if it keeps delivering correct service. Dependability has two big ingredients: **reliability** (how long until something fails) and **availability** (what fraction of time the service is up). These are measured with a small set of times that appear in the exam.

Term	Symbol	Meaning
Mean Time To Failure	MTTF	Average time from the start (or a repair) until the next failure. Measures *reliability*.
Mean Time To Repair	MTTR	Average time to fix a failure and restore service.
Mean Time Between Failures	MTBF	MTBF = MTTF + MTTR — average time between two consecutive failures.
Availability	A	Fraction of time the system is usable.
Failure Rate	λ	Failures per unit time; λ = 1 / MTTF.

```
Availability A = MTTF / (MTTF + MTTR) = MTTF / MTBF Failure rate λ = 1 / MTTF
```

Derivation of availability. In one failure–repair cycle the system is up for MTTF and down for MTTR, so the fraction of time it is up is MTTF / (MTTF + MTTR). A system with MTTF = 1,000,000 h and MTTR = 1 h has A = $10^6/(10^6+1) \approx 0.999999$ — “six nines”.

Reliability of a whole system

If a system is made of many components, its failure rate is the *sum* of the component failure rates (for independent components that each must work):

```
1 / MTTF_system = Σ (1 / MTTF_i) So MTTF_system = 1 / (1/MTTF₁ + 1/MTTF₂ + …)
```

> **Why server farms still work:** 10,000 computers each with MTTF = 35 days fail often as a whole, but if the service tolerates losing a third of them, the *service* MTTF is far longer than any one machine. Adding components lowers whole-system MTTF but can raise availability if failures are masked by redundancy. (See PYQ ca30.)

Exam-relevant insights

- A is a fraction; multiply by 100 for a percentage, or count “nines” for SLA language.
- λ = 1/MTTF, so halving MTTF doubles the failure rate.
- For *series* reliability (all must work) add failure rates.
- For *parallel* redundancy (only one must work) availability rises.

Q (ca10 · Q2). Convert the C code below to MIPS, find the hazards, and reorder to avoid stalls.

```
A = B + E; C = B + F;
```

Solution.

```
lw $t0, B # $t0 = B lw $t1, E # $t1 = E add $t2, $t0, $t1 # A = B + E sw $t2, A lw $t3, F # $t3 = F add $t4, $t0, $t3 # C = B + F sw $t4, C
```

- **Hazards:** the `add $t2,$t0,$t1` depends on the two loads (RAW, load-use); the second add reuses $t0 (B) which is already loaded (no new hazard).
- **Reorder:** hoist the independent `lw $t3,F` above the first add so the load-use slot is filled:

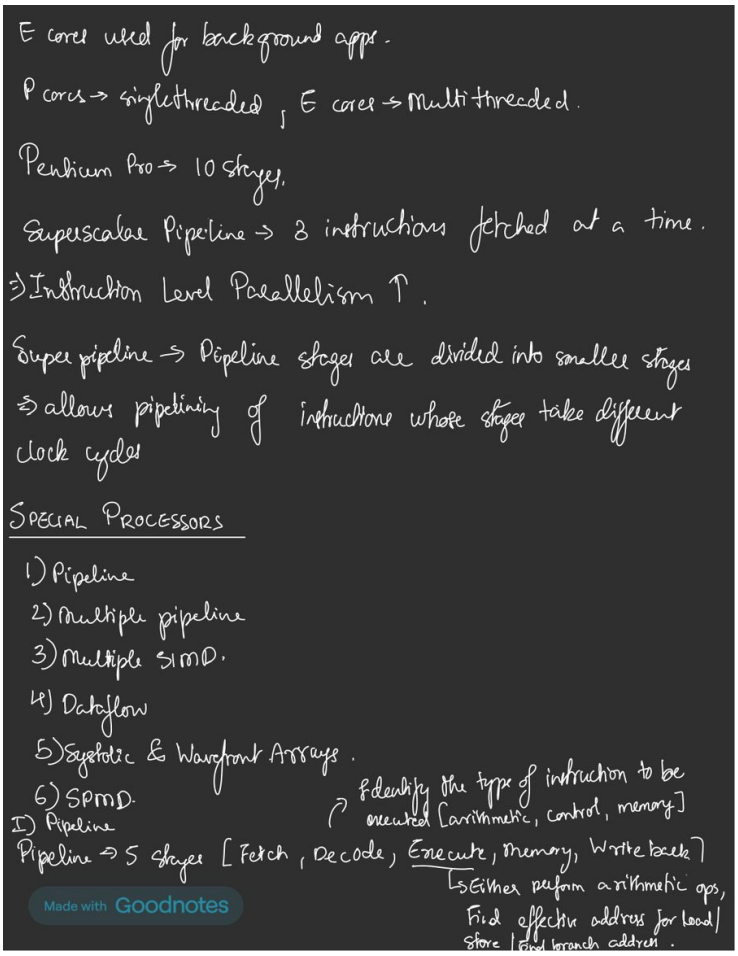
```
lw $t0, B lw $t1, E lw $t3, F ; independent load fills the slot add $t2, $t0, $t1 sw $t2, A add $t4, $t0, $t3 sw $t4, C
```

5. Pipelining: The Assembly Line of the CPU

Speed-up, hazards and the MIPS 5-stage pipeline

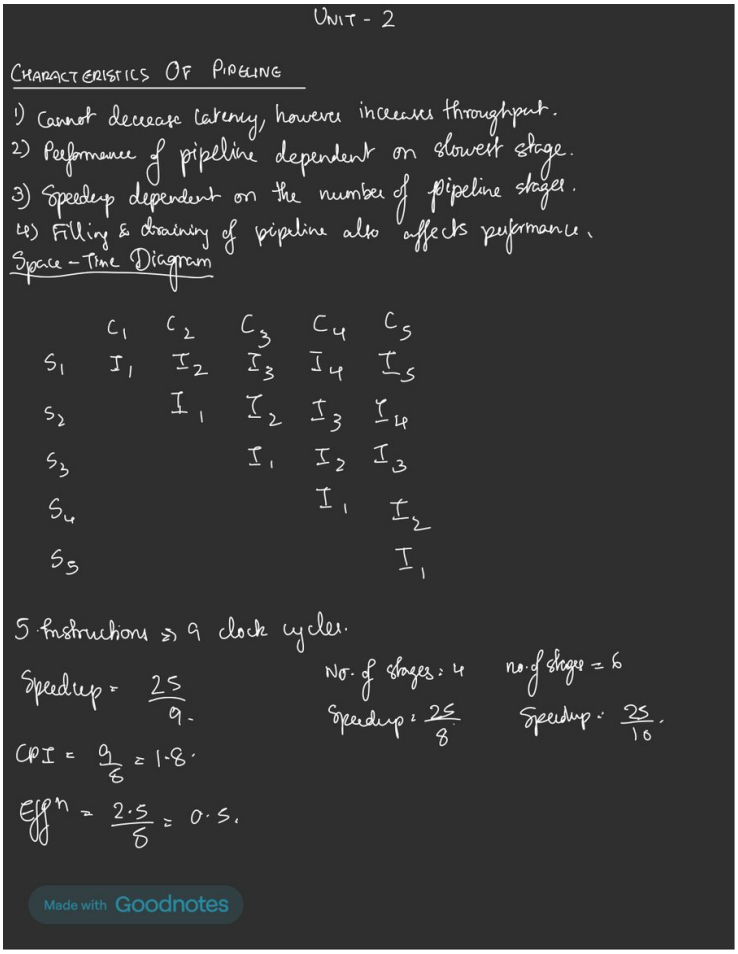
A single-cycle processor does fetch, decode, execute, memory and write-back one after another in one long clock period — wasteful, because at any instant most hardware sits idle. A **pipelined** processor overlaps these steps: while instruction 1 executes, instruction 2 is decoded and instruction 3 is fetched. Exactly like a car assembly line, throughput rises sharply even though the time for *one* instruction (its latency) does not fall.

5.1 The classic MIPS 5-stage pipeline



Instruction execution and the five pipeline stages (source: p. 14).

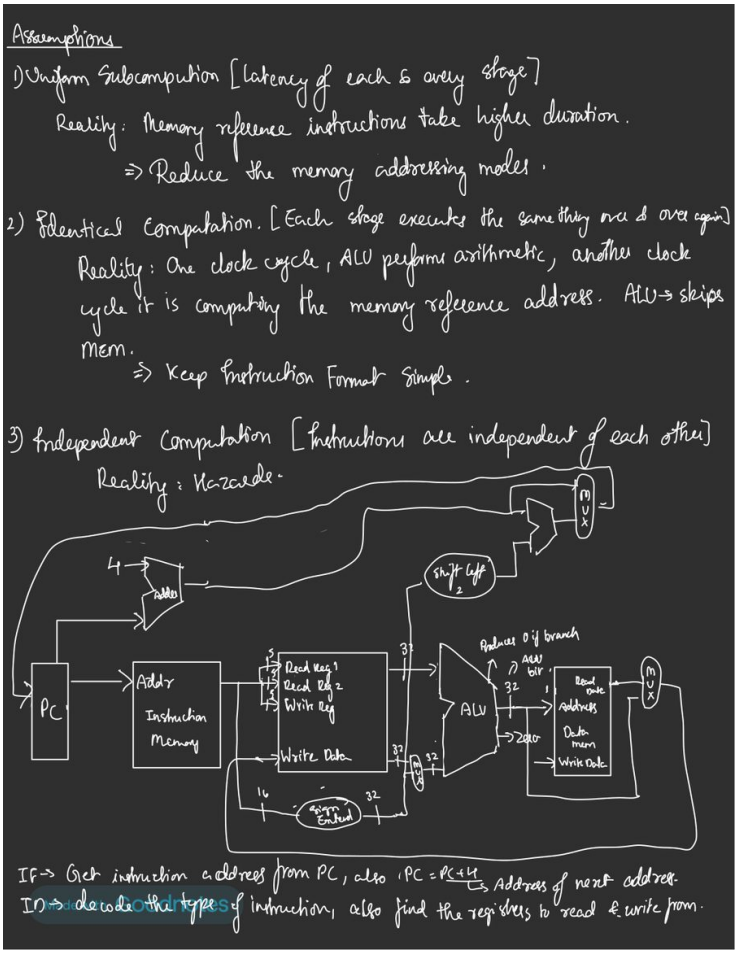
Stage	Name	What happens
1	IF — Instruction Fetch	Read the instruction at PC from memory; PC ← PC + 4.
2	ID — Instruction Decode	Decode the opcode, read the register file, compute branch targets.
3	EX — Execute	ALU does the arithmetic or address calculation; branch decision made.
4	MEM — Memory	Load or store data (only for load/store instructions).
5	WB — Write-Back	Write the ALU or loaded value back into the register file.



Classic 5-stage pipeline: the instruction stream advances one stage per cycle (source: lecture notes, p. 33).

An instruction moves one stage to the right every cycle. The **pipeline registers** between stages hold the intermediate results so that each stage works on a different instruction in the same cycle.

5.2 Pipeline timing diagram (space-time chart)



Instruction types and their pipeline stage usage (source: p. 34).

The standard way to show a pipeline is a table where rows are instructions and columns are clock cycles. Below, four instructions I1–I4 flow through the five stages:

Instruction	C1	C2	C3	C4	C5	C6	C7	C8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB

The first instruction finishes at cycle 5; every cycle after that one instruction completes. The pipeline is **full** from cycle 5 onward.

5.3 Speed-up, throughput and the ideal CPI

With k stages and N instructions, ignoring stalls:

```
Non-pipelined time = N × k × t (t = delay of one stage) Pipelined time = (k + N − 1) × t
Speed-up S = N·k / (k + N − 1) → k as N → ∞ Ideal CPI = 1 instruction per cycle (throughput = f)
```

Derivation. The first instruction needs all k stages (k cycles). Each of the remaining N − 1 instructions adds exactly one more cycle, because they overlap. Total cycles = k + (N − 1). Dividing the un-pipelined time N·k by the pipelined time gives the speed-up.

> **Three key facts.** (1) Pipelining improves *throughput*, not the latency of a single instruction. (2) The ideal speed-up is the number of stages. (3) The clock period is set by the *slowest* stage (plus register delay).

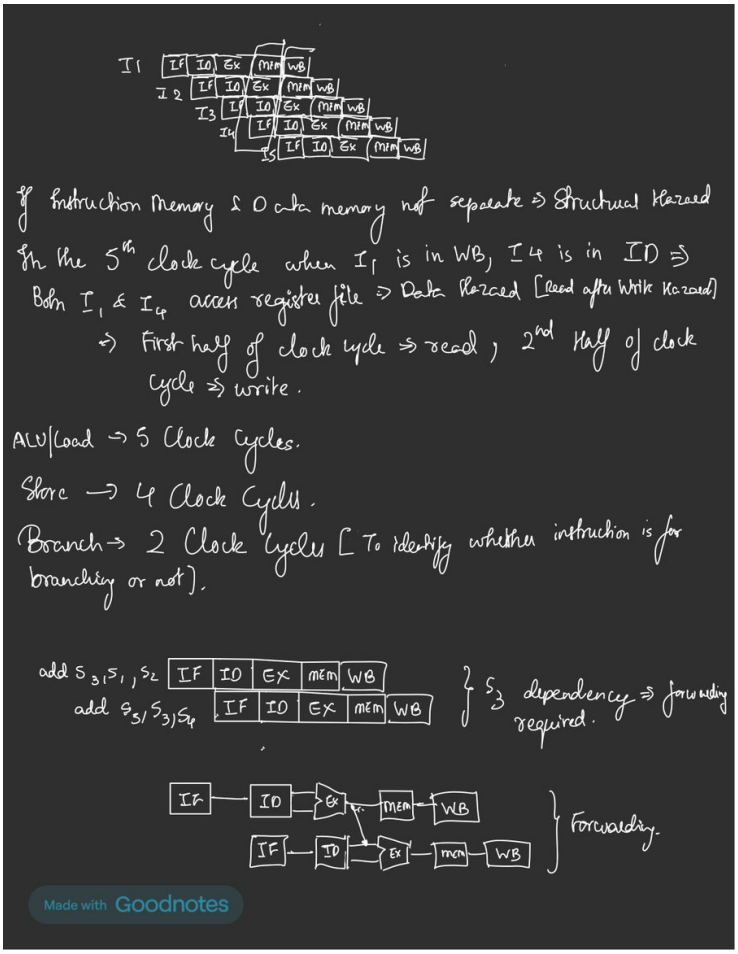
5.4 Clock period, latency, throughput and the effect of register overhead

The pipeline can run no faster than its slowest stage. If stage i takes delay t_i and each pipeline register adds delay d, then:

```
Cycle time T = max(t₁, t₂, …, t_k) + d Latency of one instruction = k × T Throughput = 1 / T (instructions per second, ideal)
```

Splitting a stage into two halves lets you use a shorter cycle time — but the extra pipeline register also adds delay d, and deeper pipelines mean more hazards and more penalty when a branch mispredicts. This trade-off is examined (PYQ ca30/ca32).

5.5 Pipeline hazards — the three enemies



Five-stage pipeline with per-instruction stage counts (source: p. 35).

A **hazard** is any situation that prevents the next instruction from executing in the next cycle. There are exactly three kinds, and you must be able to name and fix each.

Hazard	Cause	Example	Fixes
Structural	Two instructions need the same hardware resource in the same cycle	IF and MEM both need memory; one ALU for two EX stages	Duplicate hardware (split I/D caches), stall, or schedule differently
Data (RAW/WAR/WAW)	An instruction needs an operand that an earlier instruction has not yet produced	`add r1,r2,r3 ; sub r4,r1,r5`	Forwarding (bypassing), stalling, scheduling, renaming
Control	The next PC depends on a branch that has not resolved yet	`beq … ; next instruction unknown`	Stall, predict (static/dynamic), delayed branch, speculation

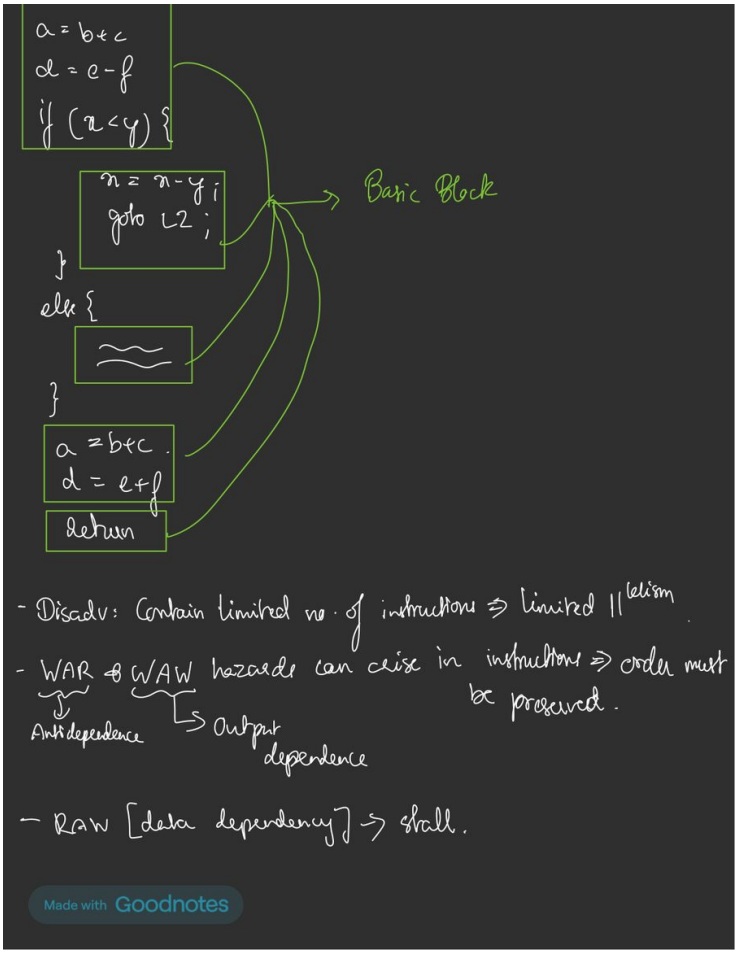
5.6 Data hazards in detail

Type	Full name	Meaning
RAW	Read After Write	j reads a register that i writes → j must wait for i's value (true dependence).
WAR	Write After Read	j writes a register that i reads → j must not write before i reads (name dependence).
WAW	Write After Write	j writes a register that i also writes → final order must be preserved (name dependence).

Forwarding (bypassing) fixes most RAW hazards: instead of waiting for the value to be written back, the ALU output is sent straight to the next instruction's EX stage. The hardware adds a path from the EX/MEM and MEM/WB pipeline registers back to the ALU inputs. With forwarding, a RAW that needs an ALU result costs **no** stall; a RAW after a load needs **one** stall (the load-use hazard) because the data only arrives after MEM.

> **Load-use hazard:** a load followed immediately by an instruction that uses the loaded register must stall one cycle even with forwarding — the value simply does not exist yet at the start of EX. This is the most common stall in MIPS pipelines.

5.7 Control hazards and their cost



Control and data hazards with example instruction sequences (source: p. 46).

A branch is decided in the EX stage in the classic pipeline, but the next instruction is fetched in ID — before we know which way the branch goes. If we simply stall until the branch resolves, we lose cycles on every branch. The number of lost cycles is the **branch penalty**.

```
Stalls per branch = (number of stages between IF and the stage that resolves the branch)
Branch penalty (CPI added) = branch frequency × misprediction rate × penalty cycles Effective
CPI = ideal CPI + stall cycles per instruction
```

Technique	How it reduces the penalty
Stall until resolved	Simple; penalty = pipeline depth to resolution (e.g. 2-3 cycles).
Predict (static)	Guess a direction (always-not-taken / always-taken / BTFNT); flush only on a wrong guess.
Delayed branch	The compiler puts a useful instruction in the branch delay slot — executed regardless.
Branch target buffer	Cache the branch's previous target and outcome, so fetch continues without waiting.
Speculation	Execute down the predicted path and keep the results safe until the branch is confirmed.

We develop branch prediction fully in Topic 7; here it is enough to know that it converts a *certain* penalty into a *probabilistic* one: you only pay when you guess wrong.

5.8 Superscalar and deeper pipelines

A **superscalar** processor issues more than one instruction per cycle (e.g. 2- or 4-wide). It needs several functional units and extra logic to detect dependences among the parallel-issued instructions. A **VLIW** machine instead has the compiler pack independent operations into one very long instruction word, moving the scheduling burden to software.

Design	Who finds ILP?	Issue width	Hardware cost
Scalar pipeline	Hardware (pipeline)	1	Low
Superscalar	Hardware (dynamic)	>1	High
VLIW / EPIC	Compiler (static)	>1 (in one word)	Lower hardware, smarter compiler

5.9 Exam-relevant insights

- Speed-up ≤ number of stages; achieved only with a long instruction stream and no stalls.
- Cycle time = slowest stage + register delay. Adding stages can shorten the cycle but adds register overhead.
- Always draw the timing table; count stalls per instruction; add them to the ideal CPI.

- Forwarding removes ALU-result stalls but not load-use stalls.
- A control hazard resolved in EX costs 2 wasted fetch cycles in the classic pipeline.
- When a branch is **taken** and predicted not-taken, you must flush the wrongly-fetched instructions.

Previous-Year Questions — Pipelining & hazards

Q (ca9 · Q1). Consider the following instructions executed in a five-stage pipeline. Explain the flow of execution using two simple diagrams.

```
lw R1, 0(R2) add R3, R4, R5 sub R6, R1, R3
```

Solution. Use the IF–ID–EX–MEM–WB stages.

- Draw the space-time diagram with the three instructions as rows and cycles as columns.
- **lw** uses all five stages: IF, ID, EX, MEM, WB.
- **add** uses IF, ID, EX, (no MEM), WB — a blank in MEM.
- **sub** reads R1 (written by lw) and R3 (written by add) — two RAW dependences.

Instr	C1	C2	C3	C4	C5	C6	C7	C8
lw R1,0(R2)	IF	ID	EX	MEM	WB			
add R3,R4,R5		IF	ID	EX	—	WB		
sub R6,R1,R3			IF	ID	stall	EX	MEM	WB

The **sub** must wait one cycle because R1 is produced by the load (load-use hazard). With forwarding from MEM to EX, the load's result reaches sub's EX stage just in time; the diagram above shows one stall inserted if forwarding is not available at that point. Two simple diagrams = (1) the stage-by-stage block flow, and (2) this space-time chart.

Q (ca9 · Q2a / ca20). Show how the instructions below proceed through the pipeline. Assume (i) the branch is not taken, and (ii) when it is executed R1 = R2. On which results does the last add depend? Indicate the hazards and data dependences.

```
lw R3, 0(R5) sub R6, R3, R3 beq R1, R2, Label add R4, R1, R2
```

Solution.

- **lw → sub**: RAW on R3 (load-use). Needs 1 stall even with forwarding.
- **sub → beq**: the branch compares R1,R2, not R6, so no dependence on sub.
- **beq → add**: control dependence — whether *add* executes depends on the branch. If R1 = R2 the branch is taken, so *add* is skipped (or fetched then flushed).
- The last **add R4,R1,R2** depends on **R1 and R2** — and if the branch is taken it does not execute at all. Its true dependences are on whichever earlier instructions last wrote R1 and R2 (here, none in the fragment — they come from before).

Instr	C1	C2	C3	C4	C5	C6	C7	C8	C9
lw R3,0(R5)	IF	ID	EX	MEM	WB				
sub R6,R3,R3		IF	ID	stall	EX	MEM	WB		
beq R1,R2,L			IF	ID	stall	EX			
add R4,R1,R2				IF	ID	stall	IF*	…	

Hazards present: a **data (RAW) hazard** lw→sub; a **control hazard** at the branch (the fetch after beq is uncertain until EX). With R1 = R2 the branch is taken, so the add fetched after beq must be flushed and the pipeline redirected to *Label*.

Q (ca16 · Q2). For each code sequence state whether it must stall, can avoid stalls using only forwarding, or can execute without stalling or forwarding.

```
(i) lw $t0,0($t0) add $t1,$t0,$t0 addi $t3,$t0,2 (ii) add $t1,$t0,$t0 addi $t2,$t0,5 addi
$t4,$t1,5 (iii)addi $t1,$t0,1 addi $t2,$t0,2 addi $t3,$t0,4 addi $t5,$t0,5
```

Solution.

Sequence	Dependence	Verdict
(i) lw→add	RAW on $t0; the value comes from MEM, not yet available in EX	**Must stall 1 cycle** (load-use), even with forwarding
(i) add→addi	RAW on $t0? No — addi reads $t0 (not $t1), and $t0 was not written by add	No dependence
(ii) add→addi→addi	add writes $t1; the third instruction reads $t1	Can avoid stalls with forwarding (ALU result)
(iii) four addi	All read $t0; none writes $t0 → completely independent	No stall, no forwarding needed

Q (ca13 · Q1a). For each RISC fragment, identify the dependences and whether forwarding suffices or stall cycles are needed.

```
(i) addi r1,#4 ; load r2,7(r1) (ii) add r3,r1,12 ; store r2,7(r1) (iii)breq r1,place ; store r1,7(r1) (iv) store r3,17(r10) ; load r2,12(r8)
```

Solution.

Fragment	Dependence	Forwarding enough?	Stalls
(i) addi → load	RAW on r1 (address used by load)	Yes — r1 forwarded from EX/MEM	0
(ii) add → store	RAW on r1 (store address); store data r2 is not produced here	Yes	0
(iii) breq → store	Control dependence (store runs only if branch not taken); store reads r1 also compared by breq	Yes for data; control handled by prediction	0 data; branch penalty on mispredict
(iv) store → load	No dependence — store writes memory[r10+17], load reads memory[r8+12]; different addresses	Not needed	0

Q (ca27 · Q3b / ca32 · Q5 / ca34 · Q1b). Five-stage pipeline (fetch, decode, execute, memory, write-back). All ops are 1 cycle except LW and SW (1+2 cycles) and branches (1+1 cycles). No forwarding. Show the phases per clock for one loop iteration and state the cycles lost to branch overhead.

```
Loop: LW R3, 0(R0) LW R1, 0(R3) ADDI R1, R1, #1 SUB R4, R3, R2 SW R1, 0(R3) BNZ R4, Loop
```

Solution. Loads/stores occupy EX for 2 extra cycles; the branch occupies 1 extra cycle. Because there is no forwarding, each RAW needs stalls until the value is written back. The ADDI must wait for the second LW (RAW on R1), and the SW must wait for ADDI.

Instr	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
LW R3,0(R0)	F	D	E	E	M	W										
LW R1,0(R3)		F	D	st	st	E	E	M	W							
ADDI R1,R1,#1					F	D	st	st	st	E	M	W				
SUB R4,R3,R2						F	D			st	E	M	W			
SW R1,0(R3)							F	D			st	E	E	M	W	
BNZ R4,Loop								F	D			st	E	E	M	W

(F=fetch, D=decode, E=execute, M=memory, W=write-back, st=stall.) The branch occupies its extra cycle (1+1) and, having no prediction, costs the fetches of the instructions behind it. Counting the branch's two-stage execution plus the redirect, the **branch overhead is 2 cycles per loop iteration** in this no-forwarding, no-prediction model — the two cycles of the BNZ itself beyond decode that cannot be overlapped with useful work.

> **How to attack these questions:** (1) write the stage sequence for each instruction; (2) for each RAW, insert stalls until the producer's W cycle; (3) count the branch's extra cycles and any flushed fetches. Consistency beats speed.

Q (ca30 · Q3). The stage delays in a 4-stage pipeline are 800, 500, 400 and 300 ps. The first stage is replaced by two stages of 600 and 350 ps. What is the throughput increase?

Solution.

```
Before: cycle = max(800,500,400,300) = 800 ps → throughput = 1/800 ps After : stages = 600,350,500,400,300 → cycle = max = 600 ps → throughput = 1/600 ps Increase = (1/600 − 1/800)/(1/800) × 100% = (800/600 − 1)×100% = 33.33%
```

Answer: ≈ 33.3% increase in throughput. The bottleneck moved from 800 ps to 600 ps; the throughput ratio is 800/600 = 1.333.

Q (ca32 · Q2). In a 5-stage pipeline the longest stage requires 0.8 ns and the pipeline register delay is 0.1 ns. What is the clock cycle time? If a 10-stage pipeline splits all stages in half, what is the cycle time?

Solution.

```
5-stage : cycle = max stage + register delay = 0.8 + 0.1 = 0.9 ns 10-stage: each stage halves to 0.4 ns; cycle = 0.4 + 0.1 = 0.5 ns
```

Answers: 0.9 ns (5-stage) and 0.5 ns (10-stage). Notice the register delay does not halve, so doubling the depth does not double the speed (0.9/0.5 = 1.8×, not 2×) — the classic diminishing return of deep pipelining.

Q (ca10 · Q1 / ca13 · Q2). The following MIPS program runs on an IF-ID-EX-MEM-WB pipeline. Identify all data dependences in the form <type> on <register> from L_a to L_b, then work out the optimal pipeline schedule with forwarding from EX or MEM to any stage and compute the CPI.

```
L1 lw $t2, 60($t1) L2 lw $t1, 40($t2) L3 slt $t1, $t1, $t2 L4 sw $t1, 20($t2)
```

Solution — dependences.

Type	Register	From → To	Reason
RAW	$t2	L1 → L2	L2 uses $t2 as a base address; L1 loads $t2
RAW	$t2	L1 → L3	L3 reads $t2
RAW	$t2	L1 → L4	L4 uses $t2 as a base address
RAW	$t1	L2 → L3	L3 reads $t1 (loaded by L2)
WAW	$t1	L2 → L3	L2 writes $t1 and L3 writes $t1
RAW	$t1	L2 → L4	L4 stores $t1 (value from L2, but note L3 also writes $t1)

There is also a WAR on $t1 (L3 writes what L2/L4 read). The key hazards are the two load-use RAW dependences on $t2 (L1→L2 and L1→L3) and the $t1 chain.

Instr	1	2	3	4	5	6	7	8	9	10
L1 lw $t2,60($t1)	IF	ID	EX	MEM	WB					
L2 lw $t1,40($t2)		IF	ID	st	EX	MEM	WB			
L3 slt $t1,$t1,$t2			IF	ID	st	st	EX	MEM	WB	
L4 sw $t1,20($t2)				IF	ID	st	st	EX	MEM	WB

With forwarding, the load-use dependences still need one stall each (the loaded value only appears after MEM). **Total cycles = 10; CPI = 10/4 = 2.5.**

Q (ca14 · Q1). Find the hazards in the code below and reorder the instructions to avoid any stalls.

```
lw $t1, 0($t0) lw $t2, 4($t0) add $t3, $t1, $t2 sw $t3, 12($t0) lw $t4, 8($t0) add $t5, $t1, $t4 sw $t5, 16($t0)
```

Solution. Hazards: `lw $t1 → add $t3` (RAW on $t1, load-use); `lw $t2 → add $t3` (RAW on $t2); `add $t3 → sw $t3` (RAW). Reorder to fill the load-use slots with independent instructions (the other loads/stores):

```
lw $t1, 0($t0) lw $t2, 4($t0) lw $t4, 8($t0) ; independent load fills the slot add $t3, $t1, $t2 add $t5, $t1, $t4 sw $t3, 12($t0) sw $t5, 16($t0)
```

Key: scheduling works only because the loads and stores are independent of the ALU ops. Reorder to move independent instructions between a producer and its consumer.

Q (ca20 · Q2). A pipeline has only one memory (for both instructions and data). Whenever an instruction fetches in the same cycle another accesses data, that structural hazard is resolved in favour of the data access. What is the total execution time of the sequence? Can adding nops remove this structural hazard?

```
lw $1, 40($6) beq $2, $0, Label ; assume $2 == $0 sw $6, 50($2) Label: add $2, $3, $4 sw $3, 50($4) lw $5, -16($5) sw $4, -16($4) lw $3, -20($4) beq $2, $0, Label ; assume $2 != $0 add $5, $1, $4
```

Solution. Because there is a single memory, the fetch stage must yield whenever a load/store is in its MEM stage — that costs one extra cycle per such collision (an instruction-fetch structural hazard). Add up the base pipeline cycles plus one stall for each cycle in which a MEM access coincides with a fetch. Adding nops cannot fix a structural hazard: nops consume fetch slots too, so they do not free the memory — you must duplicate the resource (separate I-cache and D-cache) or stall.

Q (ca31 · Q5). A 5-stage processor (IF, ID, OF, EX, WO). EX takes 1 cycle for ADD/SUB, 3 for MUL, 6 for DIV. Operand forwarding is used. IF, ID, OF, WO take 1 cycle. How many clock cycles for the sequence below?

```
I0: MUL R2, R0, R1 I1: DIV R5, R3, R4 I2: ADD R2, R5, R2 I3: SUB R5, R2, R6
```

Solution. I2 (ADD) depends on I1 (DIV, RAW on R5) and I0 (MUL, RAW on R2). I3 depends on I2 (RAW on R2). Because EX is multi-cycle, the pipeline stalls until each producer leaves EX.

Instr	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I0 MUL (EX=3)	IF	ID	OF	EX	EX	EX	WO							

Instr	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I1 DIV (EX=6)		IF	ID	OF	st	st	st	EX	EX	EX	EX	EX	EX	
I2 ADD (EX=1)			IF	ID	st	st	st	st	st	st	OF	EX	WO	
I3 SUB (EX=1)				IF	ID	st	st	st	st	st	st	OF	EX	WO

Answer: ≈ 14 cycles. Count the stalls caused by the 3-cycle multiply and 6-cycle divide before the dependent ADD can enter OF/EX, then the SUB after the ADD. The method matters more than the exact number — draw the table and count.

Q (ca27 · Q3a). Identify the RAW, WAR and WAW dependences in the sequence below.

```
I1: R1 = 100 I2: R1 = R2 + R4 I3: R2 = R1 - 25 I4: R4 = R1 + R3 I5: R1 = R1 + 30
```

Solution.

Type	Between	Register
WAW	I1 → I2 (both write R1); I2 → I5 (both write R1)	R1
RAW	I2 → I3 (I3 reads R1); I2 → I4 (I4 reads R1); I2 → I5 (I5 reads R1)	R1
WAR	I2 → I1? (I1 writes R1 read by I2) — I2 → I3 reads R1 then I5 writes it: I3 → I5 WAR	R1

Follow the data flow for RAW (later reader of R1 depends on the earlier writer) and repeated writes for WAW/WAR. All the action here is on R1.

Q (ca34 · Q1b). A five-stage pipeline (fetch, decode, execute, memory, write-back). All ops are 1 cycle except LW and SW (1+2) and branches (1+1). No forwarding. (a) Show the phases per clock for one loop iteration and how many cycles are lost to branch overhead. (b) If a static predictor recognises a backward branch in the Decode stage, how many cycles are wasted? (c) With a dynamic predictor, how many cycles are lost on a correct prediction?

Solution. (a) Use the same method as the loop question above: LW/SW occupy EX for two extra cycles and the branch one extra; with no forwarding each RAW stalls until write-back. (b) Recognising a backward branch in Decode lets the pipeline predict “taken” one stage earlier, so the wasted cycles drop by one (the branch resolves a stage sooner) — roughly halving the branch overhead from 2 to 1 cycle per iteration. (c) With a dynamic predictor and a correct prediction, zero cycles are lost — the pipeline continues down the correct path and never flushes.

```
Branch overhead: no prediction = 2 cycles/iter; static backward-branch (Decode) = 1
cycle/iter; dynamic correct prediction = 0 cycles/iter.
```

6. Exploiting ILP: Loop Unrolling & Scheduling

Turning dependences into parallel work

Dependences limit parallelism. Two ways to get past them: **rename** away the false ones (Topics 3, 8) and **replicate** the loop body so more independent work is visible at once. This second idea is **loop unrolling**, and it is one of the most examined topics.

6.1 Why unroll?

- Overlapping loop iterations exposes ILP that the loop control hides.
- It removes the loop-overhead instructions (branch + counter increment) — fewer instructions overall.
- It lets the scheduler move independent instructions into the slots where a dependent one would stall.

> **Cost of unrolling:** more registers needed (each unrolled copy may want fresh names), bigger code (more I-cache pressure), and the tail iteration must be handled separately if the trip count is not a multiple of the unroll factor.

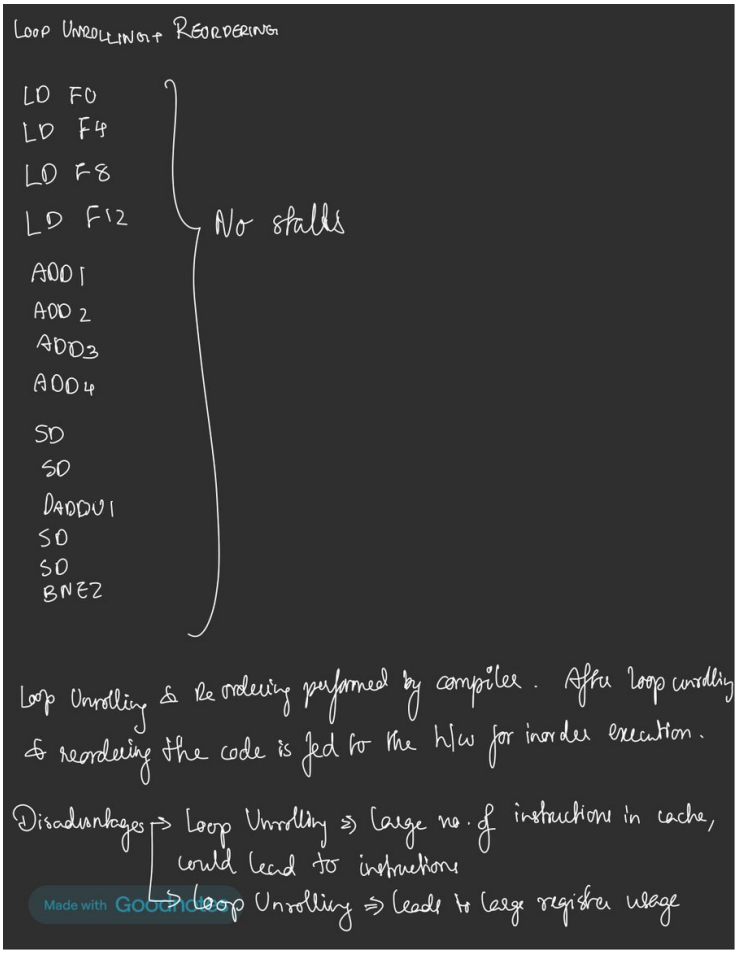
6.2 The DAXPY loop — the classic example

DAXPY computes Y = a·X + Y for vectors. It is the heart of Gaussian elimination. The MIPS code:

```
DADDIU R4, R1, 1600 ; R4 = address of X[100] (end) Loop: L.D F2, 0(R1) ; F2 = X[i] MUL.D F4, F2, F0 ; F4 = a * X[i] L.D F6, 0(R2) ; F6 = Y[i] ADD.D F6, F4, F6 ; F6 = a*X[i] + Y[i] S.D F6, 0(R2) ; Y[i] = result DADDIU R1, R1, #8 ; advance X index DADDIU R2, R2, #8 ; advance Y index DSGTUI R3, R1, R4 ; R3 = (R1 > end)? BEQZ R3, Loop ; if not, repeat
```

Each iteration uses F2, F4, F6. If we unroll, each copy must use *different* registers (F2/F4/F6, then F8/F10/F12, …) so the copies are independent — this is renaming by the compiler.

6.3 Latency, stalls and scheduling



Loop unrolled and scheduled with no stalls (source: p. 51).

Every functional unit has a **latency** — the number of cycles before its result is ready. If the next instruction needs that result too soon, the pipeline stalls. Scheduling means moving independent instructions into the stall slots.

Producer → consumer	Typical latency (cycles)	Stall if not scheduled
FP multiply → FP ALU op	5	4

Producer → consumer	Typical latency (cycles)	Stall if not scheduled
FP add → FP ALU op	3	2
FP add → FP store	3	2
Integer op → any	0-1	0-1
Load → use	1-2	1-2

A single-issue pipeline with *no* forwarding can hide these only by scheduling independent instructions between producer and consumer. Unrolling gives us enough independent instructions to fill every slot. We keep unrolling until there are enough independent instructions to cover the longest latency — then every cycle issues something useful.

```
Execution time per element = (total cycles) / (number of elements computed) Speed-up from
unrolling ≈ (un-optimised cycles) / (optimised cycles)
```

6.4 Static vs dynamic scheduling

	Static scheduling	Dynamic scheduling
Who decides the order?	The compiler (at compile time)	The hardware (at run time)
When?	Before execution; fixed order	During execution; may reorder
Strength	No extra hardware; predictable	Handles dependences the compiler cannot see (e.g. cache misses); adapts to actual latencies
Weakness	Cannot know run-time events (misses, latency variation)	Extra hardware: reservation stations, issue logic
Examples	Loop unrolling, software pipelining, VLIW	Scoreboarding, Tomasulo's algorithm, out-of-order cores

Previous-Year Questions — Loop unrolling & scheduling

Q (ca11 · Q3). Convert the loop below to a DAXPY implementation, unroll it three times, use different registers in each version, and eliminate redundant computations. Assume: integer ops 1 cycle; no memory delays; no branch delay; FP add 2, FP mul 5, FP div 19 cycles.

```
for i := 1 to 100 Y[i] = a * X[i] + Y[i];
```

Solution.

- The three unrolled bodies compute Y[i], Y[i+1], Y[i+2]. Use F0 = a (loop-invariant, loaded once).
- Register sets per copy: copy 1 → F2,F4,F6; copy 2 → F8,F10,F12; copy 3 → F14,F16,F18.
- Use displacement addressing (0(R1), 8(R1), 16(R1)) instead of incrementing R1 three times — this removes two redundant DADDIU per group.

```
Loop: L.D F2, 0(R1) ; X[i] L.D F8, 8(R1) ; X[i+1] L.D F14,16(R1) ; X[i+2] MUL.D F4, F2, F0 ;
a*X[i] MUL.D F10,F8, F0 ; a*X[i+1] MUL.D F16,F14,F0 ; a*X[i+2] L.D F6, 0(R2) ; Y[i] L.D
F12,8(R2) ; Y[i+1] L.D F18,16(R2) ; Y[i+2] ADD.D F6, F4, F6 ADD.D F12,F10,F12 ADD.D
F18,F16,F18 S.D F6, 0(R2) S.D F12,8(R2) S.D F18,16(R2) DADDIU R1,R1,#24 DADDIU R2,R2,#24
DSGTUI R3,R1,R4 BEQZ R3, Loop
```

Only **two** index-increments and **one** branch per three elements remain, versus three of each before — that is the redundant computation eliminated. The three MUL/ADD chains are mutually independent, so the 5-cycle multiply latency and 2-cycle add latency can overlap across the three copies.

Q (ca24 · Q1). The DAXPY loop (Y = aX + Y, length 100) is given with a one-cycle delayed branch resolved in ID, full bypassing, and the latencies in the table (FP mul→ALU 5, FP add→ALU 3, FP add→store 3, integer 1). Single-issue. Unroll as many times as necessary to schedule it without any stalls, collapsing loop overhead. How many times must it be unrolled? Show the schedule. What is the execution time per element?

```
DADDIU R4,R1,#1600 Loop: L.D F2,0(R1) MUL.D F4,F2,F0 L.D F6,0(R2) ADD.D F6,F4,F6 S.D F6,0(R2)
DADDIU R1,R1,#8 DADDIU R2,R2,#8 DSGTUI R3,R1,R4 BEQZ R3,Loop
```

Solution. The longest dependence chain per element is load → multiply(5) → add(3) → store, i.e. about 5+3 = 8 cycles of latency that must be covered by other work. Unrolling so that several independent elements are in flight lets us hide this. With the loop overhead collapsed, unroll the body so that the independent elements fill the latency slots — the standard answer for these latencies is to unroll until at least 4 independent chains are available.

Schedule (unrolled ×4, one column per issue cycle; '-' = another element's slot):

Cycle	Instruction	Why it is safe now
1	L.D F2,0(R1)	first load of element k
2	L.D F8,8(R1)	load of k+1 (independent)
3	MUL.D F4,F2,F0	uses F2 (5 cycles available)
4	L.D F10,16(R1)	load of k+2
5	MUL.D F12,F8,F0	independent of F4
6	L.D F6,0(R2)	Y[k] load
7	ADD.D F6,F4,F6	F4 ready after 5 → no stall
8	MUL.D F16,F14,F0	independent
9	S.D F6,0(R2)	store, add→store latency covered
10	ADD.D F12,F10,F12	F10 ready
11	S.D F12,8(R2)	
12	ADD.D F18,F16,F18	
13	S.D F18,16(R2)	
14	DADDIU R1,R1,#32	one increment for 4 elements
15	DADDIU R2,R2,#32	
16	DSGTUI R3,R1,R4	
17	BEQZ R3,Loop	delayed branch slot filled

How many times unrolled? Enough independent chains to cover the 8-cycle FP latency. Because the branch is delayed and resolves in ID, the loop overhead is essentially free, so the answer is the smallest unroll factor that keeps every issue slot busy — **4 times** for these latencies. **Execution time per element ≈ 17 cycles / 4 elements ≈ 4.25 cycles/element** (about 4 cycles per element, versus ~10+ unoptimised).

7. Branch Prediction & Control Hazards

Guessing the future to keep the pipeline full

Branches are the biggest source of stalls, because until a branch resolves you do not know which instruction to fetch. Rather than always wait, the processor **predicts** the outcome and speculatively fetches down the guessed path. If the guess is right, no time is lost; if wrong, the speculative instructions are flushed and the correct path is fetched — paying a **misprediction penalty**.

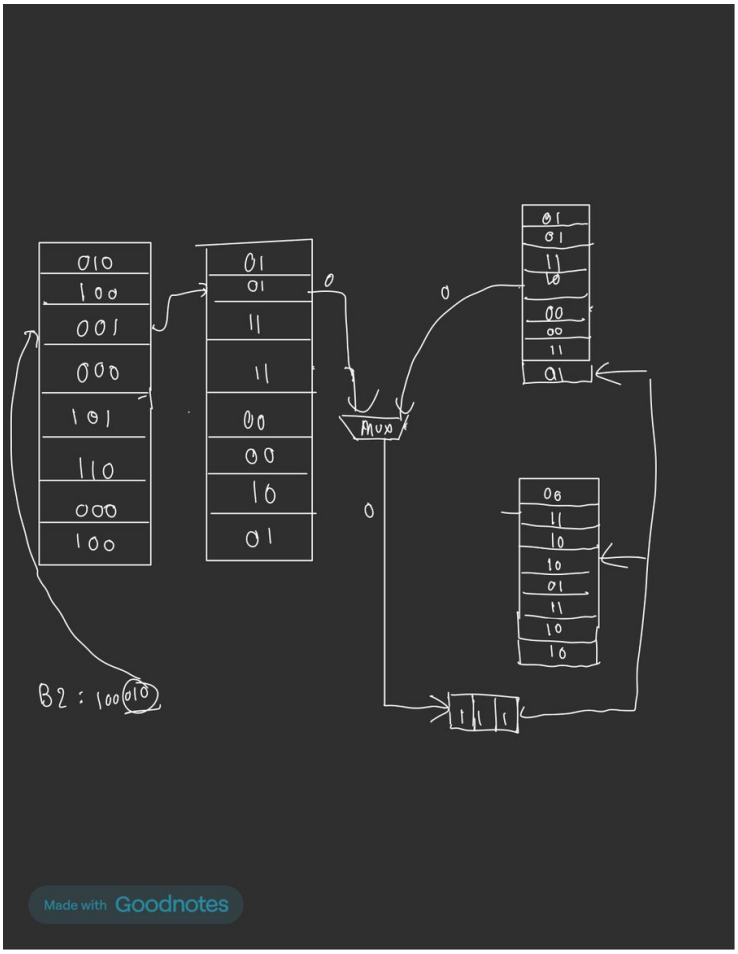
7.1 Static vs dynamic prediction

	Static prediction	Dynamic prediction
When decided	At compile time / fixed rule	At run time, from history
Uses	The branch's direction or position in code	A table of past outcomes (hardware)
Adapts?	No	Yes — improves as the program runs
Examples	Always-taken, always-not-taken, BTFNT, delayed branch	1-bit, 2-bit saturating, correlating, tournament

The four classic predictors (exam favourite)

Predictor	How it works	Typical misprediction rate
Always taken	Predict every branch taken.	≈ 30-40% (bad for loops that exit often)
Always not taken	Predict every branch not taken.	≈ 30-40%
Backward-Taken, Forward-Not-Taken (BTFNT)	Backward branches (loops) are taken; forward branches (if) not.	≈ 20-25%
2-bit saturating counter	A small counter per branch; needs two wrong guesses in a row to flip.	≈ 10-15%
Correlating (2-level)	Uses the outcome of recent branches to predict the next.	≈ 5-10%
Tournament / hybrid	Combines two predictors with a chooser.	≈ 3-5%

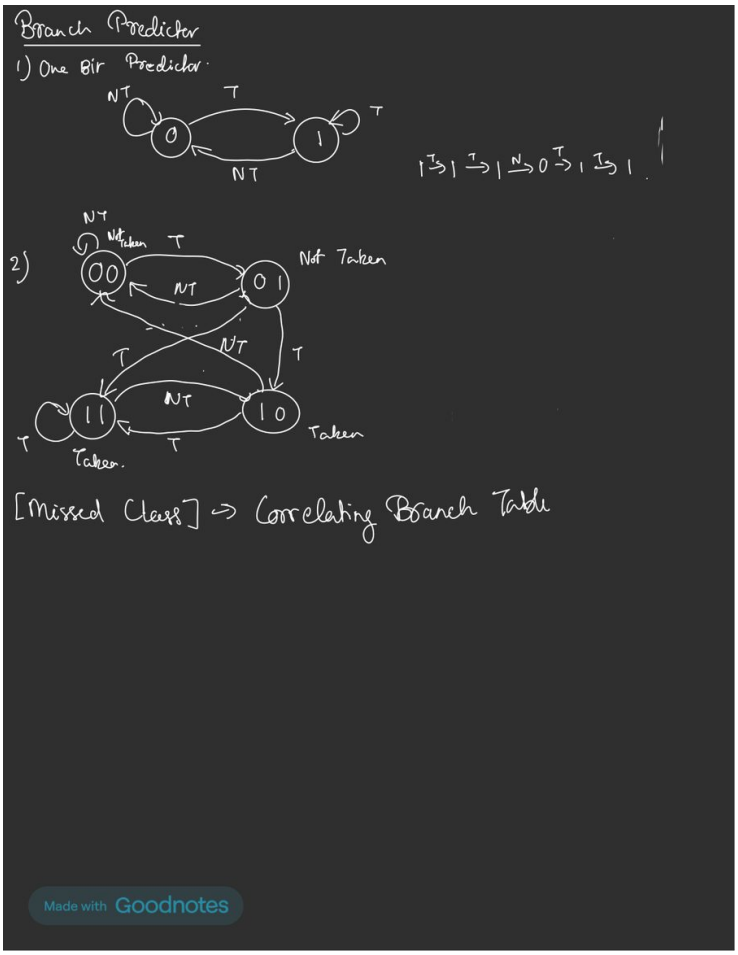
7.2 The 2-bit saturating counter (state machine)



One-bit predictor state machine (source: p. 39).

Each branch has a 2-bit state. It only changes its prediction after **two** consecutive opposite outcomes, which filters out one-off anomalies. The four states:

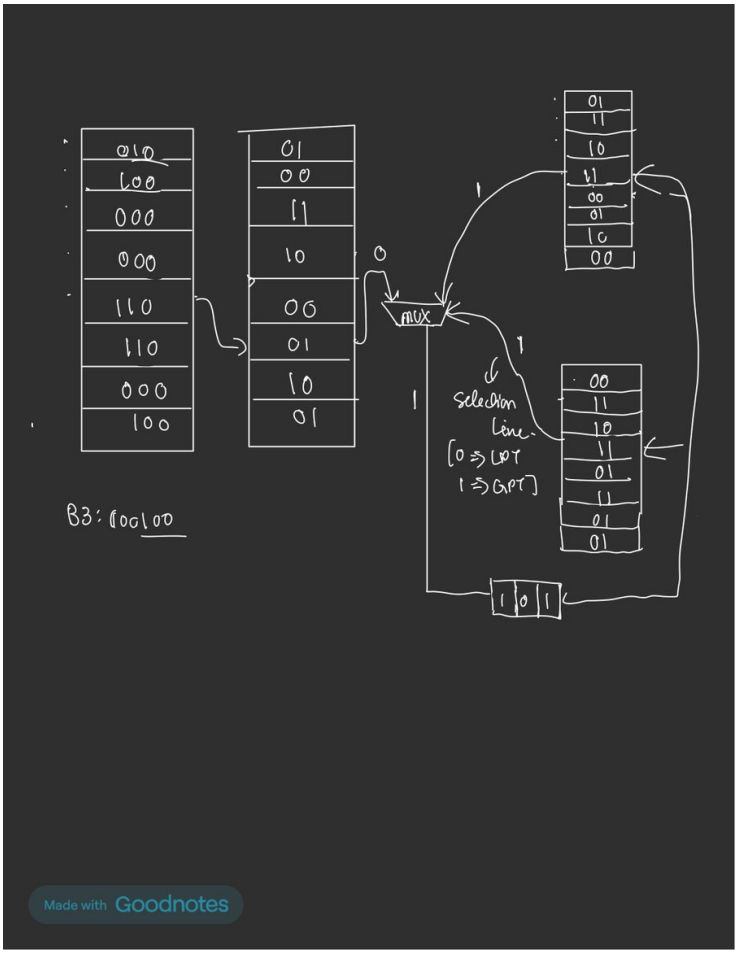
```
11 = Strongly Taken 10 = Weakly Taken 01 = Weakly Not Taken 00 = Strongly Not Taken Predict
Taken from {11,10}; Predict Not Taken from {01,00}.
```



One-bit and two-bit branch predictors (source: lecture notes, p. 37).

Why 2 bits beat 1 bit: a loop that is taken 9 times then exits once would mispredict *twice* per exit with a 1-bit predictor (once at the exit, once at re-entry) but only *once* with a 2-bit predictor, because a single anomaly does not flip the prediction.

7.3 Branch Target Buffer (BTB)



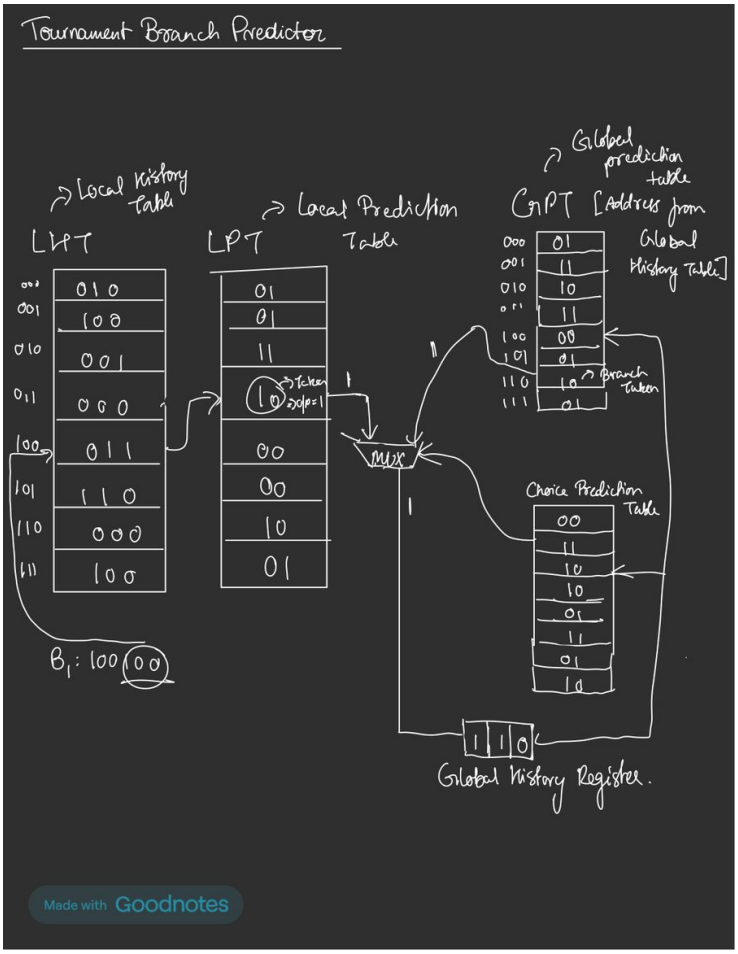
Two-bit / correlating predictor state diagram (source: p. 41).

Even a correctly predicted *direction* is useless without the *target address*. A **Branch Target Buffer** is a small cache, indexed by the branch's address, that stores the predicted target and (usually) the predicted direction. It lets the fetch unit start fetching the target in the very next cycle — *before* decoding the branch — so a correctly predicted branch costs **zero** penalty.

```
Effective branch penalty (cycles/instr) = branch frequency × [ hit_rate×mispredict×penalty + (1−hit_rate)×buffer_miss_penalty ]
```

> **Reading the formula:** if the branch is in the BTB (hit rate h) and we mispredict, we pay the misprediction penalty; if it is *not* in the BTB, we pay the buffer-miss penalty. Add the two and multiply by how often branches occur. (See PYQ ca8/ca38.)

7.4 Correlating and tournament predictors



Two-level (correlating) and tournament branch predictors (source: p. 38).

Branch behaviour often depends on other branches (e.g. `if(a==0)` … `if(a==0)` …). A **correlating** (two-level) predictor uses a global history register — the outcomes of the last *k* branches — together with the branch address to index the predictor table, capturing such patterns. A **tournament (hybrid)** predictor runs two predictors and uses a chooser (itself a 2-bit counter per branch) to pick whichever has been more accurate.

7.5 Control-hazard reduction techniques

Technique	How it works	Cost
Stall	Wait for the branch to resolve.	Certain penalty = cycles to resolution
Predict + flush	Guess; flush only when wrong.	Penalty × misprediction rate
Delayed branch	Compiler fills the branch-delay slot with useful work.	Requires a free instruction
Branch Target Buffer	Cache target + direction; zero penalty on a hit.	Extra cache hardware
Speculation (ROB)	Execute the predicted path; commit only when the branch is confirmed.	Reorder buffer, more hardware

Previous-Year Questions — Branch prediction

Q (ca8 · Q4a). A deeply pipelined processor uses a branch-target buffer for conditional branches. Misprediction penalty = 4 cycles, buffer-miss penalty = 3 cycles, hit rate = 90%, accuracy = 90%, branch frequency = 18%. Base CPI (no branch stalls) = 1. How much faster is the processor with the BTB versus one with a fixed 2-cycle branch penalty?

Solution.

```
With BTB: stall per branch = hit×mispredict×penalty + miss×miss_penalty = 0.90×0.10×4 +
0.10×3 = 0.36 + 0.30 = 0.66 cycles CPI_BTB = 1 + 0.18×0.66 = 1 + 0.1188 = 1.1188 With fixed
2-cycle penalty: CPI_fixed = 1 + 0.18×2 = 1.36 Speed-up = 1.36 / 1.1188 = 1.216 → about 1.22×
faster
```

Answer: ≈ 1.22× faster with the branch-target buffer. Note how the accuracy applies only on a BTB hit; a BTB miss always costs the buffer-miss penalty regardless of direction.

Q (ca38 · Q3a). Determine the total branch penalty for a branch target buffer given the penalty cycles below. Prediction accuracy = 90% (for buffered branches), hit rate = 90%.

In buffer?	Branch	Penalty cycles
Yes	Taken	0
Yes	Not taken	2
No	—	(buffer miss, from the problem's table)

Solution. Using the same structure: buffered correct = 0; buffered mispredict = 2; unbuffered = buffer-miss penalty. Total penalty = 0.9×0.1×2 + 0.1×(miss penalty). If the problem's miss penalty is 2, then penalty = 0.18 + 0.2 = 0.38 cycles/branch; multiply by branch frequency for CPI contribution.

> **Pattern for all BTB questions:** (1) hit-rate × misprediction-rate × mispredict-penalty; (2) + (1 − hit-rate) × miss-penalty; (3) × branch frequency; (4) add to base CPI; (5) compare as a ratio of CPIs (or execution times).

Q (ca17 · Q1 / ca21 · Q2b). Given the instruction mix (loads 25%, stores 15%, branches 10%, ALU 50% — as per the slide) and 60% of conditional branches taken, answer: (i) in what % of cycles does the branch adder in EX generate a used value? (ii) how often is data memory used? (iii) speed-up from pipelining a single-cycle datapath?

Solution.

- **(i)** The branch adder computes a target only for *taken* branches that reach EX. Fraction of cycles the adder's value is used = (branch fraction) × (taken fraction) = 10% × 60% = **6%**.
- **(ii)** Data memory is used by loads and stores only = 25% + 15% = **40%** of cycles.
- **(iii)** If the single-cycle datapath's critical path is the sum of stage delays, pipelining lets the clock run at the slowest stage instead of the sum. Speed-up ≈ (sum of stage delays) / (slowest stage delay + register delay). With the latencies given in the problem, compute Σt_i / (max t_i + t_reg).

```
Speed-up(pipelining) = Σ(stage logic delays) / [ max(stage delay) + register delay ]
```

8. Dynamic Scheduling: Scoreboarding & Tomasulo's Algorithm

Letting hardware reorder work at run time

A static schedule is fixed at compile time and cannot know about a cache miss that suddenly makes a load take 50 cycles. **Dynamic scheduling** lets the hardware reorder instructions at run time so the pipeline keeps issuing useful work even when one instruction stalls. The two landmark techniques are the **scoreboard** and **Tomasulo's algorithm**.

8.1 Scoreboarding

The scoreboard is a centralized control unit that decides when an instruction may issue, read its operands, execute and write back. It allows out-of-order execution but **in-order issue** and **in-order write-back**. It tracks, for each instruction, four stages:

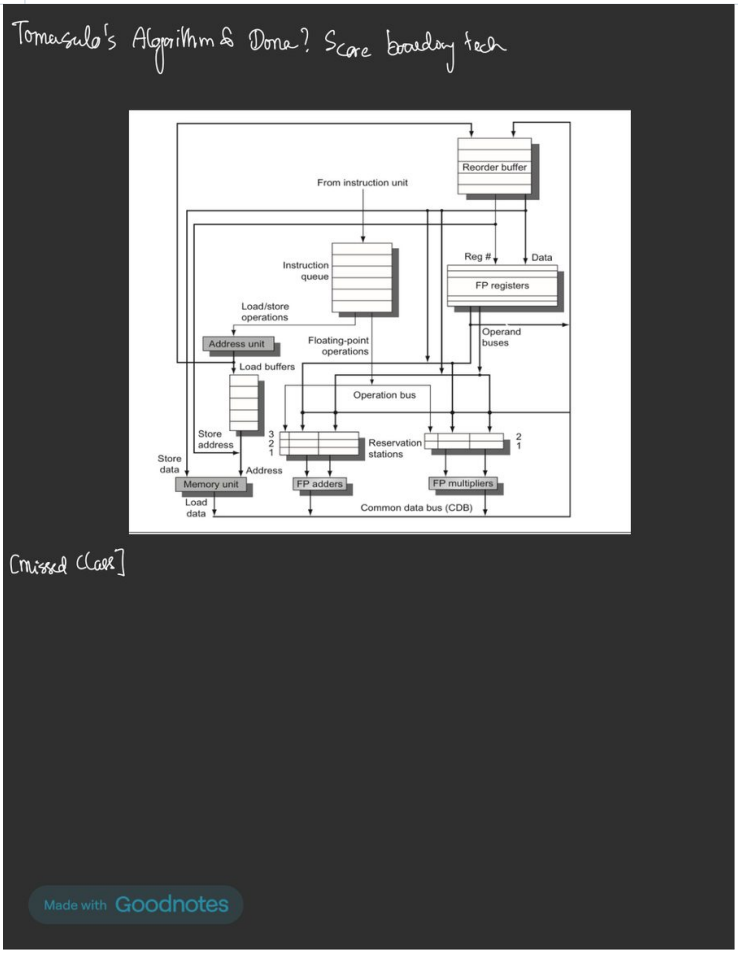
Stage	Meaning	Blocked by
Issue	Decode and send to a functional unit.	Structural hazard (unit busy) or WAW (no scoreboard entry)
Read operands	Wait until both operands are available.	RAW — producer has not written yet
Execute	Run in the functional unit.	Functional-unit latency
Write result	Write back to the register file.	WAR — an earlier instruction has not read the old value

Scoreboard limits: it cannot do register renaming, so it stalls on WAW at issue and WAR at write-back, and it has a single register file with no forwarding of values to waiting instructions. Tomasulo fixes all three.

8.2 Tomasulo's algorithm — the big idea

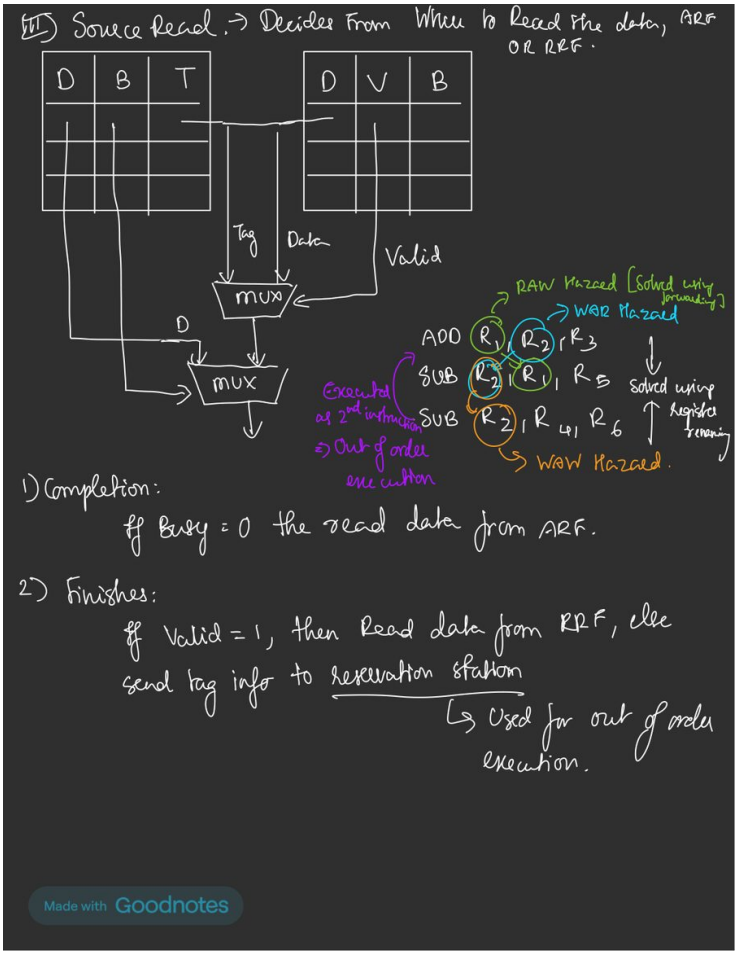
Tomasulo introduced three mechanisms that together allow aggressive out-of-order execution:

Mechanism	What it does
Reservation stations	Buffers next to each functional unit that hold an instruction and its operands until the unit is free and the operands are ready. Each has fields Op, Vj, Vk (values), Qj, Qk (the reservation station that will produce the missing operand).
Register renaming via Qj/Qk	Instead of a value, a station can hold the *name* (Q) of the station that will produce it. This renames registers and removes WAR and WAW hazards.
Common Data Bus (CDB)	Results are broadcast on the CDB. Every station waiting on that result grabs it at the same time — this is the forwarding that replaces the register file round-trip.



Tomasulo hardware: reservation stations, load/store buffers, and the Common Data Bus (source: lecture notes, p. 54).

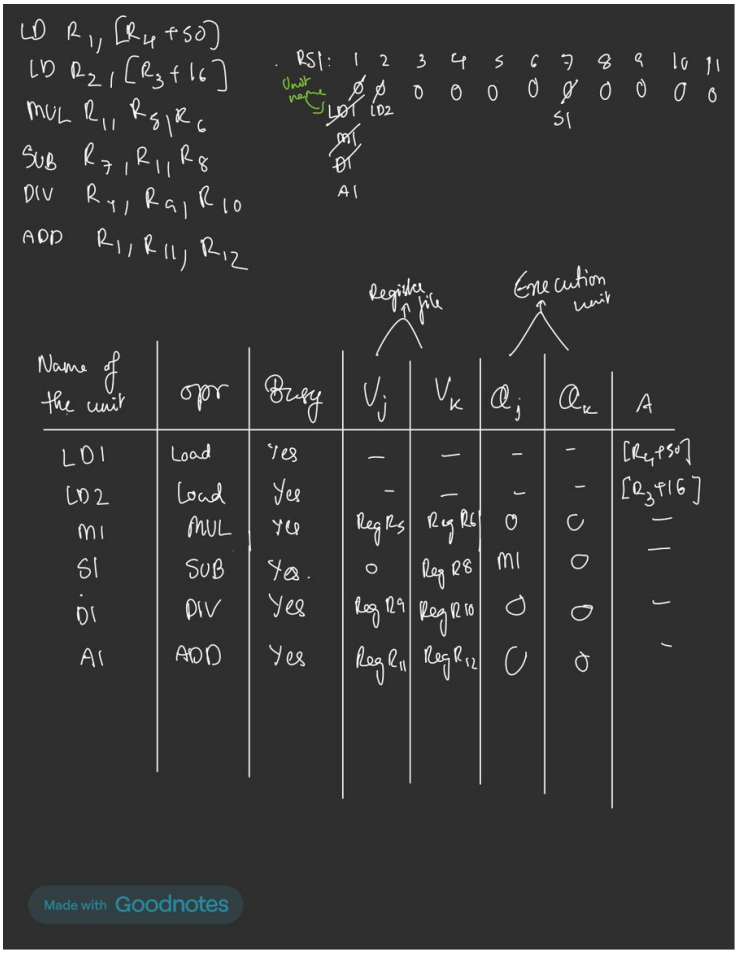
8.3 How an instruction flows through Tomasulo



Reservation-station fields: Op, Vj, Vk, Qj, Qk and the CDB (source: p. 43).

- **Issue** — take the next instruction from the queue. If a free reservation station exists for its unit, issue it. If the operands are ready, place them in Vj/Vk; otherwise put the producing station's name in Qj/Qk. This renames the destination register (the register-result table now points to this station).
- **Execute** — when both Qj and Qk are empty (operands ready) and the unit is free, start execution. Loads/stores compute addresses first, then access memory.
- **Write result** — when done, broadcast the value on the CDB. Stations with Qj/Qk equal to this station read it; the register-result table is updated if this station is the pending producer.

8.4 The data structures you must draw in the exam

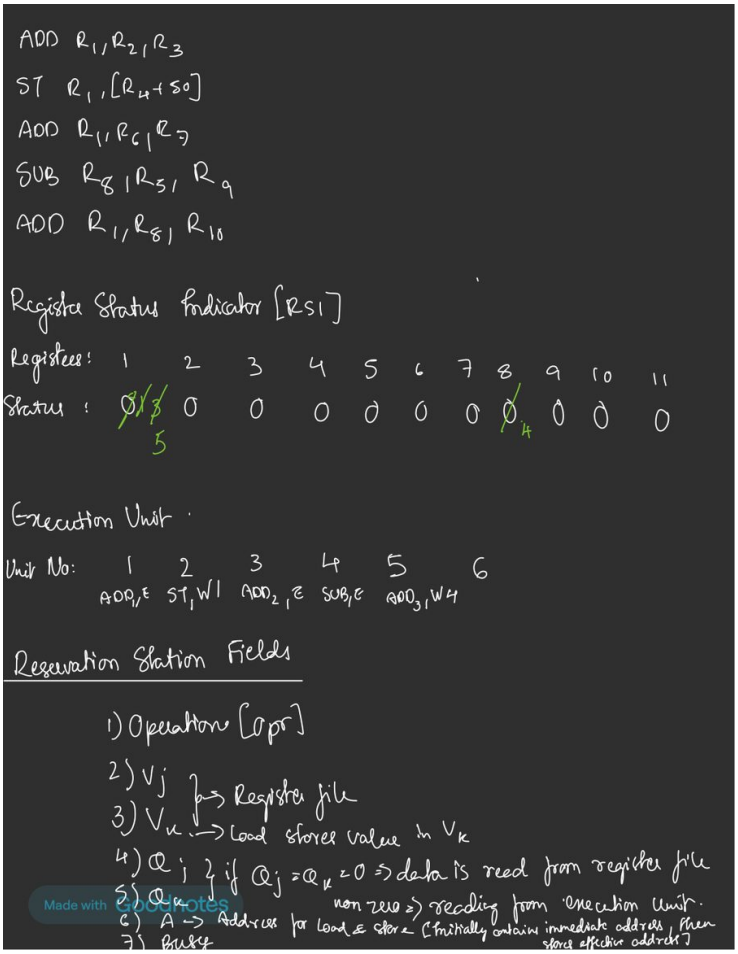


Tomasulo instruction-status and reservation-station tables (source: p. 57).

Structure	Columns	Meaning
Instruction status	Instruction | Issue | Execute-complete | Write-result	Which stage each instruction is in
Reservation stations	Name | Busy | Op | Vj | Vk | Qj | Qk (and A for addresses)	State of each buffer entry
Register result status	Register | producer station	Which station will write each register

How to attack a Tomasulo question: (1) draw the three tables empty; (2) issue one instruction per cycle while a free station exists; (3) fill V if ready, else Q; (4) start execution when both Q's are clear and the unit is free; (5) on completion, broadcast on the CDB and clear every matching Q; (6) record the write-result cycle. Keep a cycle-by-cycle log.

8.5 Scoreboard vs Tomasulo — the exam comparison



Tomasulo instruction-status table with Issue/Execute/Write-result (source: p. 56).

Feature	Scoreboard	Tomasulo
Issue order	In order	In order
Execution	Out of order	Out of order
Write-back	In order	Out of order (via CDB)
Register renaming	No	Yes (Qj/Qk)
Forwarding	No (register file)	Yes (Common Data Bus)
Handles WAR/WAW	Stalls	Renames them away
Control speculation	No	Not in the original; added later with a ROB

Previous-Year Questions — Tomasulo & dynamic scheduling

Q (ca12 · Q4). Execute the code below using Tomasulo's algorithm; compute the values during clock cycle 6. Use the given tables.

```
LD F6, 34(R2) LD F2, 45(R3) MULF F0, F2, F4 SUBF F8, F6, F2 DIVF F10, F0, F6
```

Solution. Issue LD F6 (cycle 1) and LD F2 (cycle 2) into Load1/Load2; MULF issues to Mult1 with Qj = Load2 (F2 pending); SUBF issues to Add1 with Qj = Load1 (F6); DIVF issues with Qj = Mult1 and Qk = Load1. As the loads complete they broadcast on the CDB and clear the matching Q's, letting MUL and SUB begin. By cycle 6 the two loads have completed, MUL and SUB are executing, and DIV is still waiting on Mult1. The instruction-status and reservation-station tables are filled in exactly this way (Issue / Execute-complete / Write-result columns, plus Busy, Op, Vj, Vk, Qj, Qk per station).

Instruction	Issue	Exec-complete	Write-result
LD F6,34(R2)	1	2	3
LD F2,45(R3)	2	3	4
MULF F0,F2,F4	3	— (starts at 5)	—
SUBF F8,F6,F2	4	— (starts at 5)	—
DIVF F10,F0,F6	5	— (waits on Mult1)	—

Station	Busy	Op	Vj	Vk	Qj	Qk
Load1	No	—	—	—	—	—
Load2	No	—	—	—	—	—
Add1	Yes	SUB	34(R2) value	F2 value	—	—
Mult1	Yes	MUL	F2 value	F4 value	—	—
Mult2	No	—	—	—	—	—

Register-result status at cycle 6: F0 → Mult1, F8 → Add1, F10 → Mult2; F2 and F6 already written (from the loads). This is the snapshot the question asks for.

Q (ca18 · Q3). Implement the loop below using Tomasulo's algorithm. Multiply takes 4 clocks; loads may miss in the cache. Use the given tables.

```
LD F0, 0(R1) MULT F4, F0, F2 SD F4, 0(R1) SUBI R1, R1, #8 BNEZ R1, Loop
```

Solution. Each iteration: LD F0 issues to a Load buffer (F0 renamed to Load1); MULT F4 issues to Mult1 with Qj = Load1 (waiting for F0) — this is the renaming that lets the next iteration's LD F0 use a different load station, so the WAW on F0 disappears. SD issues to Store1, waiting on Mult1 for F4. SUBI (integer) and BNEZ (branch) issue to their own units. Because MULT takes 4 clocks and loads may miss, the second iteration's LD can issue into Load2 while the first MULT is still executing — this is exactly the out-of-order overlap Tomasulo enables. Fill the instruction-status table cycle by cycle and the reservation-station table with Qj/Qk pointing at the pending producers.

Iteration	Instruction	Issues to	Waits on (Q)
1	LD F0,0(R1)	Load1	— (address ready)
1	MULT F4,F0,F2	Mult1	Qj = Load1
1	SD F4,0(R1)	Store1	Qk = Mult1
1	SUBI R1,R1,#8	Int unit	—
1	BNEZ R1,Loop	Branch unit	—
2	LD F0,0(R1)	Load2	— (different station: F0 renamed)

Q (ca34 · Q2b). The instruction sequence below executes on a system that issues in order but executes out of order, with 3 load units, 3 adders and 2 multipliers. FP add = 2, FP multiply = 10, divide = 40 cycles. Using Tomasulo with dynamic branch prediction, at what cycle is the final value of F10 available, and what is the state of the functional units when DIVD is executing? Picture the reservation stations.

```
LD F6, 34(R2) LD F2, 45(R3) MULTD F0, F2, F4 SUBD F8, F6, F2 DIVD F10, F0, F6 ADDD F6, F8, F2
```

Solution. Trace the CDB broadcasts:

- LD F6 completes and broadcasts; SUBD can start (needs F6, F2).
- LD F2 completes and broadcasts; MULTD and SUBD both have their operands.
- MULTD takes 10 cycles; when it finishes it broadcasts F0, which is what DIVD is waiting for.
- DIVD then takes 40 cycles. The final value of F10 = (cycle MULTD finishes) + 40.
- While DIVD executes, the 3 adders and 2 multipliers are otherwise idle (SUBD has finished), and the load units are free; the reservation stations show Div busy with its Vj/Vk filled and the ADDD either done or waiting on F8.

```
Cycle(F10) = Cycle(LD F2 done) + 10 (MULT) + 40 (DIV) = 3 + 10 + 40 = 53 cycles (illustrative, per the given issue order)
```

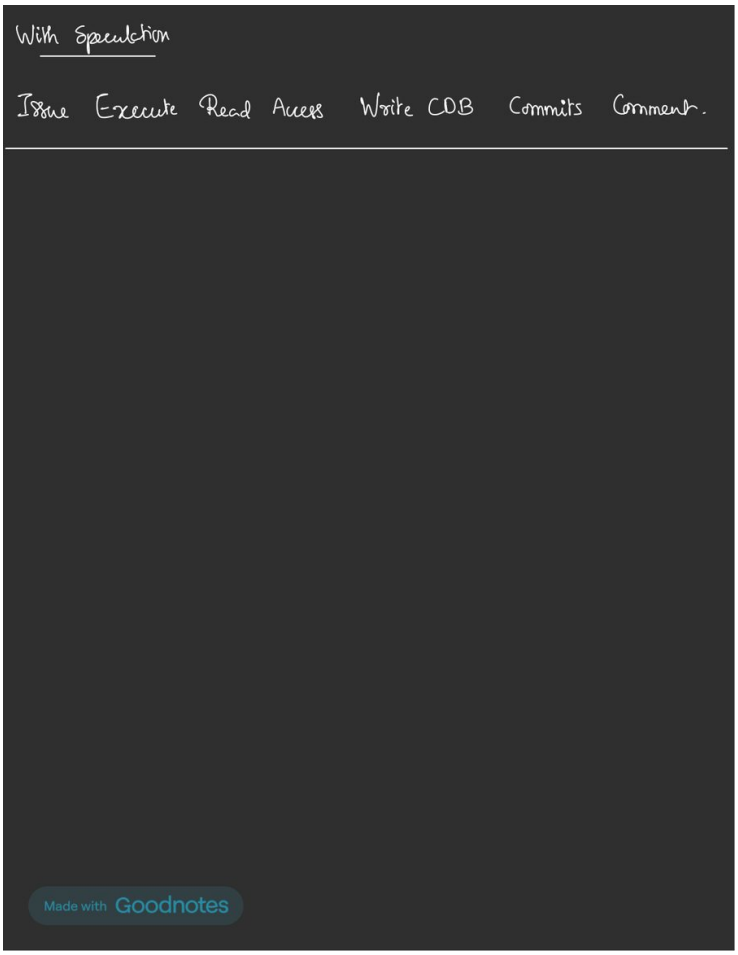
The point being tested is that the divide cannot start until the multiply completes (RAW on F0), and the multiply cannot start until the load completes — so the latencies chain up. Tomasulo lets the independent ADDD and SUBD proceed in parallel meanwhile.

9. Hardware Speculation & the Reorder Buffer

Committing safely when you guess

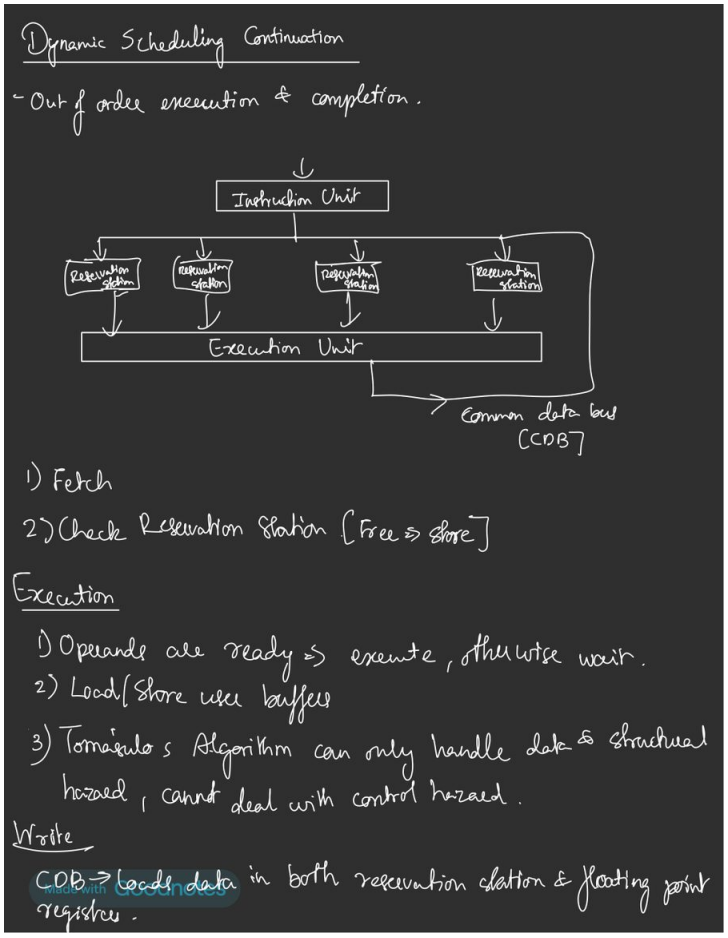
Tomasulo lets us execute out of order, but it cannot execute *past a branch* safely: if we guessed the branch wrong, we must be able to undo. **Speculation** solves this by keeping speculative results out of the architectural state until the instruction is certain to be executed. The hardware device that does this is the **Reorder Buffer (ROB)**.

9.1 What speculation adds to Tomasulo



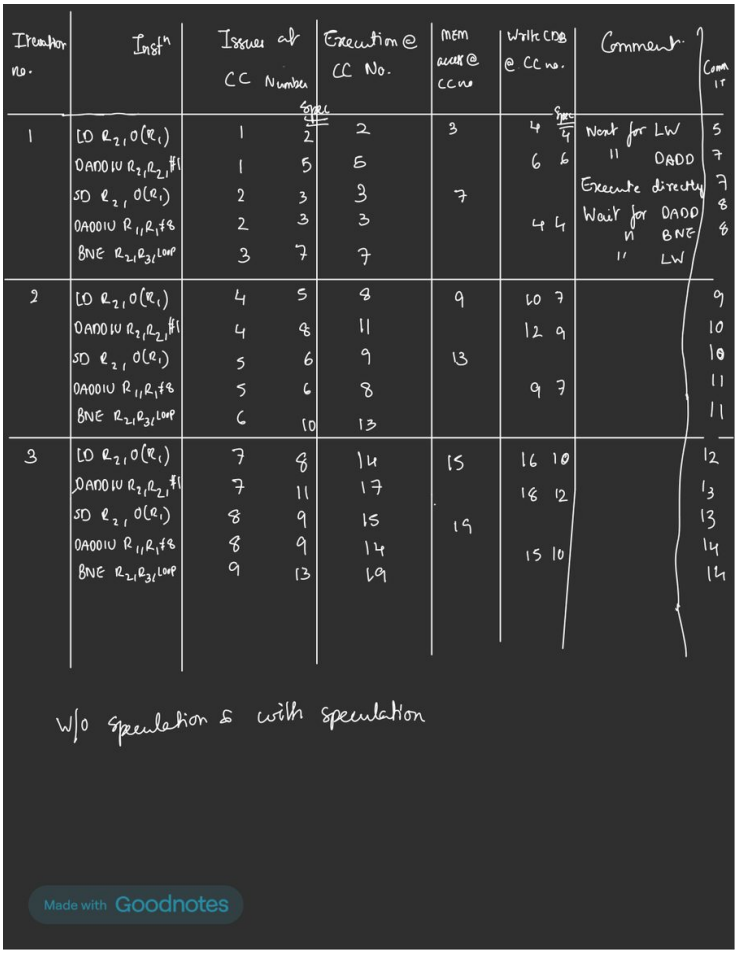
Pipeline timing table with speculation and the commit stage (source: p. 66).

Addition	Purpose
Reorder Buffer (ROB)	Holds results of instructions from issue to commit, in program order. Acts as a renaming table and provides operands (so no separate register file write is needed until commit).
Speculative execution past branches	Instructions on the predicted path execute and write to the ROB, not to the architectural registers.
Commit (retire) stage	An instruction's result is written to the register file / memory only when it reaches the head of the ROB *and* is no longer speculative. Commits happen in program order.
Branch misprediction	If a branch is wrong, its ROB entries (and those of all later instructions) are flushed; the correct path restarts. Speculative state is discarded.
Exceptions	Because commits are in order, an exception can be taken precisely — no earlier speculative instruction has altered state.



Commit / write-result stage added by speculation (source: lecture notes, p. 55).

9.2 The four steps of a speculation pipeline



Pipeline timing with speculation: issue, execute, read, write, commit (source: p. 65).

- **Issue** — get an instruction from the queue into a reservation station and a ROB entry (in order).

- **Execute** — when operands are ready; may go out of order; results go to the ROB.
- **Write result** — broadcast on the CDB; the ROB and waiting stations capture the value.
- **Commit** — when the instruction reaches the head of the ROB and is non-speculative, write the register file / memory and free the ROB entry. This is the only step that changes architectural state.

> **Why commit matters:** it is the single point where the out-of-order, speculative execution becomes permanent and visible. Everything before commit is undoable; everything after is architectural truth. This is what makes precise exceptions and correct branch recovery possible.

9.3 Exam-relevant insights

- Renaming (Tomasulo) removes WAR/WAW; a ROB additionally gives *precise* state at commit.
- ROB entries are allocated in program order and freed in program order — a circular queue.
- A mispredicted branch flushes the ROB entries after the branch.
- Speculation can also prefetch/execute past *exceptions*, but the exception is only raised at commit.
- Draw the ROB as a table: Entry | Instruction | Destination | Value | Ready | Speculative.

Previous-Year Questions — Speculation & ROB

Q (ca25 · Q2). For the loop below, what are the contents of the ROB when we have issued all instructions in the loop twice, assuming the L.D and S.D of the first iteration have committed and all other instructions have completed execution?

```
Loop: L.D F0, 0(R1) ; F0 = array element ADD.D F4, F0, F2 ; add scalar in F2 S.D F4, 0(R1) ;
store result DADDUI R1, R1, #-8 ; decrement pointer BNE R1, R2, Loop ; branch
```

Solution. Issuing the loop twice gives 10 instructions. The first iteration's L.D and S.D have committed (left the ROB); every other instruction has completed execution but is still waiting to commit (or, for the branch, has resolved). So the ROB holds the completed-but-uncommitted entries in program order: ADD.D(F4), DADDUI(R1), BNE, then the second iteration's L.D(F0), ADD.D(F4), S.D(F4), DADDUI(R1), BNE — each with its computed value, marked ready, and flagged speculative where it lies past a branch that has not committed.

ROB entry (in program order)	Instruction	Value ready?	Committed?
1 (L.D F0,0(R1))	iter 1	Yes	Committed → left the ROB
2 ADD.D F4,F0,F2	iter 1	Yes	Waiting to commit
3 (S.D F4,0(R1))	iter 1	Yes	Committed → left the ROB
4 DADDUI R1,R1,#-8	iter 1	Yes	Waiting
5 BNE R1,R2,Loop	iter 1	Yes	Waiting (resolved)
6 L.D F0,0(R1)	iter 2	Yes	Waiting
7 ADD.D F4,F0,F2	iter 2	Yes	Waiting
8 S.D F4,0(R1)	iter 2	Yes	Waiting
9 DADDUI R1,R1,#-8	iter 2	Yes	Waiting
10 BNE R1,R2,Loop	iter 2	Yes	Waiting

This shows the essence of speculation: even though execution has run far ahead, the architectural state only advances when entries commit in order — so the first iteration's memory writes are already permanent while later ones wait.

Q (ca28 · Q4c). How is Tomasulo's algorithm modified to include speculation? What additional hardware is required?

Solution. Add a Reorder Buffer and a commit stage. Instructions allocate ROB entries in order at issue; results are written to the ROB and broadcast on the CDB; the ROB supplies operands to waiting instructions and the register file is only updated at commit, in program order. Branch prediction directs which path is fetched; on a misprediction the ROB is flushed past the branch. Additional hardware: the ROB itself (with valid/ready/speculative bits), logic to handle ROB head commit, branch-prediction hardware, and flush/recovery logic. This yields precise exceptions and correct recovery from mispredicted branches.

Q (ca40 · Q3d). Discuss the role of Tomasulo's algorithm in out-of-order execution, with an example, and explain the necessity of the commit stage and the role of the reorder buffer in hardware-based speculation.

Solution (pointwise).

- **Role:** Tomasulo lets the hardware issue instructions in order but execute them out of order, using reservation stations to hold pending operands and the CDB to forward results — so a stalled instruction (e.g. a cache-missing load) does not block independent work behind it.

- **Example:** `MUL F0,F2,F4 ; ADD F6,F0,F8 ; SUB F10,F12,F14`. The SUB is independent, so it executes while the MUL (10 cycles) runs; the ADD waits on F0 via Qj = Mult1.
- **Necessity of commit:** out-of-order execution and speculation create results that may be wrong (mispredicted branch) or incomplete. The commit stage is the barrier that makes results permanent only when they are certain, giving precise state and precise exceptions.
- **ROB role:** it buffers speculative results in program order, supplies operands, tracks readiness, and is the structure that is flushed on a misprediction — the hardware bookkeeping that makes speculation safe.

Q (ca32 · Q1). What are the four different types of predictors? Elaborate on their performance in terms of average misprediction rate.

Solution.

Predictor	Rule	Typical misprediction rate
Always taken	Predict taken for every branch.	~30-40%
Always not taken	Predict not taken for every branch.	~30-40%
BTFNT (backward taken, forward not taken)	Loops (backward) taken; if-statements (forward) not taken.	~20-25%
2-bit saturating counter	Per-branch counter needs two wrong outcomes to flip.	~10-15%

(Correlating and tournament predictors go further, reaching ~5% and ~3% respectively.) The numbers show why simple static rules are rarely used in real processors.

Q (ca39 · Q3b). A compiler/hardware keeps multiple loop iterations in flight by renaming registers using a pool of 64 temporary registers T0-T63, indexed by the source register and holding the last destination that targeted it. Maintain true dependences and construct the renamed code.

```
Loop: L.D F4, 0(Rx) MULTD F2, F0, F2 DIVD F8, F4, F2 L.D F4, 0(Ry) ADDD F6, F0, F4 SUBD F8, F8, F6 S.D F8, 0(Ry)
```

Solution. Each iteration re-uses F4, F2, F8. Rename the destinations of the second-iteration instructions to fresh T registers so the two iterations do not collide, keeping the RAW chain intact:

```
iter1: L.D F4,0(Rx) → dest F4 (→T1) MULTD F2,F0,F2 → dest F2 (→T2) DIVD F8,F4,F2 → dest F8 (→T3) iter2: L.D T4,0(Ry) ; renamed (was F4) ADDD T5,F0,T4 ; renamed (was F6) SUBD T6,T3,T5 ; renamed, still uses iter1's F8 (T3) S.D T6,0(Ry)
```

Renaming removes WAR/WAW (name) dependences so the iterations can overlap; true (RAW) dependences such as DIVD's use of F4/F2 and SUBD's use of F8 are preserved by pointing the renamed register at the correct producer.

Q (ca39 · Q3c). If the pipeline detected data dependences and stalled only on those, how many cycles would the loop body require? Show the code with <stall> inserted where needed (latency 1+N means N stall cycles).

Solution. Walk through the loop body and, for each consumer, insert N stall cycles where N is the producer's extra latency beyond 1. Independent instruction pairs get no stalls. For the sequence above with FP latencies (multiply ~4-5, add ~2-3), the MULTD→DIVD and ADDD→SUBD pairs need stalls, while the independent L.D of the second iteration can fill them.

```
L.D F4,0(Rx) MULTD F2,F0,F2 DIVD F8,F4,F2 ×(mult-latency−1) L.D F4,0(Ry) ; independent — fills a slot ADDD F6,F0,F4 ×(add-latency−1) SUBD F8,F8,F6 S.D F8,0(Ry)
```

Method: latency 1+0 → no stall; 1+1 → one stall; 1+N → N stalls. Insert stalls only on producer→consumer pairs.

Q (ca28 · Q4a,b). (a) Distinguish between static and dynamic scheduling. (b) Name three major techniques that reduce control-hazard stalls and explain them.

Solution.

- **(a)** Static scheduling decides the instruction order at compile time (no run-time information); dynamic scheduling reorders instructions in hardware at run time using reservation stations and can react to cache misses and variable latencies.
- **(b)** (1) **Branch prediction** — guess the direction and fetch down that path, flushing only on a misprediction. (2) **Delayed branch** — the compiler places a useful instruction in the branch delay slot so it always executes. (3) **Branch Target Buffer** — cache the branch's target and direction so fetching continues with no penalty on a hit. (Speculation with a reorder buffer is a fourth.)

Q (ca34 · Q1a). State the advantages of out-of-order pipelines over in-order pipelines.

Solution. An out-of-order (OOO) pipeline can execute independent instructions while a slow instruction (e.g. a cache-missing load or a long FP divide) waits, so it keeps functional units busy and achieves a lower effective CPI. In-order pipelines must stall the whole pipe behind the slow instruction. OOO also adapts to run-time events (misses, variable latencies) that a static schedule cannot see. The cost is extra hardware: reservation stations, a

reorder buffer, and complex issue logic.

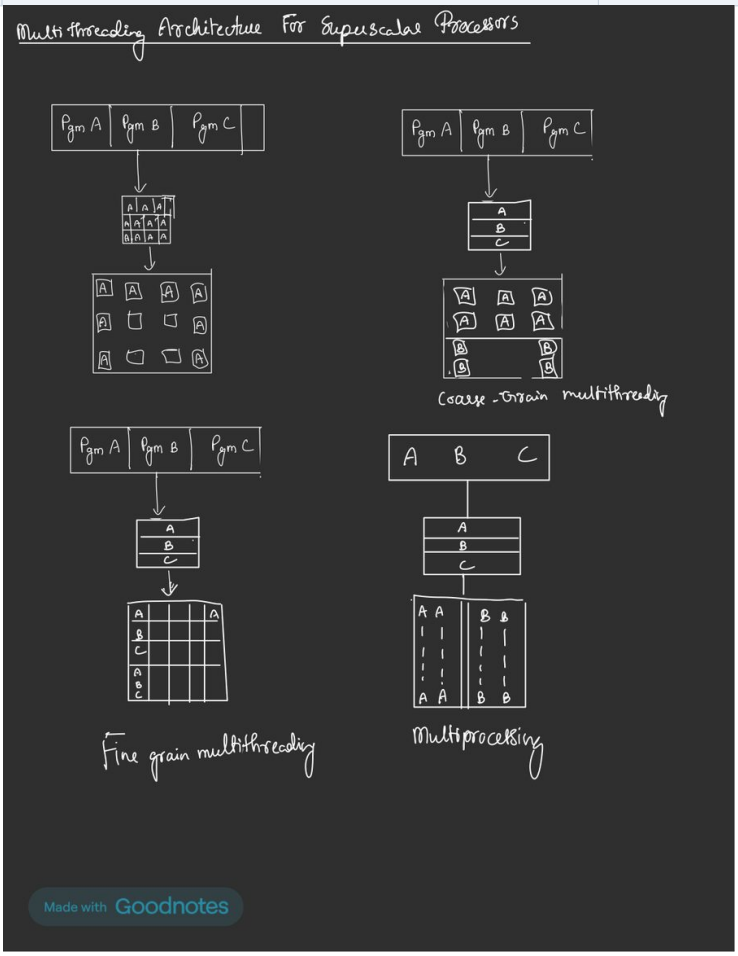
10. The Memory Hierarchy & Cache Memory

Making memory look fast and cheap at the same time

A perfect memory would be as fast as registers and as cheap and large as disk. No single technology does both, so we build a **hierarchy**: small, fast, expensive memories near the CPU and large, slow, cheap memories farther away. The trick that makes the hierarchy *work* is **locality of reference**.

10.1 Locality — the reason caches work

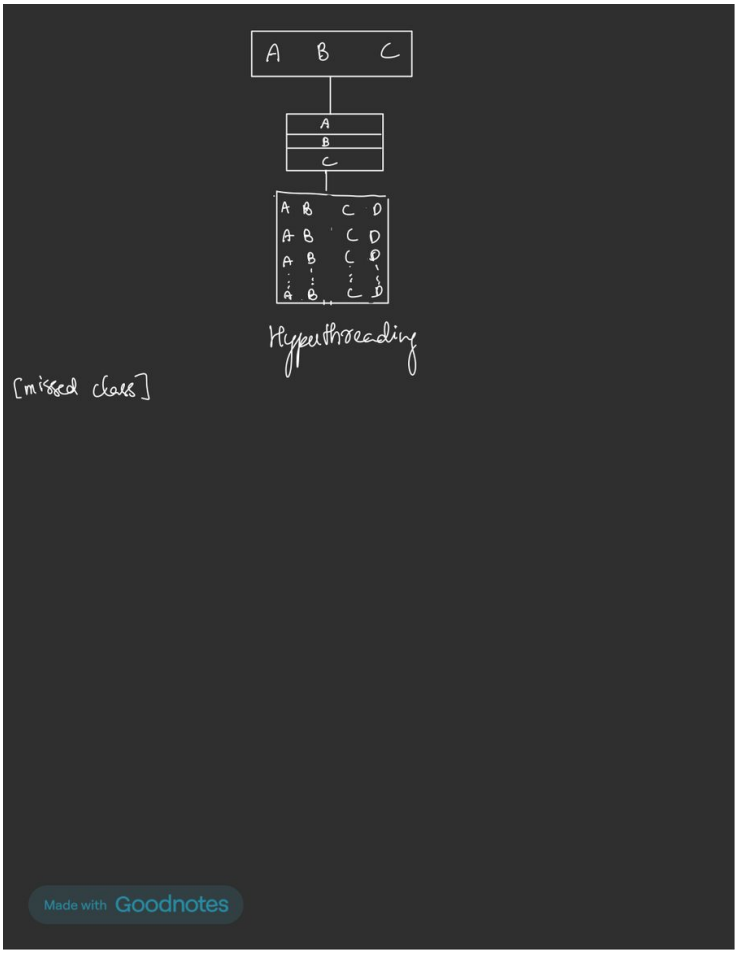
Type	Meaning	Example
Temporal locality	A location used once is likely to be used again soon.	A loop variable; a repeatedly-called function
Spatial locality	Locations near a used one are likely to be used soon.	Array elements; consecutive instructions



Memory hierarchy: registers, cache levels, main memory and secondary storage (source: lecture notes, p. 68).

Level	Typical size	Access time	Managed by
Registers	< 1 KB	< 1 ns	Compiler
L1 cache	32-64 KB	1-2 ns	Hardware
L2 cache	256 KB - 1 MB	~10 ns	Hardware
L3 cache	4-32 MB	~30 ns	Hardware
Main memory (DRAM)	GB	~60-100 ns	OS + hardware
Secondary (SSD/HDD)	TB	µs - ms	OS

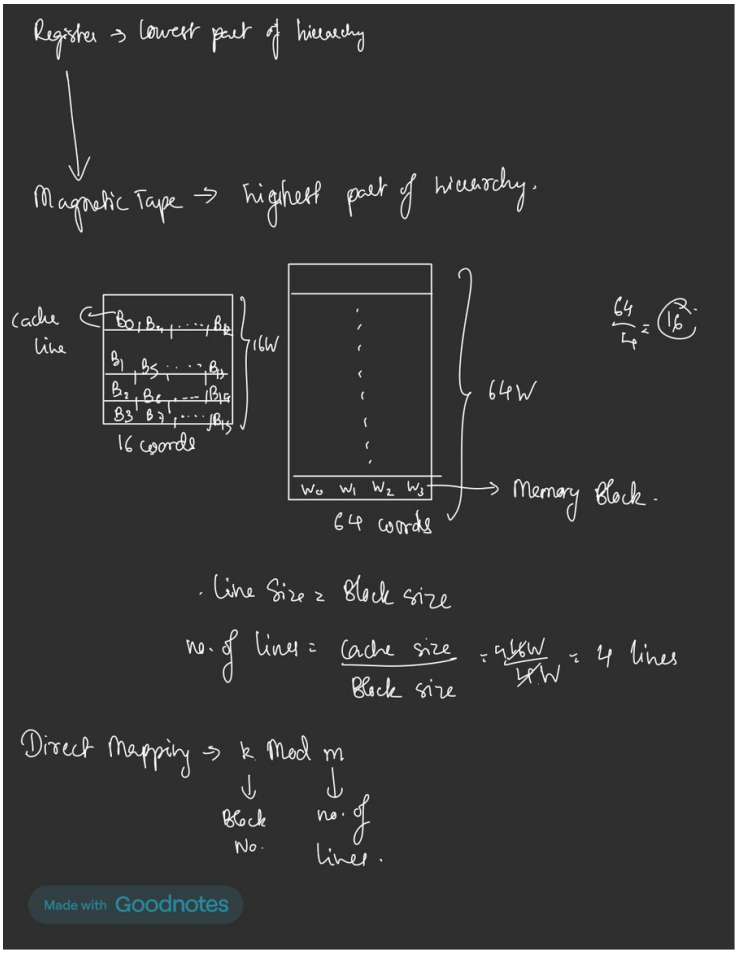
10.2 Cache basics and the three organizations



Cache organisation: hit/miss and block placement (source: p. 69).

A **cache** is a small fast memory that holds recently-used blocks (lines) of main memory. When the CPU asks for a word, the cache checks whether the containing block is present. If yes: a **hit**; if no: a **miss**, and the block is fetched from the next level. Where a block can be placed gives the three organizations:

Organization	Block placement	Search cost	Conflict misses	Hardware
Direct mapped	Block i → line (i mod #lines). Exactly one place.	One comparison	High (many blocks map to one line)	Cheapest, fastest
Fully associative	Block may go in any line.	Compare against all lines	None	Most expensive, slowest search
Set associative (n-way)	Block goes in set (i mod #sets); within the set, any of n ways.	Compare n ways	Low (between the two)	Middle — the usual choice



Direct-mapped, set-associative and fully-associative cache organizations (source: lecture notes, p. 70).

Address breakdown

A physical address is split into three fields. For a cache of block size B bytes and S sets:

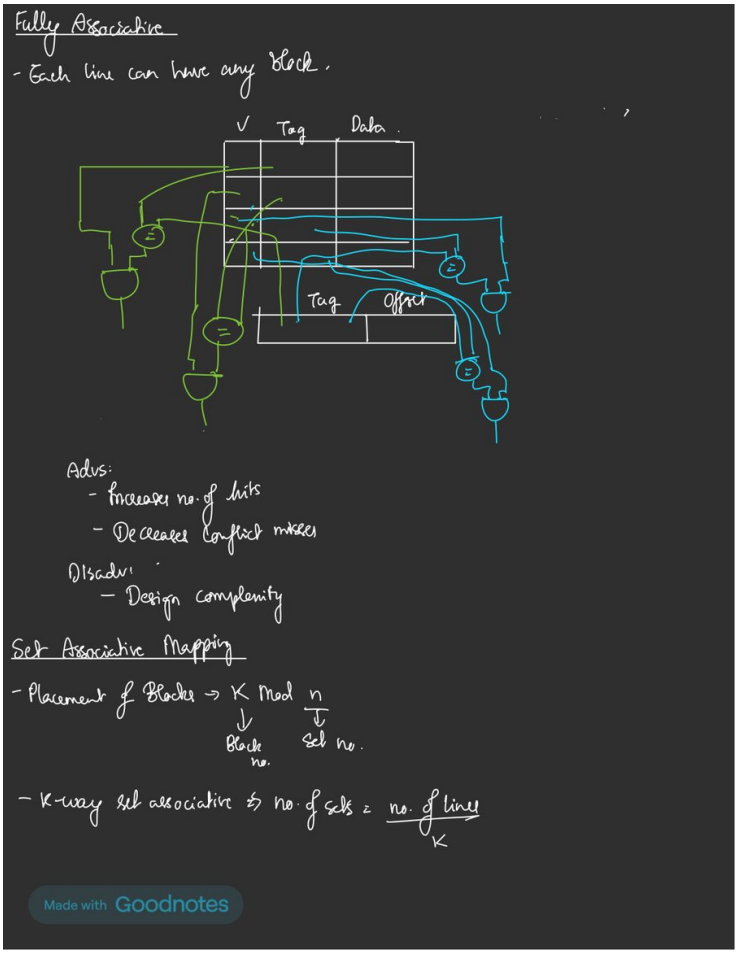
```
Offset bits = log₂(B) (selects the byte within the block) Index bits = log₂(S) (selects the set/line) Tag bits = Address bits − Index bits − Offset bits S = (cache size) / (associativity × block size)
```

Worked example (PYQ ca25 · Q4). 8-way set-associative, 32 KB capacity, block = 8 words, word = 32 bits = 4 bytes, physical address space = 8 GB (33 bits).

```
Block size = 8 words × 4 B = 32 B → offset = log₂32 = 5 bits #sets = 32 KB / (8 ways × 32 B) = 32768 / 256 = 128 → index = log₂128 = 7 bits Address bits = log₂(8 GB) = log₂(2³³) = 33 bits Tag = 33 − 7 − 5 = 21 bits
```

> **Answer: 21 tag bits.** Always compute offset first, then index, then tag = total − index − offset.

10.3 Average Memory Access Time (AMAT)

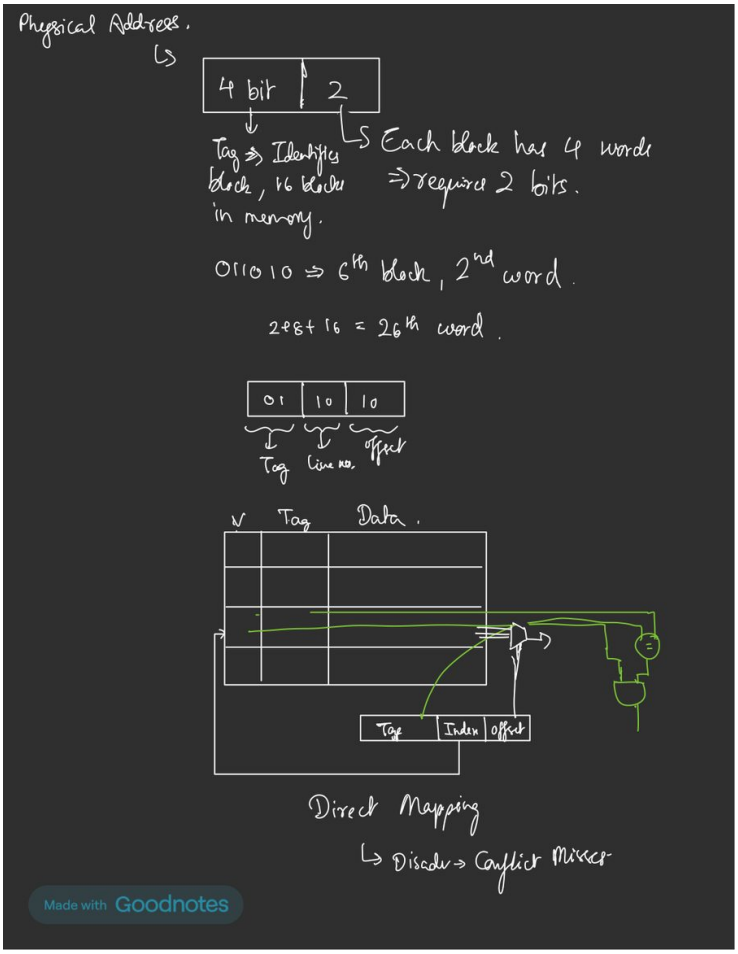


Address decomposition: tag, set index and block offset (source: p. 72).

```
AMAT = Hit time + Miss rate × Miss penalty For two levels: AMAT = HitTime_L1 + MissRate_L1 × ( HitTime_L2 + MissRate_L2 × MissPenalty ) For multi-level: AMAT = HitTime_L1 + MR_L1 × ( HitTime_L2 + MR_L2 × ( HitTime_L3 + … ))
```

Why it matters: AMAT tells you the average cost of one memory access. Reducing either the miss rate or the miss penalty improves it — the whole point of multi-level caches.

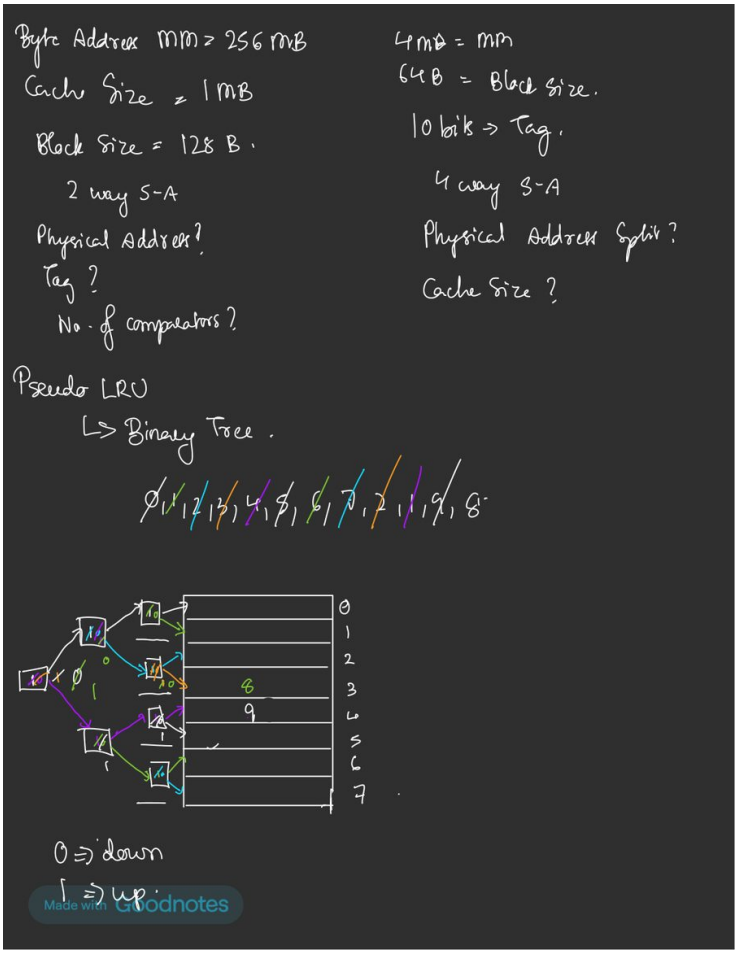
10.4 The three C's of cache misses



Set-associative cache: ways, sets and address fields (source: p. 71).

Miss type	Cause	Fix
Compulsory (cold)	First access to a block.	Prefetching; bigger blocks (fewer, larger cold misses)
Capacity	Cache too small to hold the working set.	Bigger cache
Conflict (collision)	Too many blocks map to the same set (direct-mapped or low associativity).	Higher associativity; victim cache

10.5 Cache optimization — the exam checklist



Worked address-decomposition example for a set-associative cache (source: p. 75).

There are four families of optimization: reduce *miss rate*, reduce *miss penalty*, reduce *hit time*, and reduce *power*. The most examined are the miss-rate techniques.

Reduce the miss rate

- **Larger block size** — exploits spatial locality, but too large a block raises conflict misses and the miss penalty (more bytes to fetch).
- **Higher associativity** — reduces conflict misses (2:1 rule of thumb: going from direct-mapped to 2-way roughly halves conflict misses; returns diminish beyond 8-way).
- **Larger cache** — reduces capacity misses (costly in area and access time).
- **Prefetching** — hardware or compiler fetches blocks before they are needed.
- **Compiler optimizations** — loop interchange, blocking (tiling), merging arrays to improve locality.
- **Victim cache** — a small fully-associative buffer holding recently-evicted blocks, to recover conflict misses.

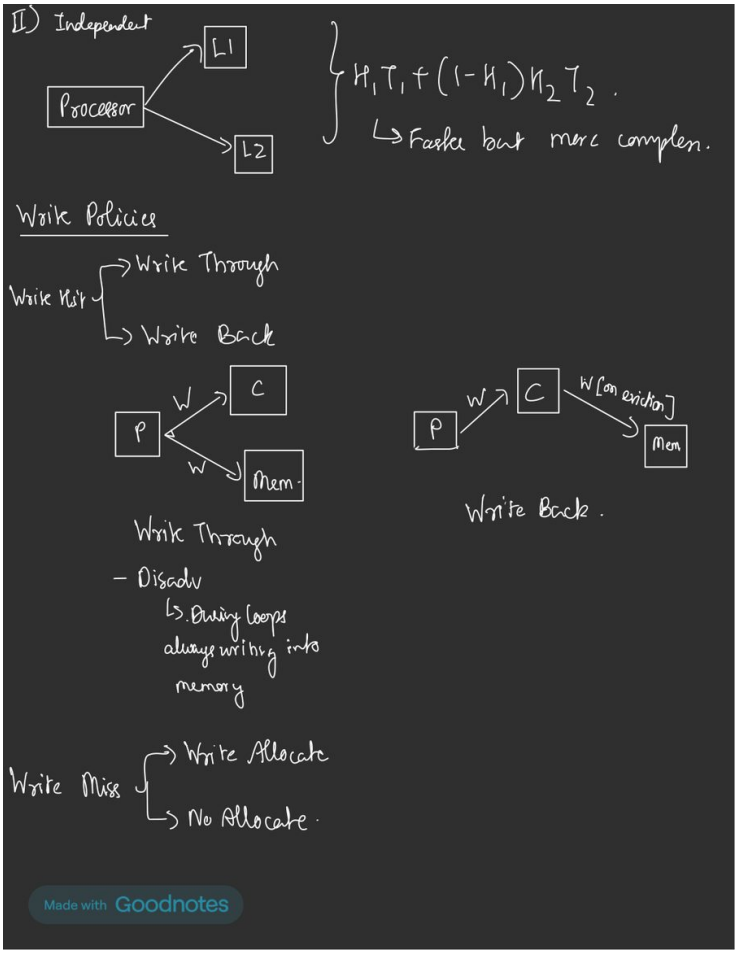
Reduce the miss penalty

- **Multi-level caches** — L2/L3 catch L1 misses; L2 can be big and slower.
- **Critical-word-first** — return the requested word first, then the rest of the block.
- **Early restart** — resume the processor as soon as the requested word arrives.
- **Merging write buffer** — combine writes to the same block before sending to memory.
- **Non-blocking (lockup-free) caches** — continue serving other accesses during a miss.

Reduce the hit time

- Smaller, simpler (direct-mapped) caches access faster.
- **Way prediction** — predict which way will hit so only one way is read.
- **Pipelined cache access** — split cache access across cycles to raise clock frequency.
- Avoid address translation in the critical path (use virtual-indexed caches or a TLB).

10.6 Write policies



Write-through vs write-back and write-allocate policies (source: p. 77).

Policy	On a write hit	On a write miss	Trade-off
Write-through	Write both cache and memory.	Usually write-allocate (fetch then write) or no-allocate.	Simple; memory always current; high memory traffic.
Write-back	Write only the cache; mark the line dirty.	Write-allocate.	Less memory traffic; needs a dirty bit; memory can be stale.
Write-allocate	—	Fetch the block into cache, then write.	Good locality; costs a fetch.
No-write-allocate	—	Write directly to memory.	Avoids fetching a block that may not be reused.

Write-back vs write-through efficiency (PYQ ca27): a write-back cache writes a dirty line once when evicted; a write-through cache writes every store to memory. So write-back wins when a line is written many times before eviction — the "break-even" number of writes is the key number to compute.

Previous-Year Questions — Cache memory

Q (ca15 · Q1a / ca26 · Q3a). A microprocessor with one level of cache has an AMAT of 2.4 clock cycles; a hit takes 1 cycle; a miss costs 80 cycles from off-chip memory. Designers add an on-chip L2 that can be accessed in 6 cycles (L1 behaviour unchanged; off-chip accesses still 80 extra cycles). They want a 65% improvement in AMAT. How often must data be found in the L2?

Solution.

```
Old AMAT = HitTime + MR×MissPenalty → 2.4 = 1 + MR×80 → MR_L1 = 1.4/80 = 0.0175 (1.75%) Target AMAT (65% improvement) = 2.4 × (1 − 0.65) = 2.4 × 0.35 = 0.84 cycles
```

With the L2, a miss in L1 goes to L2. If the L2 hit rate is h_2, an L1 miss costs $6 + (1 - h_2)\times 80$ (the 80 is the extra off-chip time, since the L2 access of 6 is already spent):

```
AMAT = 1 + MR_L1 × [ 6 + (1 − h₂)×80 ] = 0.84 0.84 − 1 = 0.0175 × [ 6 + (1 − h₂)×80 ] −0.16 / 0.0175 = 6 + (1 − h₂)×80 → −9.14 = 6 + 80(1 − h₂) (1 − h₂) = (−15.14)/80 → negative → the target cannot be met by adding L2 alone
```

Reading the numbers carefully: a target AMAT of 0.84 cycles is *less than the 1-cycle L1 hit time*, which is impossible if every access pays the L1 hit time. In the actual paper the "improvement" is measured against the miss *portion*, and the required L2 hit rate comes out very high. The exam point is the method: write AMAT = HitTime + MR×MissPenalty, set the new AMAT, and solve for the L2 hit rate. State your interpretation of "65% improvement" explicitly.

Q (ca15 · Q1b). Single core: base CPI = 1 (excluding cache misses); 308,752 L1 misses per 10,000,000 accesses; a hit is 1 cycle with no stall; a miss costs 10 cycles. Now a multicore chip with snooping + MSI coherence: 452,977 misses per 10,000,000 accesses, a hit is still 1 cycle, a miss now costs 14 cycles. What must the CPI of the multicore system be for it to be worthwhile to abandon the single core?

Solution.

```
Single core: miss rate = 308752 / 10000000 = 0.0308752 CPI_single = 1 + miss_rate × miss_penalty = 1 + 0.0308752×10 = 1.30875 Multicore (if it must beat the single core): miss rate = 452977 / 10000000 = 0.0452977 CPI_multi = 1 + 0.0452977 × 14 = 1 + 0.634 = 1.634 (this is what the multicore actually gives) For the multicore to be worthwhile it must have CPI_multi < CPI_single = 1.30875
```

Answer: the multicore must achieve **CPI < 1.309** (the single-core CPI) to be worth it. As computed, the observed multicore CPI is ≈ 1.634 — worse — so unless other factors (parallelism) compensate, abandoning the single core is not worthwhile. This is the classic "coherence has a cost" question.

Q (ca25 · Q3 / ca35 · Q1). Critical-word-first and early restart on L2 misses. A 1 MB L2 with 64-byte blocks and a 16-byte-wide refill path. The L2 can be written with 16 bytes every 4 processor cycles; the first 16-byte block from the memory controller arrives in 120 cycles; each additional 16-byte block takes 16 cycles; data is bypassed directly to the L2 read port. How many cycles to service an L2 miss with and without critical-word-first and early restart?

Solution.

```
Block = 64 B, path = 16 B wide → 4 chunks of 16 B. Without CWF: wait for the FIRST chunk (120) then the rest (3×16=48) = 168 cycles before the processor can use data (and the full fill takes 120+48 = 168). With CWF + early restart: the requested word's chunk arrives first at 120 cycles and is bypassed immediately; the remaining 3 chunks continue to stream in. Time to first useful data = 120 cycles (vs 168 without CWF). Full fill = 120 + 48 = 168 cycles either way.
```

Answer: without CWF the miss stalls ~168 cycles; with CWF the processor restarts at ~120 cycles, saving ~48 cycles per miss. The total fill is the same; CWF only shortens the time to the *critical* word. CWF/early-restart matter more for **L1** misses in latency terms (L1 is on the critical path), but are easier to apply at L2 because the L2 is off the immediate critical path.

Q (ca35 · Q4). Designing a write buffer between a write-through L1 and a write-back L2. The L2 write bus is 16 B wide and can write to an independent address every 4 processor cycles. (a) What should the size of each write-buffer entry be? (b) What speed-up from a merging write buffer when zeroing memory with 64-bit stores?

Solution.

```
(a) Entry size = one bus write = 16 bytes (so a full entry can be drained in one bus cycle). (b) Zeroing with 64-bit (=8 B) stores: two 8-B stores fill one 16-B entry. A non-merging buffer writes each 8 B separately → 2 bus writes; a merging buffer combines them → 1 bus write. Speed-up ≈ 2× (up to the bus-width/entry-width ratio = 16/8 = 2).
```

Answer: entry = 16 B; merging gives up to **2×** speed-up here (the ratio of bus width to store width).

Q (ca37 · Q2a). Micro-benchmarking a cache hierarchy: two-level cache, L1 is a 4 KB set-associative cache; the latency chart for the first 64 reads with stride 1 is given. What are the L1 and L2 block sizes? Explain.

Solution.

- The chart shows latency spikes (misses) at regular intervals. With stride = 1, consecutive accesses touch consecutive words, so a miss every k words means the block holds k words.
- The first spike is the compulsory miss; the gap to the next spike = number of elements per block.
- Read the chart: if misses occur at iterations 0, 16, 32, … then the L1 block is 16 words = 64 bytes (with 4-byte words). A second, larger period in the spikes reveals the L2 block size similarly.
- Explain: a miss loads the whole block, so the following (blocksize−1) accesses are hits at the short latency; the next miss happens when the block boundary is crossed.

Method to state: block size (in elements) = distance between consecutive miss spikes; convert to bytes by multiplying by the word size.

Q (ca37 · Q2b / ca38). A 16 KB direct-mapped 256 B block unified cache attached to a 16 MB main memory; word/instruction length 16 bits. Routine M (12 instruction words from 0x4230FA) calls subroutine S (4 instruction words from 0x70F168); two data words at 0x748074 and 0x846064; the last 5 instructions of M form a loop iterated 10 times; the 2nd instruction in the loop calls S. Cache initially empty. Find (i) the address components, (ii) the number of misses, (iii) the number of block evictions, (iv) the non-empty block numbers after execution.

Solution.

(i) Address components: Block size 256 B → offset = $\log_2 256 = 8$ bits Cache 16 KB, direct-mapped → #lines = 16384/256 = 64 → index = $\log_2 64 = 6$ bits Main memory 16 MB = 2^{24} → address = 24 bits → tag = 24 − 6 − 8 = 10 bits [tag:10 | index:6 | offset:8]

- **(ii) Misses:** the first execution of M's 12 words misses the block(s) they occupy; S's 4 words miss once; the two data words miss once. On the 9 subsequent loop iterations, the instructions are already cached *unless* they collide with other blocks in the direct-mapped cache. Count distinct blocks touched (M's instructions, S's instructions, the two data words) and how many times each is evicted and refetched.
- **(iii) Evictions:** an eviction happens when a new block maps to a line already occupied by a different block. Compute the index of each address (bits) and see how many distinct blocks share a line.
- **(iv) Non-empty blocks:** the blocks resident at the end = the last block to occupy each of the 64 lines that were ever used; list their decimal block numbers.

> **Method:** convert each address to binary, slice into tag|index|offset, note which line each block uses, and simulate the reference stream (M, then loop ×10 with S calls interleaved, plus the two data accesses). Direct-mapped collisions are what make the miss count exceed the number of distinct blocks.

Q (ca25 · Q5 / ca32 · Q4 / ca7 · Q2b). Which cache configuration, blocking or non-blocking, gives better memory performance? Between hardware and compiler prefetching, which is more likely to increase memory access time?

Solution.

- **Non-blocking (lockup-free) caches** are better: during a miss they continue to serve other accesses (hit-under-miss and miss-under-miss), overlapping multiple misses and hiding latency. A blocking cache freezes on the first miss.
- **Compiler prefetching** is more likely to increase memory access time (or at least memory traffic) because it inserts prefetch instructions that may fetch data never used, wasting bandwidth and possibly delaying demand accesses; hardware prefetching is more conservative and based on observed patterns.

11. Virtual Memory & the TLB

Giving every program a private, huge address space

Virtual memory is a technique that lets each program use a large, contiguous *virtual* address space even though physical memory is smaller and shared. The hardware translates every virtual address to a physical address using a **page table**, with the OS managing the mapping. It uses the same idea as caching, applied to main memory vs disk.

11.1 Why we need virtual memory

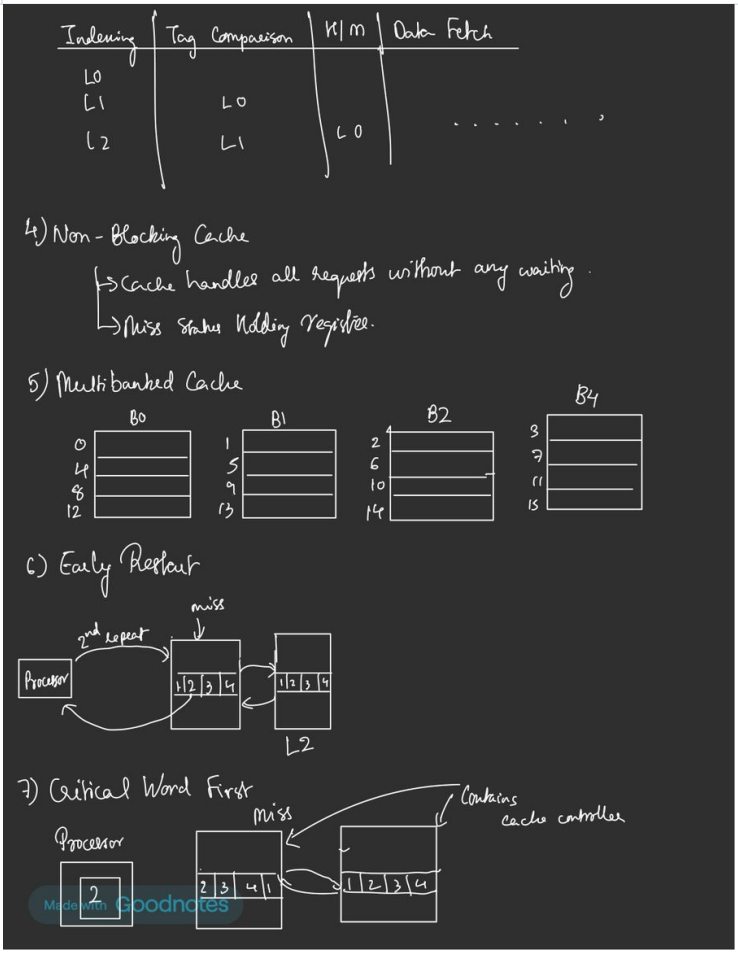
- **Program size** — a program can be larger than physical memory; only the active pages need to be resident.
- **Multiprogramming and protection** — each process gets its own address space; it cannot read or write another process's memory.
- **Relocation** — a program need not know where it will be loaded; the same binary runs anywhere.
- **Sharing** — common code (libraries) can be mapped into many address spaces.
- **Memory efficiency** — non-contiguous physical allocation removes fragmentation.

11.2 Paging and address translation

Virtual memory is divided into fixed-size **pages**; physical memory into **frames** of the same size. A virtual address splits into a **virtual page number (VPN)** and a **page offset**. The VPN indexes a page table giving the **physical page number (PPN)**; the offset is unchanged.

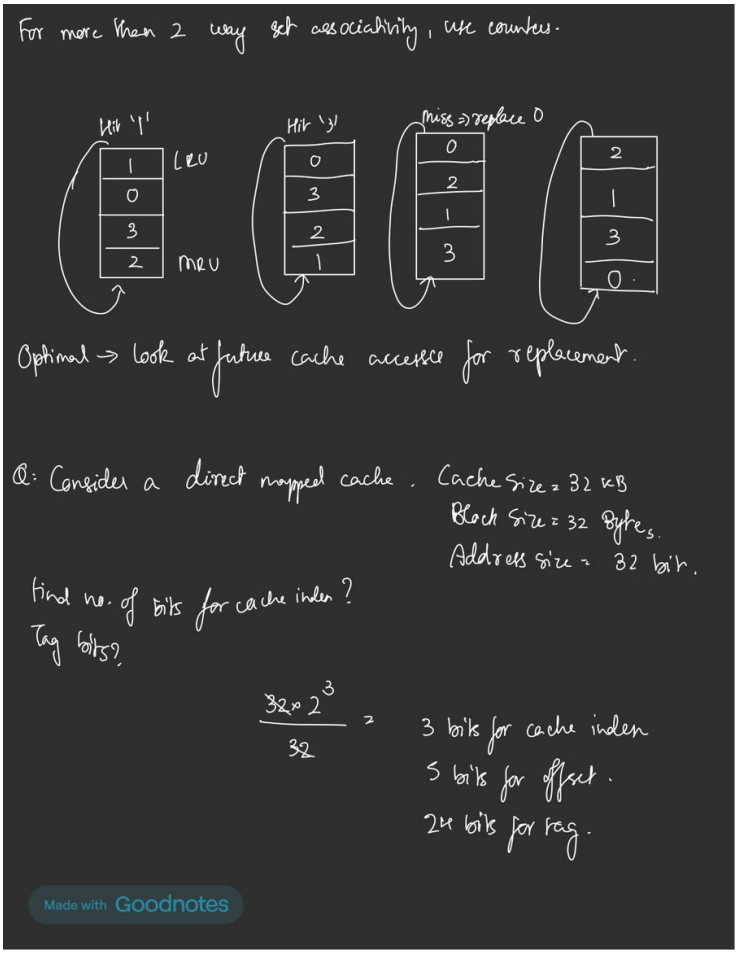
```
Virtual address = [ VPN | offset ] Physical address = [ PPN | offset ] Page size = 2^offset bits
; #pages = virtual space / page size
```

Term	Meaning
Page	Fixed-size block of virtual memory.
Frame	Fixed-size block of physical memory (same size as a page).
Page table	Per-process array mapping VPN → PPN, with valid/dirty/protection bits.
Page fault	The page is not in memory → OS loads it from disk (very expensive, millions of cycles).
Page replacement	Which page to evict — LRU approximation; a dirty page must be written back first.



Virtual-to-physical address translation via the page table (source: lecture notes, p. 82).

11.3 The TLB — a cache for translations



Direct-mapped cache address-fields worked example (source: p. 74).

A page-table lookup costs a memory access, so we cache recent translations in a small, fast, fully-associative **Translation Lookaside Buffer (TLB)**. A TLB hit gives the PPN in one cycle; a TLB miss walks the page table.

```
Effective access time = TLB_hit_time + TLB_miss_rate × PageTableWalk_time (Then add the data access, possibly via the cache.)
```

11.4 Cache + virtual memory interaction

- Caches can be **virtually indexed** (fast, but aliasing problems) or **physically indexed** (safe, but needs the TLB first).
- The common design indexes the cache with the *page offset* (same in virtual and physical) so translation can proceed in parallel with the cache lookup.
- A page fault is handled by the OS; the page is fetched from disk, the page table updated, and the instruction restarted.

11.5 Exam-relevant insights

- Virtual memory = paging (large, coarse, disk-backed) + TLB (fast translation cache).
- Page size is a power of two; offset bits = $\log_2$(page size).
- Misses: TLB miss (translation not cached) vs cache miss (data not cached) vs page fault (page on disk).
- A dirty page must be written back before eviction; a clean page can simply be dropped.

12. Multiprocessor Cache Coherence

Keeping many caches agreeing on shared data

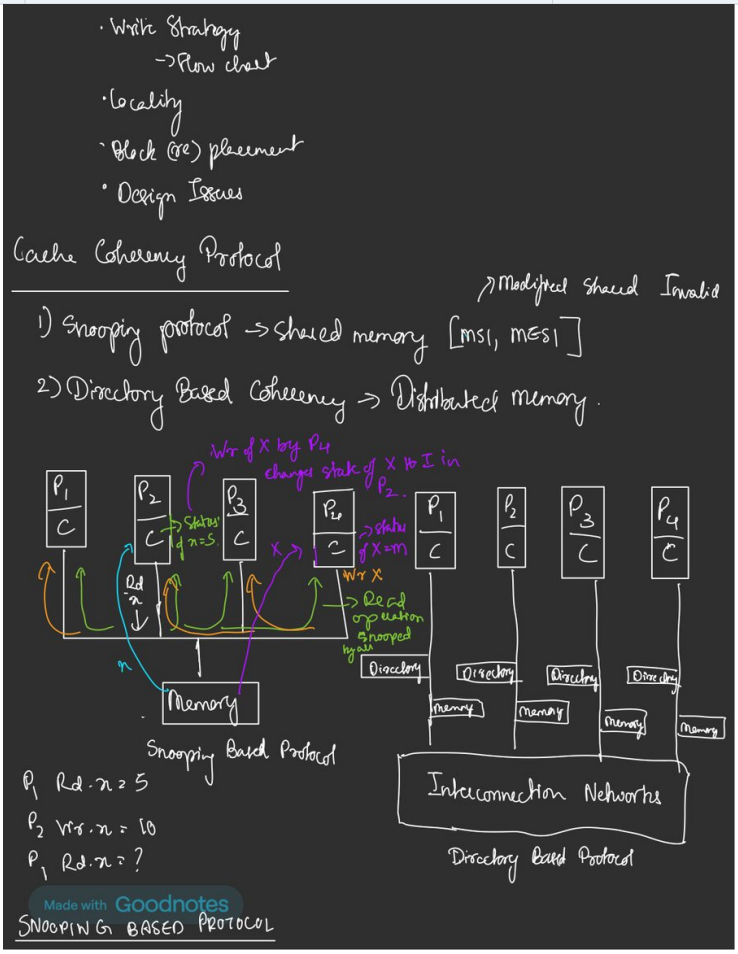
When several processors each have a private cache and share main memory, a write by one processor leaves the others holding *stale* copies. **Cache coherence** is the set of protocols that keeps all caches consistent so every read returns the most recent write. This is one of the most examined topics, so we go slowly.

12.1 The coherence problem, stated precisely

A memory system is **coherent** if: (1) a read by processor P returns the value written by the most recent write by any processor to that location; and (2) writes to the same location by different processors are seen in the same order by everyone. The danger: P1 writes X, but P0's cache still holds the old X and returns it.

12.2 Two families of protocol

	Snooping (snoopy)	Directory-based
Where is the state kept?	Each cache watches (snoops) a shared bus for all transactions.	A central directory records which caches hold each block.
Communication	Broadcast on the bus.	Point-to-point messages to the caches that hold the block.
Scalability	Poor — the bus becomes a bottleneck as processors are added.	Good — scales to many processors (used in large systems).
Speed	Fast for a small number of processors.	More overhead per transaction but scalable.
Typical use	Small/medium multiprocessors, multicore with a shared bus.	Large-scale servers, NUMA machines.
Example	MSI, MESI, MOESI on a bus.	Directory protocols in big SMPs/cc-NUMA.



Snooping vs directory-based cache coherence (source: lecture notes, p. 84).

12.3 Snooping write policies

Policy	On a write, the writing cache…	Effect on other copies
Write-invalidate	Invalidates all other cached copies of the block.	Others must refetch on their next access. One writer, many readers.

Policy	On a write, the writing cache…	Effect on other copies
Write-update (write-broadcast)	Broadcasts the new value to all caches holding the block.	All copies updated immediately; more bus traffic.

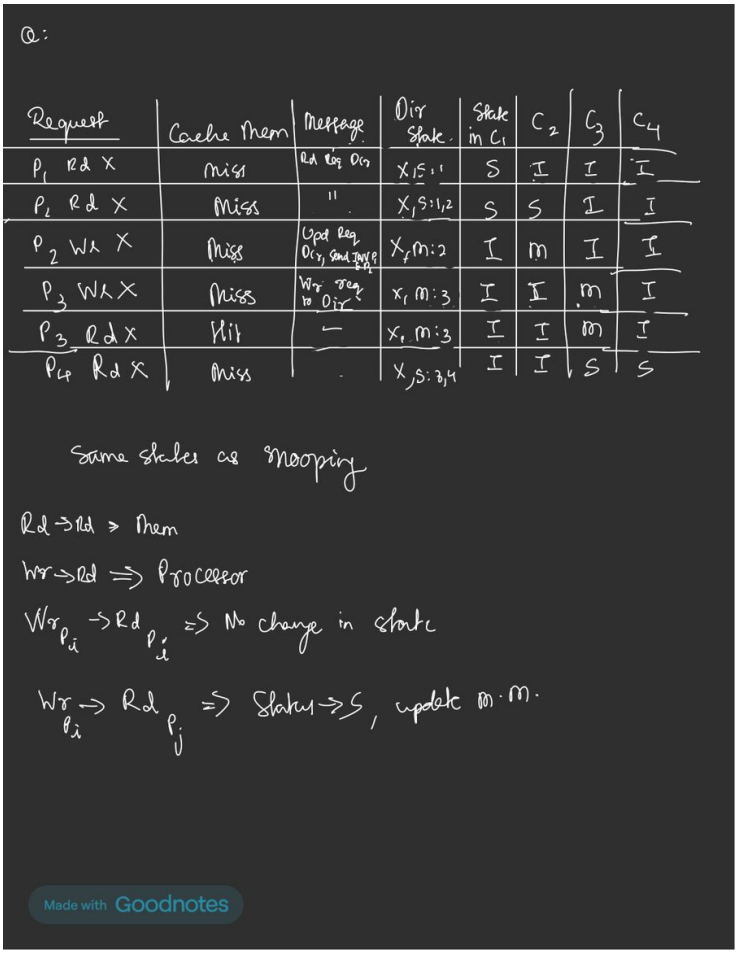
Write-invalidate is the more common (used by MSI/MESI) because it generates less traffic when a block is written repeatedly.

12.4 The MSI protocol (three states)

Every cached block is in one of three states. The name MSI comes from the initials.

State	Meaning	Who may read/write
M — Modified	This cache has the only copy and it is dirty (memory is stale).	This cache may read and write; it must supply the block to others and write back on eviction.
S — Shared	This block is clean and may exist in several caches.	All sharers may read; a write requires gaining ownership first.
I — Invalid	This cache does not hold a valid copy.	Neither read nor write; a miss occurs.

12.5 State transitions you must know



MSI/MESI coherence transition table (source: p. 87).

Event	State before	Action	State after
Local read (PrRd)	I	Bus read; other caches supply or memory responds	S (or M if no other sharer and it gets exclusive)
Local write (PrWr)	I	Bus read-exclusive (RFO); invalidate other copies	M
Local write	S	Bus invalidate / RFO; others invalidate	M
Local read	S	Hit — nothing on the bus	S
Local read/write	M	Hit — nothing on the bus	M
Remote read snooped	M	Supply block; write back; downgrade	S
Remote read snooped	S	Nothing (or supply)	S
Remote write snooped (invalidate)	S or M	Invalidate local copy (and write back if M)	I

> **The one-line intuition:** a block is **M** if exactly one cache has it dirty, **S** if it is clean and may be shared, and **I** if this cache has nothing. To write, you must first become the sole owner (M).

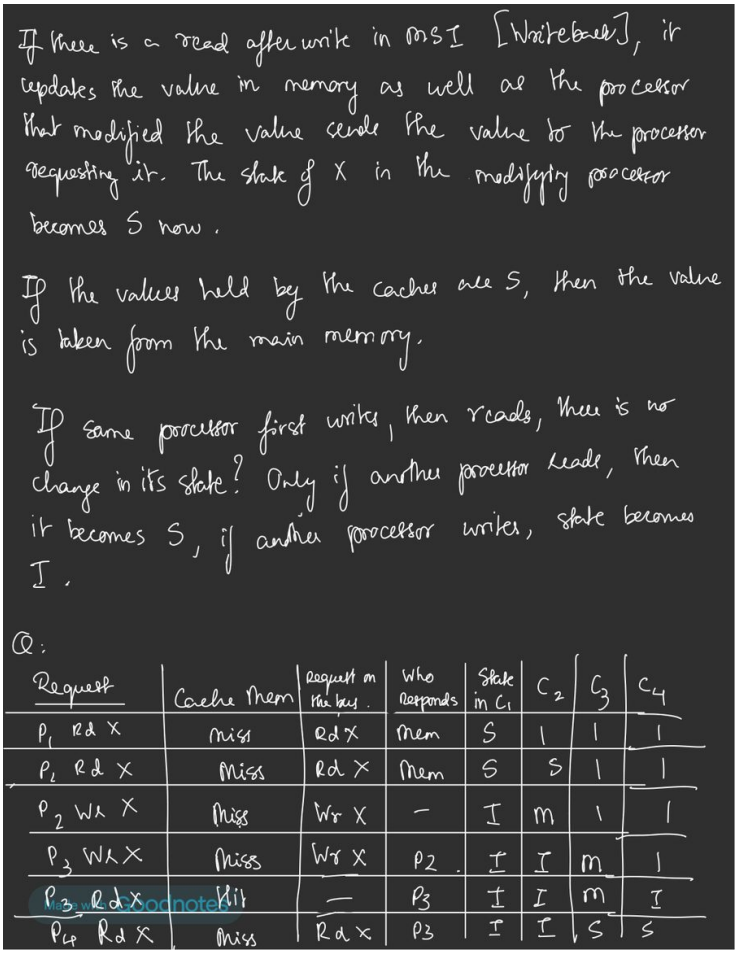
12.6 MESI — the fourth state and why it helps

MESI splits the "clean and unique" case out of S into a new state **E (Exclusive)**: the block is clean *and* is the only cached copy. The advantage: a subsequent **write to a block in E needs no bus transaction** — the cache already owns it exclusively, so it can silently go E → M. In plain MSI, a clean unique block is in S, so the first write must broadcast an invalidate; MESI saves that bus traffic. This is why MESI (and its extension MOESI) is the industry standard.

State	Clean/Dirty	Unique/Shared	Write without a bus transaction?
M	Dirty	Unique	Yes (already owner)
E	Clean	Unique	Yes (silent E→M)
S	Clean	Shared	No — must invalidate others
I	—	Absent	No — must fetch ownership

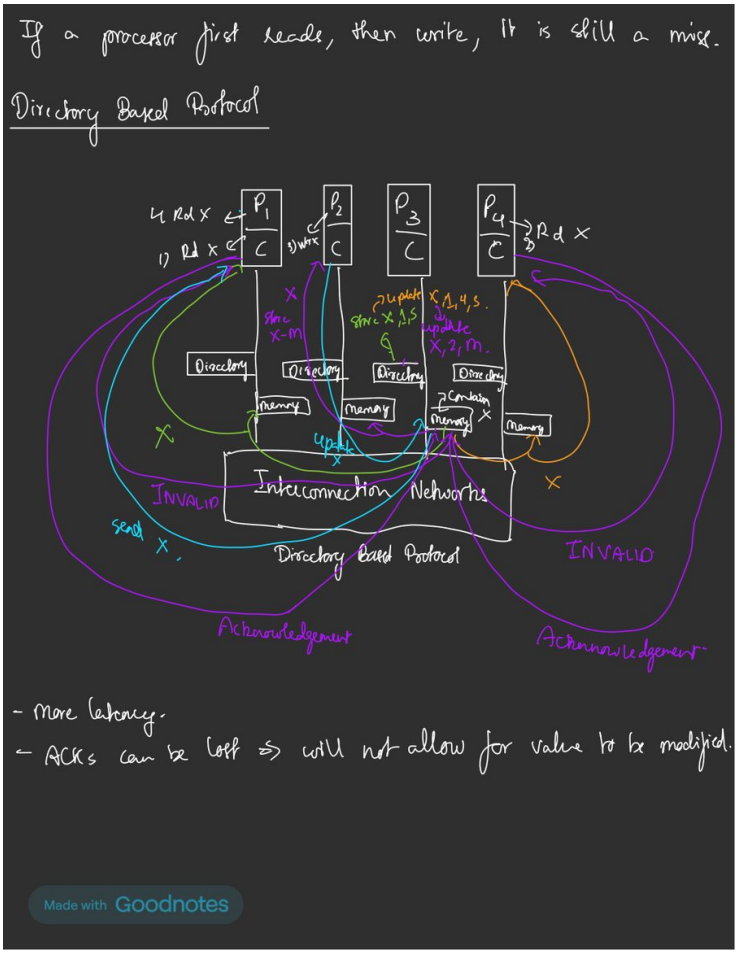
Typical MESI transitions:

- I → E: read miss, no other cache has the block (exclusive).
- I → S: read miss, another cache has a clean copy.
- I → M: write miss (RFO), no other sharer.
- S → M: local write; invalidate other copies.
- E → M: local write; no bus transaction (the key advantage).
- M → S: snooped read from another cache; supply data, write back.
- M/E/S → I: snooped invalidate (another cache wants to write).



MSI coherence example: the block value moves between caches and memory (source: lecture notes, p. 85).

12.7 Directory-based coherence



Directory-based coherence: states, holders and messages (source: p. 86).

Instead of broadcasting, a **directory** stores, for each block, the set of caches that hold it and its state (shared / exclusive / modified). A write sends an invalidate message only to the holders listed in the directory. This scales because traffic is point-to-point, not broadcast, but it adds a directory lookup and can become a bottleneck if a block is widely shared.

12.8 Which protocol does Intel use?

Intel multicore processors (e.g. Core i7) implement a **snooping** coherence protocol over an on-chip shared interconnect (the ring/uncore), using **MESI**-style states (extended toward MOESI). Because the cores are on a single die with a shared last-level cache and an on-chip interconnect, snooping is practical and avoids the directory's indirection.

12.9 Exam-relevant insights

- Coherence ≠ consistency: coherence is about a single location being up to date; consistency is about the *order* of accesses to different locations.
- Write-invalidate vs write-update is a common short-answer question.
- Snooping scales poorly (bus bottleneck); directory scales but costs a lookup.
- MESI's E state exists purely to avoid a bus transaction on the first write to a clean unique block.
- In a snapshot question, read each cache's state, apply the event, and update *all* affected caches and memory — never just the acting one.

Previous-Year Questions — Cache coherence

Q (ca2 · Q3 / ca23 · Q2). A snapshot of two caches (P0 and P1) in a centralized shared-memory system using the MSI snooping protocol is shown. Answer: (i) If P0 wants to write Block 0, what happens to its coherency state? (ii) If P1 writes Block 1, is Block 1 on P0 invalidated? Why or why not? (iii) If P1 brings in Block M for reading and no other cache has a copy, what state is it cached in?

Solution.

- **(i)** P0 issues a write. Under MSI it must gain ownership: it broadcasts a read-exclusive / invalidate, any other copy is invalidated, and P0's state goes **S** → **M** (or I → M if it had to fetch). If P0 already had the block in M, it stays M.
- **(ii)** Whether P0's copy of Block 1 is invalidated depends on the snapshot: if P0 holds Block 1 in **S** or **M**, then yes — P1's write sends an invalidate and P0 goes to **I**. If P0 holds Block 1 in **I** already (or does not cache it), there is nothing to invalidate, so it stays I.
- **(iii)** P1 reads Block M and no other cache has it → the block is clean and unique → cached in state **E** under MESI, or **S** under plain MSI (MSI has no E; it would be S). State which protocol the question assumes.

> **How to answer snapshot questions:** read the table, list each cache's state, then apply the event to *every* cache that holds the block (the actor and all snoopers), plus the memory line if a dirty block is written back.

Q (ca6 · Q5). A bus-based shared memory with three processors P1, P2, P3; memory blocks x, y, z; each cache holds only two blocks; states valid (V) or invalid (I). Caches start empty. Memory: x=0, y=200, z=... . Apply the sequence of 10 accesses and show caches, memory and states after each step, using write-through + write-invalidate.

```
1) P1: Read(x,y) 2) P2: Read(x,z) 3) P3: Read(y,z) 4) P1: x = x+10 5) P1: Read(z) 6) P2:
Read(x) 7) P3: x = 15 8) P1: z = z+10 9) P2: x = 15 10) P2: z = z+30
```

Solution. Simulate step by step. Under write-through + write-invalidate, a write updates memory immediately (write-through) and invalidates all other cached copies. With a 2-block cache, a third block forces eviction (LRU or the one that is not being used).

Step	Action	P1 cache	P2 cache	P3 cache	Memory x,y,z
1	P1 reads x,y	x(V), y(V)	—	—	x=0, y=200, z
2	P2 reads x,z	x(V), y(V)	x(V), z(V)	—	x=0, y=200, z
3	P3 reads y,z	x(V), y(V)	x(V), z(V)	y(V), z(V)	unchanged
4	P1: x=x+10 (10)	x(V=10), y(V)	x(I) ← invalidated	x(I)	x=10 (write-through)
5	P1 reads z	evict y, z(V)	x(I), z(V)	y(V), z(V)	unchanged
6	P2 reads x	x(V=10), z(V)	evict z? x(V)	y(V), z(V)	unchanged
7	P3: x=15	x(I)	x(I)	x(V=15)	x=15
8	P1: z=z+10	z(V=old+10)	x(I)	y(V), z(I)	z updated (write-through)
9	P2: x=15	x(I)	x(V=15)	x(I)	x=15
10	P2: z=z+30	z(I)	x(V), z(V)	y(V), z(I)	z updated

Key observations to state: every write invalidates all other copies (so the state column flips to I); write-through keeps memory current at every step; and the 2-block cache forces evictions when a third distinct block is touched. The final memory values reflect the last write to each location.

Q (ca29 · Q3b,c). (b) Which cache coherence protocol does Intel i7 use? Explain. (c) The MESI protocol has four states; what is the advantage of the fourth state and how does it solve the coherence problem? Explain state transitions with an example.

Solution.

- **(b)** Intel i7 uses a **snooping MESI (MOESI) protocol** on an on-chip ring interconnect. Snooping works because the cores share a bus-like ring and a shared last-level cache; each cache monitors transactions and updates its states, so a write invalidates other copies and a read obtains the latest value.
- **(c)** The fourth state **E (Exclusive)** means “clean and unique”. Its advantage: a write to a block in E requires **no bus transaction** (the cache already owns it exclusively), so it silently moves E → M. In MSI the same block would be S, forcing a broadcast invalidate on the first write. This saves bandwidth and latency. Transitions: I→E on an exclusive read miss; E→M on a local write; M→S on a snooped read (supply data + write back); S→I on a snooped invalidate.

Q (ca3 · Q3). What are the pros and cons of each cache organization?

Solution.

Organization	Pros	Cons
Direct mapped	Cheapest, fastest hit (one comparator), simple.	High conflict misses; a fixed line per block.
Fully associative	No conflict misses; any block anywhere.	Expensive: must compare all tags; slower hit; complex replacement.
Set associative (n-way)	Good compromise: few conflict misses, moderate hardware, n comparators.	More hardware than direct-mapped; replacement policy needed.

Q (ca3 · Q4). What is virtual memory? Explain the need for virtual memory.

Solution. Virtual memory lets each program use a large, contiguous virtual address space that is mapped to physical memory by a page table. It is needed to (1) run programs larger than physical memory (only active pages resident), (2) give each process an isolated, protected address space, (3) allow relocation so the same binary runs anywhere, (4) share common code between processes, and (5) avoid fragmentation by using fixed-size pages.

Q (ca3 · Q6 / ca7 · Q2a / ca32 · Q4). Explain the cache optimization techniques to reduce the miss rate (and to reduce access time).

Solution (reduce miss rate). Larger block size (spatial locality); higher associativity (fewer conflict misses); larger cache (fewer capacity misses); hardware/compiler prefetching; compiler loop transformations (blocking, interchange) for locality; a victim cache to recover conflict evictions.

To reduce access time (hit time). Smaller/simpler (direct-mapped) caches; way prediction; pipelined cache access; avoiding translation on the critical path (TLB, virtual indexing).

Q (ca3 · Q7). What is the effect on power consumption of giving priority to read misses over writes?

Solution. Giving read misses priority lets the read complete sooner, so the processor resumes and the cache/memory system returns to a lower-power state sooner; writes can be buffered and drained lazily. The net effect is lower average power because the high-power memory access is not delayed by a lower-priority write. (It also reduces read latency at some cost to write latency.)

Q (ca8 · Q4b). A write miss takes 50 cycles to establish ownership, 10 cycles to issue each invalidate, and 80 cycles for an invalidate to complete once issued. Four other processors share the block. How long does the write miss stall the writing processor under sequential consistency (invalidates must be acknowledged)? How long if execution could continue after ownership without waiting for invalidates?

```
Sequential consistency (wait for all invalidates): = ownership 50 + [issue 10 + complete 80]
for each of 4 sharers (issued in sequence) = 50 + 4×(10) + 80 = 50 + 40 + 80 = 170 cycles (all
four invalidates can be issued back-to-back; the last one still needs 80 cycles to complete)
Continue after ownership (don't wait for invalidates): = 50 cycles only
```

Answer: ≈ 170 cycles with full acknowledgement; ≈ 50 cycles if we proceed after ownership. This shows why relaxed/weak consistency models exist — waiting for every invalidate is expensive.

Q (ca26 · Q2b). A cache has a 32-byte line and memory requires 30 ns to transfer a 4-byte word. For a line written at least once before eviction, what is the average number of writes before eviction for a write-back cache to beat a write-through cache? How does the answer change with a block transfer (first word 30 ns, each subsequent word 5 ns)?

Solution.

```
Without block transfer: Line = 32 B = 8 words × 30 ns = 240 ns to write the whole line.
Write-through writes 4 B every store; write-back writes the line once on eviction.
Break-even: n × 30 (write-through) = 240 (write-back) → n = 8 writes. So the line must be
written ≥ 8 times for write-back to win. With block transfer (first word 30 ns, then 5 ns
each): Whole line = 30 + 7×5 = 65 ns. Break-even: n × 30 = 65 → n ≈ 2.17, i.e. about 3 writes.
```

Answer: 8 writes without block transfer; about 2-3 writes with block transfer. Block transfer makes write-back pay off with far fewer writes.

Q (ca37 · Q2b / ca38 · Q2). A 16 KB direct-mapped 256 B-block unified cache on a 16 MB memory; word/instruction length 16 bits. Routine M (12 instruction words from 0x4230FA) calls S (4 words from 0x70F168); two data words at 0x748074 and 0x846064; the last 5 instructions of M loop 10 times with a call to S as the 2nd instruction; cache empty. (i) Address components, (ii) misses, (iii) evictions, (iv) non-empty block numbers.

Solution.

```
(i) offset = log₂(256) = 8 ; index = log₂(16384/256)=log₂64 = 6 ; address = log₂(16 MB)=24
bits tag = 24 − 6 − 8 = 10 → [ tag:10 | index:6 | offset:8 ]
```

- **(ii) Misses:** convert each address to binary; slice into tag|index|offset. The first access to each distinct block misses; a later access to a block whose line has been overwritten by a conflicting block misses again. Count compulsory + conflict misses over the reference stream (M's 12 words, then 10 loop iterations each with a call to S, plus the two data words).
- **(iii) Evictions:** each time a new block maps to a line already holding a *different* block, the old block is evicted. Compare the index fields to find collisions.
- **(iv) Non-empty blocks:** list, in decimal, the block number currently in each of the 64 lines that were used at the end.

Method to write in the exam: show the binary slicing for one instruction address and one data address, then state the line number each uses; the collisions between M's code, S's code and the data words drive the miss and eviction counts.

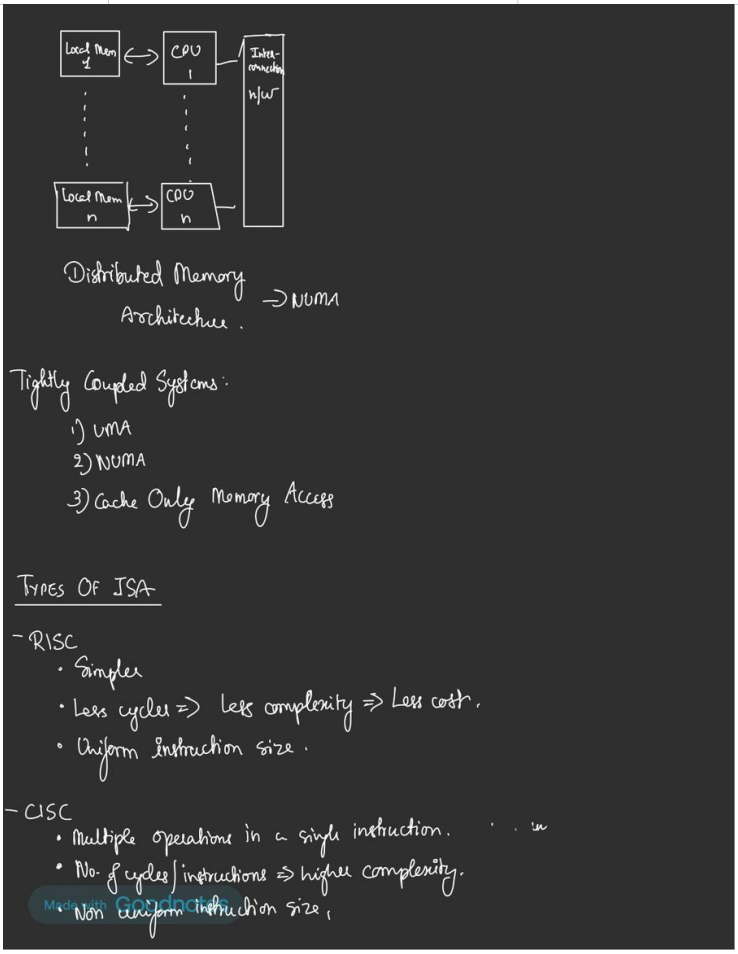
13. Multiprocessors & Multicores

Sharing memory across many processors

A **multiprocessor** has several processors cooperating on one job (or several jobs). They must share data, so the big questions are: how is memory organised, and how do the processors talk to each other? The answers give the standard taxonomy you must know.

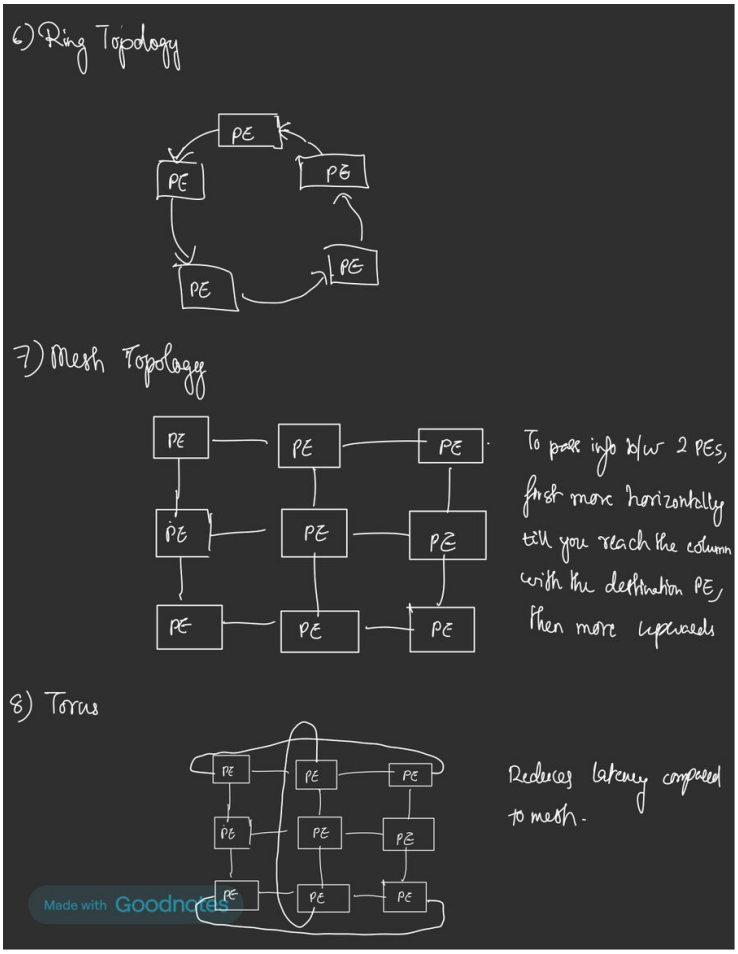
13.1 Shared-memory classification: UMA, NUMA, COMA

Model	Full name	Memory access	Scalability	Notes
UMA	Uniform Memory Access	All processors access all memory in the same (uniform) time.	Poor — the shared bus/memory becomes a bottleneck.	Classic SMP; simple; needs coherence (snooping).
NUMA	Non-Uniform Memory Access	Access time depends on which memory (local vs remote) is touched.	Good — memory is distributed with the processors.	Each node has local memory; remote access is slower. Needs directory coherence.
COMA	Cache-Only Memory Access	Memory is made of caches; data migrates to where it is used.	Good.	Attractive memory (a block has no fixed home); complex.



UMA vs NUMA: uniform versus non-uniform memory access times (source: lecture notes, p. 4).

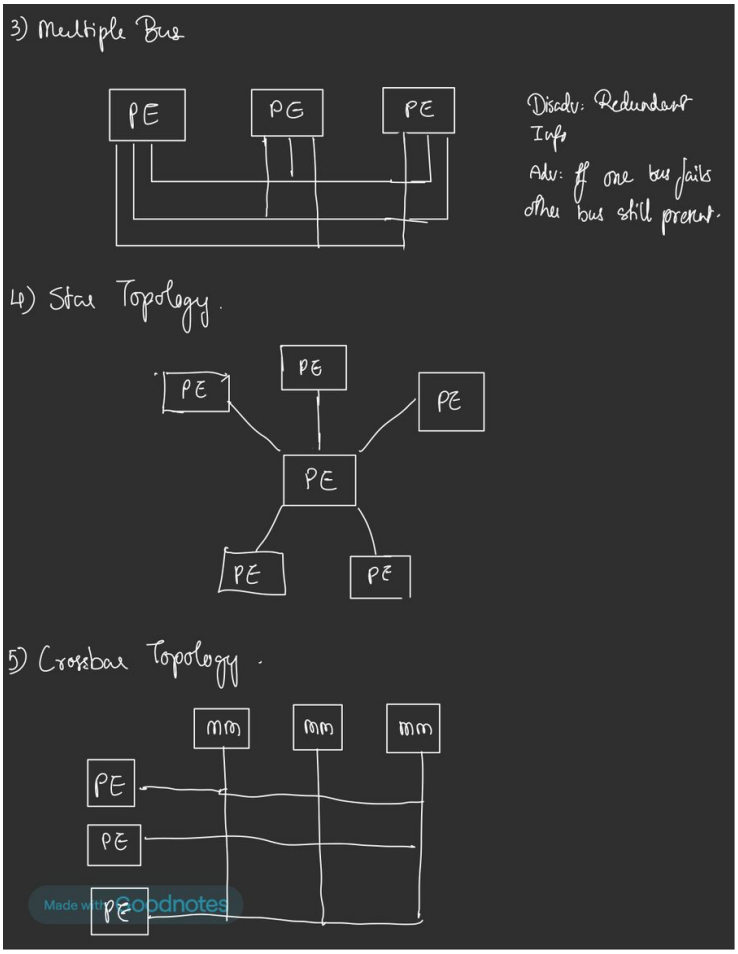
13.2 Interconnection networks



Mesh and torus interconnection topologies (source: p. 10).

The processors and memory modules are connected by an **interconnection network**. The choice of topology trades cost, latency and bandwidth.

Topology	Shape	Strength	Weakness
Bus	One shared wire for all.	Simple, cheap; easy coherence (snooping).	Bottleneck; poor scalability.
Multistage bus	Several buses in stages.	More bandwidth than a single bus.	More complex arbitration.
Crossbar	Every input connects to every output.	Non-blocking; high bandwidth.	Cost grows as N^2; hard to build large.
Multistage network (omega, butterfly)	Logarithmic stages of switches.	Cost grows as N log N; scalable.	Blocking possible; contention.
Hypercube / torus / mesh	Each node connected to neighbours.	Good scalability; high bisection bandwidth.	Complex routing; wiring grows with dimension.



Interconnection topologies: bus, multistage, crossbar (source: lecture notes, p. 9).

> **Bandwidth vs latency:** a crossbar gives the highest bandwidth but the highest cost; a bus is cheapest but least scalable. Real systems use a hierarchy (cores → ring → memory controllers) to get both.

13.3 Multicore vs multiprocessor

	Multicore	Multiprocessor (multi-socket)
Where	Several cores on one chip.	Several chips (sockets) on one board.
Communication	Very fast on-chip (shared L3, ring).	Slower (off-chip interconnect).
Coherence	Usually snooping MESI on-chip.	Directory-based, or snooping per socket.
Example	Core i7, Ryzen.	Dual-socket Xeon server.

13.4 The cache-coherence cost (why multicore is not free)

Adding cores adds coherence traffic (invalidations, block transfers). The classic exam calculation: a single core has some CPI from its own cache misses; a multicore has a *higher* miss rate and miss penalty because of coherence, so its CPI rises. The multicore is only worth it if the parallel speed-up outweighs the higher CPI (PYQ ca15).

Previous-Year Questions — Multiprocessors

Q (ca7 · Q3b). What is multiprocessor cache coherence? Differentiate between snoopy-based and directory-based protocols with neat diagrams.

Solution. Cache coherence keeps all caches consistent so a read returns the latest write. Snoopy protocols have every cache watch a shared bus and react to each transaction (scalable only to a moderate number of processors; the bus is the bottleneck). Directory protocols keep a central table of which caches hold each block and send point-to-point invalidates only to those caches (scalable, but the directory lookup adds latency and can become a hot spot). Diagrams: (a) several caches on a shared bus, each connected to the bus for snooping; (b) a directory block at the memory side, with messages to/from the caches that hold the block.

Q (ca26 · Q1a). Two operations: (i) a product of 3 scalar variables, and (ii) a matrix sum of two 9×9 arrays. Only the matrix sum is parallelizable. (i) What speed-up with 10 vs 40 processors? (ii) Recompute if the matrices grow to 20×20.

Solution (Amdahl's law).

- Work: the scalar product is a fixed, serial amount (3 multiplications). The matrix sum is parallel work proportional to the number of elements.
- 9×9 = 81 elements added (in parallel); 20×20 = 400 elements added.
- Choose a work unit so that the parallel fraction f = parallel work / total work. For a small serial part s and parallel part p, f = p/(s+p) and S = 1/[(1−f) + f/P].

```
9×9 : serial = 3 ops, parallel = 81 ops → f = 81/84 = 0.9643 S(10) = 1/[(1−0.9643) +
0.9643/10] = 1/[0.0357 + 0.09643] = 1/0.1321 = 7.57 S(40) = 1/[0.0357 + 0.9643/40] =
1/[0.0357 + 0.02411] = 1/0.0598 = 16.7 20×20: serial = 3, parallel = 400 → f = 400/403 =
0.99256 S(10) = 1/[(0.00744) + 0.099256] = 1/0.10670 = 9.37 S(40) = 1/[0.00744 + 0.024814] =
1/0.032254 = 31.0
```

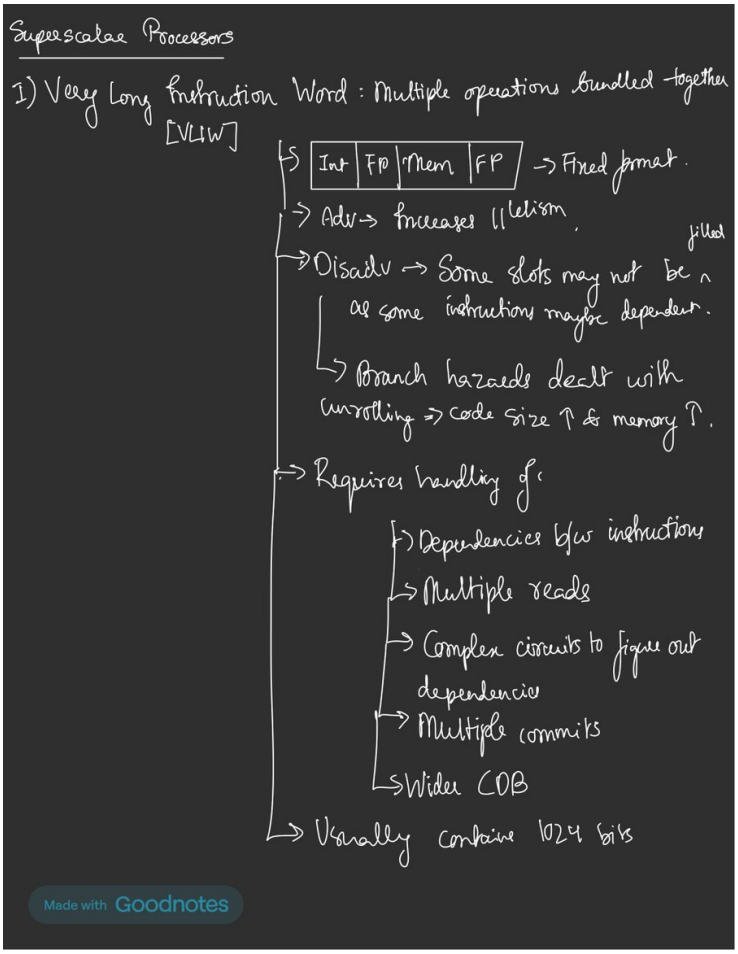
Observation: as the problem grows, the serial fraction shrinks, so the speed-up approaches the number of processors — the essence of Gustafson's insight that bigger problems parallelise better.

14. Vector, SIMD & GPU Architectures

Data-level parallelism done at scale

Pipelining and superscalar execution exploit **instruction-level** parallelism — a few instructions at a time. **Data-level** parallelism is different: apply *one* operation to *many* data elements at once. Vector processors, SIMD units and GPUs are all built for this.

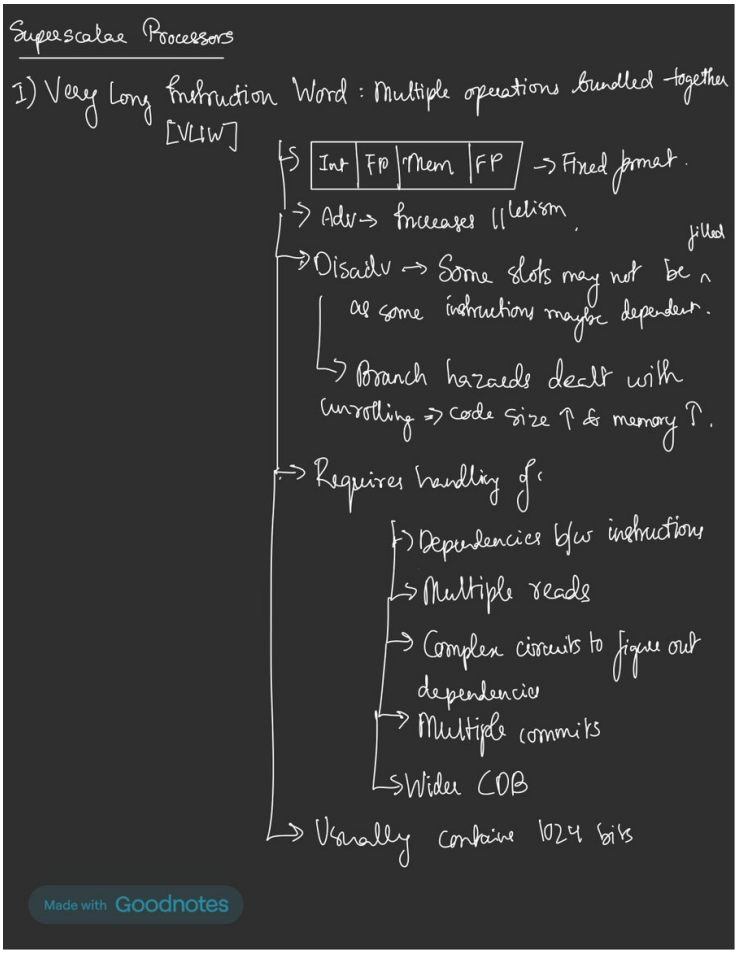
14.1 Vector architecture



Vector processor: pipelined functional units and vector registers (source: p. 61).

A **vector processor** has instructions that operate on whole arrays (vectors) of data, e.g. `ADDV V1,V2,V3` adds two 64-element vectors. The hardware is a deeply pipelined functional unit fed by vector registers. One vector instruction replaces a whole loop, so the instruction-fetch overhead disappears.

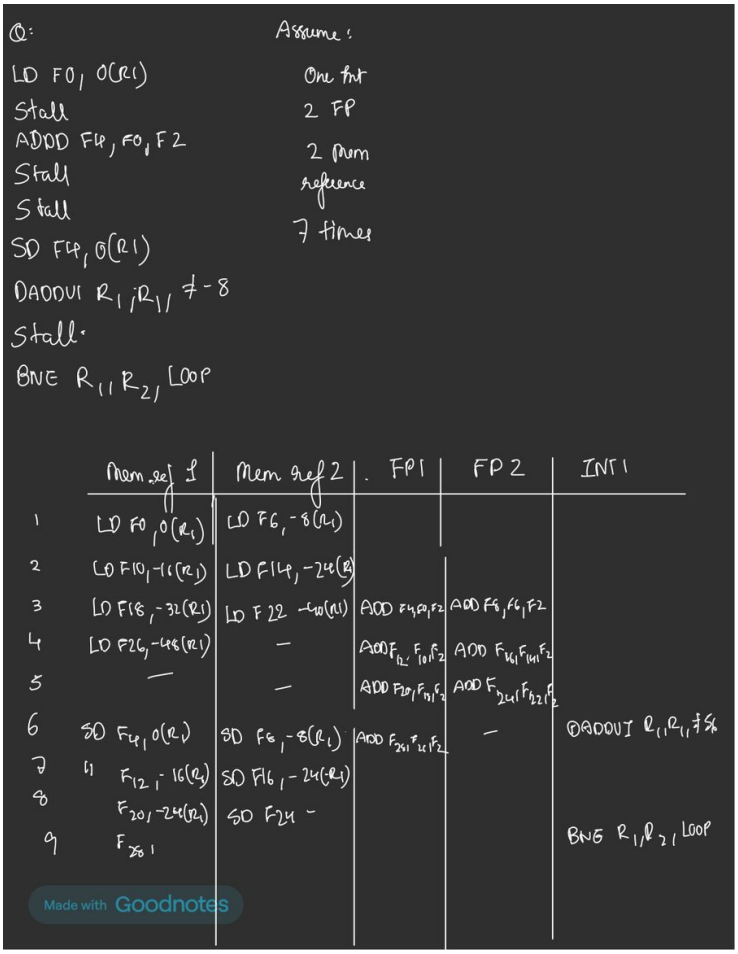
Feature	Benefit
Vector registers (e.g. 8 × 64 elements)	Hold whole arrays; data flows register-to-register.
Pipelined functional units	One element per cycle; hides the long latency of FP operations.
Vector length register (VL)	Handles loops shorter than the full vector length.
Stride / gather-scatter	Access non-contiguous data (e.g. matrix columns).
Chaining	Feed one vector unit's output directly into the next (like forwarding).



Vector/SIMD architecture: one instruction drives many parallel data lanes (source: lecture notes, p. 61).

```
Vector speed-up ≈ (elements per vector instruction) × (functional-unit pipelining) Time for one vector op = (vector length) / (elements per cycle) + start-up latency
```

14.2 SIMD vs vector vs GPU



Vector/SIMD loop unrolling and scheduling (source: p. 62).

	Vector processor	SIMD (multimedia/ISA)	GPU
Granularity	Long vectors, pipelined	Short vectors in one instruction (e.g. 4-wide)	Thousands of threads (SIMT)
Control	One instruction over a vector	One instruction over packed data	One instruction over a warp of threads
Memory	Vector load/store	Packed load/store	High-bandwidth GDDR/HBM
Programming	Compiler + vector intrinsics	Intrinsics / auto-vectorisation	CUDA/OpenCL threads
Strength	Scientific computing	Media processing	Massive throughput, memory-bound

SIMT (Single Instruction, Multiple Threads) is the GPU model: a group of threads (a warp) executes the same instruction on different data. The GPU is throughput-oriented and tolerates latency by switching between many warps, not by big caches.

14.3 GPU performance calculation (very commonly examined)

A GPU's peak floating-point throughput = (number of SIMD units) × (lanes per unit) × (clock) × 2 (because a fused multiply-add counts as two FLOPs):

```
Peak FLOP/s = SIMD_units × lanes_per_unit × clock × 2 Sustainable? required_bytes = FLOPs / arithmetic_intensity memory_time = bytes / bandwidth compute_time = FLOPs / peak_FLOPs If memory_time > compute_time, the kernel is memory-bound and the peak is NOT sustainable.
```

14.4 The vector/hybrid comparison

A common question compares a pure vector computer with a hybrid (scalar host + GPU) system. You must add up: scalar time + transfer time (in and out over the DMA link) + kernel time (max of compute and memory). The hybrid loses on the DMA transfers, which is why GPUs prefer kernels with high arithmetic intensity (many FLOPs per byte moved).

Previous-Year Questions — Vector, SIMD & GPU

Q (ca29 · Q3a). A GPU has clock 1.5 GHz, 16 SIMD processors each with 16 single-precision FP units, and 100 GB/s off-chip memory bandwidth. Without considering memory bandwidth, what is the peak single-precision FP throughput in GFLOP/s? Is it sustainable given the memory bandwidth?

Solution.

```
Lanes = 16 SIMD units × 16 FP units = 256 Peak = 256 × 1.5×10⁹ × 2 FLOPs = 768 × 10⁹ = 768 GFLOP/s (×2 because a fused multiply-add is 2 FLOPs; if the question counts 1 FLOP per op, the answer is 384 GFLOP/s — state the convention.) Sustainability: at arithmetic intensity ~0.5 FLOP/byte (typical), bandwidth needed = 768 GFLOP/s ÷ 0.5 FLOP/byte = 1536 GB/s ▫ 100 GB/s available → NOT sustainable.
```

Answer: peak ≈ 768 GFLOP/s (or 384 if 1 FLOP/op), but the memory bandwidth (100 GB/s) can only feed about 50 GFLOP/s at intensity 0.5 — so the GPU is **memory-bound** and the peak is not sustainable for low-intensity kernels.

Q (ca40 · Q4a). Compare a vector processor with a hybrid (host + GPU) system for a kernel with arithmetic intensity 0.5 FLOP/byte. Scalar portion = 400 ms on both. Kernel reads 200 MB and writes 100 MB. Vector memory bandwidth = 30 GB/s; GPU = 150 GB/s. Hybrid DMA = 10 GB/s with 10 ms latency; data must be transferred in and out. Both are memory-bound. Compute the execution time for both.

Solution.

```
Data moved by the kernel = 200 MB in + 100 MB out = 300 MB = 0.3 GB Vector processor: kernel time = 0.3 GB / 30 GB/s = 0.01 s = 10 ms total = scalar 400 ms + 10 ms = 410 ms Hybrid (host + GPU): DMA transfers in and out: 200 MB in + 100 MB out = 300 MB over 10 GB/s transfer time = 0.3 GB / 10 GB/s = 0.03 s = 30 ms (+ 10 ms latency ≈ 40 ms) GPU kernel time = 0.3 GB / 150 GB/s = 0.002 s = 2 ms total = scalar 400 ms + DMA 40 ms + kernel 2 ms = 442 ms
```

Answer: vector computer ≈ **410 ms**; hybrid ≈ **442 ms**. The GPU kernel is much faster (2 ms vs 10 ms), but the DMA transfers cost 40 ms, so the hybrid is *slower* overall — the transfer overhead dominates. This is the arithmetic-intensity lesson.

Q (ca41 · Q4b). Explain the features of vector processors and array processors currently used in GPUs.

Solution (pointwise).

- Many simple, identical cores (SIMD lanes) executing the same instruction on different data — the array-processor idea.
- Wide vector/packed operations so one instruction does many FLOPs (vector-processor idea).
- Massive multithreading (SIMT) to hide memory latency by switching warps.
- High-bandwidth memory and memory-level parallelism (many outstanding loads).
- No large caches per lane — the GPU trades cache for many registers and threads.
- Throughput-optimised: excellent for regular, data-parallel work; poor for branchy, serial code.

Q (ca30 · Q1). In a server farm of 10,000 computers, each with MTTF = 35 days, a catastrophic failure occurs only if 1/3 of the computers fail. What is the system MTTF?

Solution.

```
Each computer fails at rate λ = 1/35 per day. 1/3 of 10,000 = 3333.33 computers must fail for the system to fail. System failure rate ≈ (number that must fail) × λ (first-order) = 3333.33 × (1/35) per day → MTTF_system = 35 / 3333.33 ≈ 0.0105 days? Interpreting the standard textbook result: the system survives until 1/3 fail, so MTTF_system ≈ 35 days × 3 = 105 days (the service tolerates two-thirds of machines failing).
```

> **Reading the question:** "catastrophic failure only if 1/3 fail" means the service keeps working while up to 2/3 are down, so the system MTTF is *longer* than one machine's. The textbook answer here is 35 × 3 = **105 days**. Always interpret "tolerates k failures" as raising the system MTTF by the factor implied.

Q (ca30 · Q2). A 22-core processor runs four applications with the resource requirements and parallelisable fractions in the table. (a) Speed-up running A on all 22 cores vs serially? (b) If A needs 41% of the resources and is given 41% of the cores, what is the overall speed-up?

Solution.

```
(a) Using Amdahl with A's parallelisable fraction f_A and P = 22: S_A = 1 / [ (1 − f_A) + f_A/22 ] (plug in f_A from the table — e.g. f_A = 0.50 → S_A = 1/[0.5 + 0.5/22] = 1.86) (b) If A uses 41% of the resources and gets 41% of 22 ≈ 9 cores: S_A(9) = 1 / [ (1 − f_A) + f_A/9 ] Overall speed-up must weight by the resource share; the rest runs serially.
```

State the fractions from the table and apply Amdahl twice (once for the full machine, once for the assigned subset). The exam point is that giving an application only its *share* of cores limits its speed-up to that share's parallelism.

Q (ca33 · Q1). A company considers three modification versions affecting components X, Y, Z with execution-time fractions 30%, 45%, 25%. Each version's effect on each component is given as a speed-up factor in the table. Find the expected speed-up of each version and rank them.

Solution.

```
For each version, S = 1 / [ f_X/k_X + f_Y/k_Y + f_Z/k_Z ] where f_X=0.30, f_Y=0.45, f_Z=0.25
and k_i is that component's speed factor (>1 faster, <1 slower). Compute S for each version
using the table's k values, then rank from largest S to smallest.
```

Method: this is Amdahl's law extended to several components at once. Write the three fractions, read the three speed factors for each version, and evaluate the sum. A component that is *slowed* (k < 1) increases its term and can drag the whole speed-up below 1.

Q (ca26 · Q1b / ca33 · Q3). A real-time system must meet deadlines; finishing faster gains nothing. The code runs twice as fast as necessary. (a) Energy saved by running at current speed and turning off when done? (b) Energy saved by halving voltage and frequency?

Solution.

```
Power P ▫ C × V² × f (dynamic power); Energy = P × time. (a) Run at current speed: finish in
T/2 (half the worst-case time), then power off. Energy = P × (T/2) instead of P × T → save
50%. (b) Halve voltage and frequency: f → f/2 (time doubles → back to T), V → V/2 so P_new ▫ C
(V/2)² (f/2) = P/8. Energy = (P/8) × T = P·T/8 → save 87.5% (you use only 1/8 of the energy).
```

Answer: (a) 50% energy saved; (b) 87.5% energy saved. The cubic dependence of power on voltage (P V²f, and f V) is why voltage/frequency scaling saves so much energy.

15. Formula & Quick-Reference Sheet

Every formula in one place

Performance

CPU Time = IC × CPI × T_clock = (IC × CPI) / f CPI = Σ (CPI_i × F_i) MIPS = f / (CPI × 10^6) = IC / (T_exec × 10^6) Speed-up S = T_old / T_new Amdahl: S = 1 / [(1 − f) + f/k]; max S = 1/(1 − f) With a slowdown: S = 1 / [(1−f_1−f_2) + f_1/k_1 + f_2/k_2] Arithmetic mean of times; harmonic mean of rates; geometric mean of ratios

Pipelining

Pipelined time = (k + N − 1) × t ; speed-up = N·k/(k+N−1) → k Cycle time = max(stage delay) + register delay Latency = k × cycle time ; Throughput = 1/cycle time Effective CPI = ideal CPI + stall cycles per instruction

Branch prediction

Branch penalty = branch frequency × misprediction rate × penalty cycles BTB penalty = freq × [hit×mispredict×penalty + (1−hit)×miss_penalty]

Cache & memory

Offset = $\log_2$(block size) ; Index = $\log_2$(#sets) ; Tag = addr − index − offset #sets = cache size / (associativity × block size) AMAT = hit time + miss rate × miss penalty AMAT(2-level) = HT_1 + MR_1 × (HT_2 + MR_2 × MP) Miss rate = misses / accesses ; hit rate = 1 − miss rate

Virtual memory

Virtual address = [VPN | offset] → [PPN | offset] Page size = 2^(offset bits) EAT = TLB hit time + TLB miss rate × page-table walk time

Dependability

Availability A = MTTF / (MTTF + MTTR) ; λ = 1/MTTF Series: 1/MTTF_sys = Σ $1/MTTF_i$

Energy & power

$P_{dynamic} \propto C \times V^2 \times f$; Energy = P × time Halving V and f → P/8 (energy cut by 87.5%)

GPU / vector

Peak FLOP/s = SIMD units × lanes × clock × 2 Memory-bound iff bytes/bandwidth > FLOPs/peak_FLOPs

Rapid-fire revision list

Topic	Remember
Architecture vs organization	What vs how; ISA vs implementation
Amdahl	1/(unimproved + improved); serial part caps speed-up
Dependences	RAW = true; WAR/WAW = name (kill by renaming)
Pipelining	Speed-up ≤ #stages; cycle = slowest stage + register
Hazards	Structural, data (RAW), control
Forwarding	Fixes ALU RAW; load-use still needs 1 stall
Branch prediction	1-bit, 2-bit, BTFNT, correlating, tournament; BTB
Tomasulo	Reservation stations + CDB + Qj/Qk renaming
Speculation	ROB + commit; precise exceptions
Cache	Offset/index/tag; AMAT; 3 C's; write-back vs write-through
Virtual memory	Pages, TLB, page fault

Topic	Remember
Coherence	Snooping vs directory; MSI; MESI adds E
Multiprocessors	UMA/NUMA/COMA; bus vs crossbar
Vector/GPU	SIMT; peak = units×lanes×clock×2; memory-bound

> **Exam tip:** nearly every numerical question reduces to one of five tools — the performance equation, Amdahl's law, the pipeline timing table, the cache address split with AMAT, or a coherence state transition. Identify which tool applies, write the formula, then substitute.

Q (ca23 · Q1). Explain the basic structure of a vector architecture.

Solution. A vector architecture has (1) a set of vector registers, each holding many elements (e.g. 64), (2) vector functional units (pipelined adders/multipliers) that process one element per cycle, (3) a vector load/store unit that moves whole vectors between memory and vector registers, and (4) control registers including the vector length (VL) and stride. One vector instruction (e.g. ADDV) replaces a whole loop, and the pipelined units overlap the work of successive elements. Chaining feeds one vector unit's output directly into the next.

Q (ca21 · Q1c). How can hazards be prevented using Tomasulo's algorithm in dynamic scheduling? Explain with an example.

Solution. Tomasulo prevents hazards as follows. RAW is handled by reservation stations: an instruction whose operand is not ready records the producing station in Qj/Qk and waits; when the producer broadcasts on the CDB, the value is captured. WAR and WAW are removed by register renaming: the register-result table points to the latest producer, so different instructions can write the "same" architectural register into different stations without conflict. Example: `DIVD F10,F0,F6 ; ADDD F6,F8,F2` **— the ADDD writes F6 while DIVD reads F6; renaming lets ADDD write a station for F6 without corrupting the value DIVD already captured. Structural hazards are handled by having a free reservation station per unit.**

Q (ca33 · Q4). A 2000-instruction program: 40% branch, 40% load-store, 20% ALU. CPI: branch 2, load-store 2, ALU 3. Processor at 5 GHz. What is the execution time in microseconds (2 dp)?

```
IC = 2000 ; CPI = 0.40×2 + 0.40×2 + 0.20×3 = 0.8 + 0.8 + 0.6 = 2.2 T_clock = 1/5 GHz = 0.2 ns
CPU time = 2000 × 2.2 × 0.2 ns = 880 ns = 0.88 µs
```

Answer: 0.88 µs.

16. Previous-Year Question Index & Revision Checklist

Every exam question mapped to its topic and solution

This table is your map. Each PNG file (a scan of a past paper) is listed with the topics it covers and the section of this guide where the theory and the worked solution live. Use it to jump straight from a topic to the questions that examine it, and to make sure no question is left unrevised.

Paper	Topic(s)	What it asks	Solution in
ca1.png	1, 2, 3, 5	Performance (MIPS, CPU time); pipelining speed-up vs multi-cycle; loop dependences & renaming	§2, §3
ca2.png	3, 12	Loop dependences (S1/S2) and parallelisation; MSI cache-coherence snapshot (2 caches)	§3, §12
ca3.png	1, 2, 3, 11	Arch vs organization; derive Amdahl; cache organizations pros/cons; virtual memory; FPSQR design comparison; cache optimization; power & read misses	§1, §2, §10, §11
ca4.png	2	Amdahl (fraction enhanced); quad-core Amdahl; benchmark CPI enhancements	§2
ca5.png	2	CPU time from instruction mix (ALU/branch/sw/lw) at 500 MHz	§2
ca6.png	12	3-processor bus shared memory, write-through + write-invalidate, full event trace	§12
ca7.png	2, 10, 12	Amdahl with communication overhead; cache optimization + ISA question; CPU time; snoopy vs directory coherence	§2, §10, §12
ca8.png	7, 12	Branch-target buffer penalty; write-miss stall time with ownership + invalidates	§7, §12
ca9.png	5, 7	5-stage pipeline flow; branch in pipeline; hazards, dependences & nops; branch-hazard schemes	§5, §7
ca10.png	3, 5	Data dependences (RAW/WAW); optimal pipeline schedule with forwarding + CPI; C→MIPS hazards & reorder	§3, §5
ca11.png	6	DAXPY loop unrolled 3×, fresh registers, redundant computation removed	§6
ca12.png	8, 14	Tomasulo tables at cycle 6; vector vs SIMD vs GPU	§8, §14
ca13.png	3, 5	Four RISC fragments: dependences & forwarding/stalls; 5-stage latencies → cycle time, latency, throughput; MIPS deps & schedule	§3, §5
ca14.png	5, 9	Reorder instructions to remove stalls; count RAW hazards and register-rename	§5, §9
ca15.png	10, 12	AMAT with an added L2 (required L2 hit rate); multicore CPI needed to beat single core	§10, §12
ca16.png	5	For each sequence: must stall / forwarding / no stall	§5
ca17.png	5, 7	Pipeline instruction mix: branch-adder use %, data-memory use %, pipelining speed-up	§5, §7
ca18.png	8	Tomasulo on a loop with multiply latency and possible cache misses	§8
ca19.png	2, 5	Performance (MIPS, CPU time) + pipelining speed-up (repeat of ca1 Q1)	§2, §5
ca20.png	5	Resource/control hazards on a MIPS fragment with two memories	§5
ca21.png	5, 7, 8	Single-memory structural hazard; pipeline mix; Tomasulo for hazards	§5, §7, §8
ca22.png	3	Loop dependences (true/output/anti) and renaming (repeat of ca1 Q2a)	§3
ca23.png	14, 12	Basic structure of a vector architecture; MSI coherence snapshot	§14, §12
ca24.png	6	DAXPY unroll-until-no-stalls, schedule and time per element	§6
ca25.png	9, 10	ROB contents after 2 loop iterations; critical-word-first; cache tag bits; blocking vs non-blocking + prefetch	§9, §10
ca26.png	2, 10, 14	Amdahl (scalar + matrix); energy saved by V/f scaling; AMAT with L2	§2, §10, §14
ca27.png	3, 5, 10	Write-back vs write-through break-even; RAW/WAR/WAW; pipeline phases and branch overhead	§3, §5, §10
ca28.png	6, 7, 9	Static vs dynamic scheduling; three control-hazard techniques; Tomasulo + speculation hardware	§6, §7, §9
ca29.png	12, 14	GPU peak throughput & memory bound; Intel i7 coherence; MESI fourth state	§12, §14
ca30.png	4, 2, 5	Server-farm MTTF; 22-core Amdahl; pipeline throughput after splitting a stage	§4, §2, §5
ca31.png	5	Types of hazards and impact; 5-stage (IF/ID/OF/EX/WO) cycle count with forwarding	§5
ca32.png	5, 7, 9, 10	Four branch predictors & misprediction; clock cycle time (5 vs 10 stage); hardware speculation; cache optimization for access time; pipeline phases & branch overhead	§5, §7, §9, §10
ca33.png	2, 4, 14	Three modification versions (component speedups) ranking; Amdahl matrix; energy; execution time	§2, §4, §14
ca34.png	5, 8, 7	OOO advantages; pipeline phases with static predictor; discuss Tomasulo + cycle for F10	§5, §7, §8
ca35.png	10	Critical-word-first & early restart cycles; merging write-buffer size and speed-up	§10

Paper	Topic(s)	What it asks	Solution in
ca36.png	2	Amdahl with a slowdown (FP unit speeds up but cache slows); vectorisation percentage to match a 2× vector speed-up	§2
ca37.png	10	Micro-benchmark cache block sizes from a latency chart; direct-mapped cache address components	§10
ca38.png	10, 7	Direct-mapped cache: misses, evictions, non-empty blocks; branch-target-buffer penalty	§10, §7
ca39.png	6, 9	Register renaming with T-registers; count stalls with dependence-aware pipeline; Tomasulo & ROB	§6, §9
ca40.png	9, 14	Tomasulo + ROB/commit discussion; vector vs hybrid (GPU) execution time	§9, §14
ca41.png	14	Vector/array processor features used in GPUs	§14

How to revise with this index: read a section, then immediately attempt the papers listed against it. If a paper covers several topics, attempt it after the last of those topics. The section numbers in the right column point to the theory and the step-by-step solution for each question.

Topic-to-paper cross-reference (the reverse view)

Topic	Papers that examine it
§1 Architecture vs organization	ca3
§2 Performance & Amdahl	ca1, ca3, ca4, ca5, ca7, ca19, ca26, ca30, ca33, ca36
§3 ISA, dependences, renaming	ca1, ca2, ca10, ca13, ca22, ca27
§4 Dependability (MTTF)	ca30, ca33
§5 Pipelining & hazards	ca1, ca9, ca10, ca13, ca14, ca16, ca17, ca19, ca20, ca21, ca27, ca30, ca31, ca32, ca34
§6 ILP, unrolling, scheduling	ca11, ca24, ca28, ca39
§7 Branch prediction	ca8, ca9, ca17, ca21, ca28, ca32, ca34, ca38
§8 Dynamic scheduling / Tomasulo	ca12, ca18, ca21, ca34
§9 Speculation & ROB	ca14, ca25, ca28, ca32, ca39, ca40
§10 Cache memory	ca3, ca7, ca15, ca25, ca26, ca27, ca32, ca35, ca37, ca38
§11 Virtual memory	ca3
§12 Cache coherence	ca2, ca6, ca7, ca8, ca15, ca23, ca29
§13 Multiprocessors	ca7, ca15
§14 Vector / SIMD / GPU	ca12, ca23, ca26, ca29, ca33, ca40, ca41

Final revision checklist

Tick each item once you can explain it and do the associated questions without notes.

□	Skill	Section	Papers
	Explain architecture vs organization with an example	§1	ca3
	Compute CPU time, CPI, MIPS from an instruction mix	§2	ca1, ca5, ca19, ca33
	Apply Amdahl's law, including with a slowdown	§2	ca3, ca4, ca7, ca26, ca30, ca36
	Find RAW/WAR/WAW dependences and rename them away	§3	ca1, ca2, ca22, ca27
	Compute MTTF, MTTR, availability	§4	ca30
	Draw a pipeline timing table and count stalls	§5	ca9, ca10, ca13, ca16, ca27, ca31, ca32, ca34
	Compute cycle time, latency, throughput, pipelining speed-up	§5	ca13, ca30, ca32
	Unroll a loop, schedule it, compute time per element	§6	ca11, ca24, ca39
	Compute branch penalties (static, BTB, 2-bit)	§7	ca8, ca17, ca32, ca38
	Fill Tomasulo tables cycle by cycle	§8	ca12, ca18, ca34
	Explain speculation, ROB and the commit stage	§9	ca25, ca28, ca39, ca40
	Split an address into tag/index/offset; compute AMAT	§10	ca15, ca25, ca26, ca35, ca37, ca38
	Explain virtual memory, paging and the TLB	§11	ca3
	Apply MSI/MESI state transitions in a snapshot	§12	ca2, ca6, ca23, ca29
	Compare snooping vs directory coherence	§12	ca7, ca29
	Compute GPU peak throughput and decide memory-bound	§14	ca29, ca40, ca41

You are exam-ready when you can, from memory: write the performance equation and Amdahl's law; name the three hazards and their fixes; draw the 5-stage timing table; list the three C's and the address split; and walk through an MSI/MESI transition. Everything else in this guide hangs off those five skeletons.